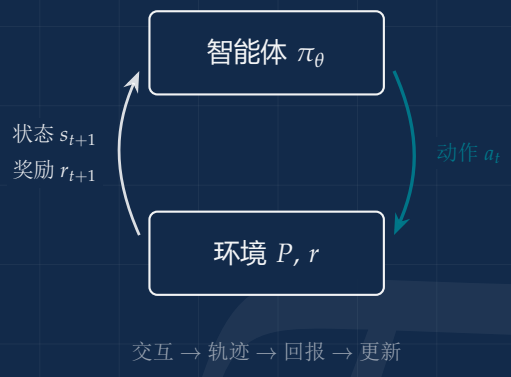
RESEARCH MONOGRAPH ▪ 2026

强化学习

从试错学习到推理智能

历史脉络 · 数学推导 · 算法谱系 · 应用版图 · 前沿进展

从Bellman 方程到策略梯度，从DQN 到RLHF/RLVR：一份以严格推导为主线、以证据分级为准绳的强化学习全景研究。



2026 年10 月

Prepared with XeLaTeX ▪ v1.0

摘要

强化学习（reinforcement learning, RL）研究的是这样一个问题：一个处于未知环境中的决策者，如何仅凭与环境的交互所得的经验信号，逐步改进自己的行为，使长期累积回报最大化。它既是最优控制与动态规划在统计学习条件下的延续，也是心理学中“试错—强化”思想的可计算化，更是当代大语言模型推理能力与智能体行为的核心训练范式。本报告以“问题—数学—算法—应用—边界”为主线，给出一份兼顾历史纵深、推导严谨性与工程证据的强化学习全景研究。

本报告的主要工作与结论如下。

1. **诞生的必然性。**强化学习并非某一篇论文的突然发明，而是三条思想脉络在1950—1980 年代的交汇：以Bellman 动态规划为代表的最优控制传统，以Thorndike 效果律、Pavlov 条件反射和神经科学多巴胺信号为代表的试错学习传统，以及以Samuel 跳棋程序、Minsky 的SNARC 为代表的机器自主学习传统。马尔可夫决策过程（MDP）把“延迟回报下的序贯决策”压缩为一个可分析的数学对象，使“信用分配”从哲学问题变成可计算的期望问题。
2. **算法史的五次范式转移。**本报告将发展历程概括为：动态规划与表格型TD/Q-learning（1957—1992）解决“有模型/无模型下的收敛性”；函数逼近与策略梯度（1992—2012）解决“连续状态与连续动作”；深度强化学习（2013—2017）解决“高维感知输入”；信任域、最大熵、模型基与离线学习（2015—2021）解决“稳定性、样本效率与数据复用”；人类反馈与可验证奖励驱动的LLM 强化学习（2017—2026）解决“不可微、难定义之目标”的优化问题。每一次转移都不是对前一范式的否定，而是把新的表示、新的数据来源或新的约束嵌入同一个Bellman/策略梯度骨架。
3. **数学核心可以被一条主线串起。**贝尔曼期望算子与最优算子在无穷范数下均为压缩映射，因而值函数唯一且值迭代几何收敛；策略梯度定理把“对策略参数的导数”化为“对动作对数概率的加权期望”，使不可微的回报得以通过似然比求导；信任域方法用KL 散度约束把策略改进转化为单调改进下界；RLHF 把人类偏好经Bradley–Terry 模型转成标量奖励，再以KL 正则的RL 目标求解；DPO 进一步消去显式奖励模型，得到闭式偏好损失；GRPO 以组内相对优势替代价值网络，显著降低LLM 推理任务的内存开销。本报告对这些结论给出从假设到结论的完整推导，而非仅罗列公式。
4. **应用效果必须分层评估。**在规则完备、仿真廉价、奖励可验证的领域——围棋、Atari、星际争霸、赛车、芯片布局、矩阵乘法与排序算法发现、数据中心冷却、托卡马克等离子体控制——强化学习已经取得可复核的、往往超越人类专家或传统启发式的效果。在推荐、广告、金融、医疗等“奖励延迟、反事实不可观测、试错代价高”的领域，强化学习更多

以离线评估、约束优化或人机协同的方式产生增量价值，公开的端到端因果效应证据仍然有限。在大语言模型领域，RLHF 已成为对齐流程的标准组件，RLVR（可验证奖励强化学习）在数学、代码与STEM 推理上展现出显著的测试时扩展能力，但其“是否真正提升了通用推理”仍存在争议。

5. **当前瓶颈从“能不能学”转向“奖励是否可信、环境是否真实、评估是否有效”。**奖励劫持（reward hacking）、分布偏移、样本效率、长程信用分配、以及基准污染与评估饱和，是2026 年最需要正视的问题。未来一段时间的突破，可能更多来自“可交互环境+ 可验证奖励+ 世界模型”的基础设施，而非单一损失函数的修改。

在方法上，本报告以原始论文、会议/期刊版本与官方技术报告为第一手依据，辅以权威机构博客与基准榜单；对无法独立复核的产业数据，明确标注来源与证据等级。全文公式采用标准数学排版，所有下标、上标、期望与算子均按其在概率图模型中的语义书写。附录给出符号表、算法速查表与完整参考文献，便于作为课程讲义或研究手册使用。

关键词：强化学习；马尔可夫决策过程；贝尔曼方程；策略梯度；深度强化学习；人类反馈强化学习；可验证奖励；大语言模型；智能体

证据分级： A（同行评审论文/官方技术报告，可复核）；B（权威机构博客/官方文档，数据具体但缺独立复现）；C（产业案例/媒体报道，方向可信、效应量需谨慎）；D（本报告的推断与评论）。

目　录

摘要 ii

研究导读：如何阅读这份报告 xii

第1章　强化学习诞生的背景 1

1.1 一个思想史的提问 1

1.2 三条思想源流 1

1.2.1 源流一：最优控制与动态规划 2

1.2.2 源流二：心理学中的效果律与神经科学中的奖励预测误差 2

1.2.3 源流三：早期人工智能中的机器自主学习 3

1.3 问题形式化：智能体与环境的交互环路 3

1.4 为什么是马尔可夫决策过程？ 4

1.5 与相邻学科的边界 5

1.6 历史必然性与偶然性 6

1.7 本章小结 6

第2章　数学基础：马尔可夫决策过程与贝尔曼算子 7

2.1 马尔可夫决策过程 7

2.1.1 占用度量与目标函数 8

2.2 贝尔曼方程 9

2.2.1 策略评估的贝尔曼方程 9

2.2.2 最优贝尔曼方程 10

2.3 贝尔曼算子与压缩映射 10

2.4 策略迭代与值迭代 12

2.4.1 策略改进定理 12

2.4.2 策略迭代 13

2.4.3 值迭代 14

2.5 探索的数学：从老虎机到UCB 15

2.6 函数逼近与“死亡三角” 15

2.6.1 从表格到函数逼近 16

2.6.2 半梯度TD 与投影 16

2.6.3 死亡三角 16

2.6.4 深度RL 的工程补救 16

2.7 两个重要扩展 ····· 17
2.7.1 部分可观测马尔可夫决策过程 ····· 17
2.7.2 平均奖励准则 ····· 17
2.8 本章小结 ····· 18

第3章 发展历程：五次范式转移与关键算法 19

3.1 为什么要用“范式转移”而不是“年表” ····· 19
3.2 史前史与奠基（1930s—1957） ····· 19
3.3 第一范式：表格型时序差分与收敛性（1957—1989） ····· 20
3.3.1 从动态规划到时序差分 ····· 20
3.3.2 Q-learning 与离策略控制 ····· 20
3.3.3 Actor–Critic 与REINFORCE ····· 21
3.3.4 TD-Gammon：第一次“深度”强化学习 ····· 21
3.4 第二范式：函数逼近与策略梯度（1989—2012） ····· 21
3.4.1 函数逼近的理论困难 ····· 21
3.4.2 策略梯度定理与自然梯度 ····· 22
3.4.3 探索、选项与搜索 ····· 22
3.4.4 第二范式的遗产与局限 ····· 22
3.5 第三范式：深度强化学习与稳定性（2013—2017） ····· 23
3.5.1 DQN：经验回放与目标网络 ····· 23
3.5.2 从A3C 到PPO：策略优化的工程化 ····· 23
3.5.3 AlphaGo 与AlphaZero：搜索与学习的结合 ····· 24
3.6 第四范式：模型基、离线学习与对齐（2017—2021） ····· 24
3.6.1 模型基RL 与世界模型 ····· 24
3.6.2 离线RL：从历史数据中学习 ····· 25
3.6.3 分布型RL 与最大熵RL ····· 25
3.6.4 RLHF：把人类偏好变成奖励 ····· 25
3.7 第五范式：偏好、可验证奖励与推理智能（2022—2026） ····· 26
3.7.1 RLHF 的规模化与DPO ····· 26
3.7.2 GRPO 与RLVR ····· 26
3.7.3 推理模型与测试时扩展 ····· 27
3.7.4 Agentic RL：从单步生成到长程决策 ····· 27
3.8 算法进步的五个机制 ····· 27
3.9 里程碑年表 ····· 28
3.10 本章小结 ····· 30

第4章 值函数方法的严格推导：从TD 到分布式RL 31

4.1 TD(0)：贝尔曼期望方程的随机逼近 ····· 31
4.1.1 从贝尔曼方程到更新规则 ····· 31
4.1.2 它优化的是什么目标？ ····· 32
4.1.3 偏差与方差：TD 与蒙特卡洛的对偶 ····· 33
4.2 TD(λ) 与资格迹 ····· 33
4.2.1 前向视角：λ-回报 ····· 33
4.2.2 后向视角：资格迹 ····· 33
4.3 SARSA：在策略控制 ····· 34
4.4 Q-learning：离策略最优控制 ····· 35
4.4.1 算法与算子 ····· 35
4.4.2 最大化偏差 ····· 35
4.4.3 Double Q-learning ····· 35
4.5 DQN：深度值函数逼近 ····· 36
4.5.1 损失与梯度 ····· 36
4.5.2 经验回放为什么有效？ ····· 36
4.5.3 Double DQN ····· 37
4.5.4 优先经验回放与重要性采样 ····· 37
4.5.5 Dueling 与噪声网络 ····· 37
4.6 分布式RL：从期望到分布 ····· 38
4.6.1 为什么需要分布？ ····· 38
4.6.2 C51：分类分布与投影 ····· 38
4.6.3 QR-DQN：分位数回归 ····· 39
4.7 本章小结 ····· 39

第5章 策略优化方法的严格推导：从策略梯度到SAC 40

5.1 策略梯度定理 ····· 40
5.1.1 问题设定与轨迹似然 ····· 40
5.1.2 主定理 ····· 41
5.2 REINFORCE 与基线 ····· 42
5.2.1 REINFORCE 估计量 ····· 42
5.2.2 基线为什么不改变期望？ ····· 43
5.2.3 优势函数与actor–critic ····· 43
5.3 广义优势估计（GAE） ····· 44
5.3.1 从 n 步优势到指数加权 ····· 44
5.4 自然梯度与TRPO ····· 45
5.4.1 策略改进下界 ····· 45

5.4.2 从下界到信赖域 47
5.5 PPO：用裁剪替代约束 47
5.5.1 裁剪目标 47
5.5.2 PPO 的完整损失 48
5.6 确定性策略梯度：DPG 与DDPG 48
5.6.1 DPG 定理 49
5.6.2 DDPG 的实现 49
5.7 最大熵RL：SAC 50
5.7.1 最大熵目标 50
5.7.2 软贝尔曼方程 50
5.7.3 软策略迭代 51
5.8 算法谱系与选择 51
5.9 本章小结 52
第6章 模型基、搜索与离线强化学习 53
6.1 模型基强化学习 53
6.1.1 基本框架 53
6.1.2 Dyna：真实经验与模拟经验的混合 53
6.1.3 世界模型：Dreamer 与隐空间想象 54
6.1.4 MuZero：学习“足够用于规划”的模型 55
6.2 搜索：从UCT 到AlphaZero 55
6.2.1 蒙特卡洛树搜索与UCT 55
6.2.2 AlphaGo：先验、价值与搜索 56
6.2.3 AlphaZero 的目标与训练 56
6.2.4 从AlphaZero 到推理模型 56
6.3 离线强化学习 57
6.3.1 问题设定与分布偏移 57
6.3.2 理论直觉：集中度系数 57
6.3.3 BCQ：限制动作支持集 57
6.3.4 CQL：保守Q 学习 58
6.3.5 IQL：隐式Q 学习 58
6.3.6 Decision Transformer：把RL 变成序列建模 59
6.4 三条路线的比较 59
6.5 本章小结 60
第7章 从人类反馈到可验证奖励：LLM 时代的强化学习 61
7.1 RLHF 的问题设定 61

7.1.1 从“给奖励”到“学奖励” 61
7.1.2 KL 正则目标的闭式最优策略 61
7.2 奖励建模：Bradley–Terry 模型 63
7.2.1 偏好概率模型 63
7.2.2 奖励模型的偏差来源 63
7.3 PPO-RLHF：把语言生成当作序列决策 63
7.3.1 Token 级MDP 64
7.3.2 KL 塑形的奖励 64
7.3.3 PPO 目标与实现 64
7.4 DPO：消去奖励模型的闭式偏好优化 65
7.4.1 从最优策略反解奖励 65
7.4.2 DPO 的梯度 65
7.4.3 DPO 的局限 66
7.5 GRPO：组内相对优势与无critic 优化 66
7.5.1 动机：critic 太贵 66
7.5.2 目标函数 66
7.5.3 为什么组内均值是合理的基线？ 67
7.5.4 GRPO 与PPO 的对比 67
7.6 RLVR：可验证奖励强化学习 67
7.6.1 奖励的结构性变化 68
7.6.2 DeepSeek-R1：纯RL 激励推理 68
7.6.3 “RLVR 是否真的提升了推理?” 69
7.6.4 奖励劫持的新形态 69
7.7 智能体强化学习：从单步生成到长程决策 69
7.7.1 POMDP 形式化 69
7.7.2 信用分配的三个层次 70
7.7.3 环境与奖励的基础设施 70
7.8 本章小结 70
第8章 应用版图与效果评估：从超人类游戏到产业系统 72
8.1 如何判断“强化学习有效”？ 72
8.1.1 三个层次的证据 72
8.1.2 效应量的四种陷阱 72
8.1.3 一个可操作的评估清单 73
8.1.4 应用场景的六种结构 73
8.2 游戏与博弈：强化学习的“望远镜” 74

8.2.1 从TD-Gammon 到Atari ····· 74
8.2.2 AlphaGo、AlphaZero 与AlphaStar ····· 74
8.2.3 赛车、算法发现与芯片布局 ····· 74
8.3 机器人与具身智能：从仿真到现实 ····· 75
8.3.1 为什么机器人是RL 的“试金石” ····· 75
8.3.2 腿式locomotion ····· 75
8.3.3 操作与通用机器人 ····· 76
8.3.4 效果与边界 ····· 76
8.4 大语言模型与智能体：从对齐到推理 ····· 76
8.4.1 RLHF 的效果 ····· 76
8.4.2 RLVR 与推理模型 ····· 77
8.4.3 智能体 ····· 77
8.5 推荐、广告与营销：延迟回报的用武之地 ····· 77
8.5.1 为什么推荐适合RL ····· 77
8.5.2 效果与证据 ····· 78
8.5.3 反馈回路与“推荐茧房” ····· 78
8.6 芯片、数据中心与系统：可验证的工程收益 ····· 78
8.6.1 芯片布局 ····· 78
8.6.2 数据中心冷却 ····· 78
8.6.3 网络与视频流 ····· 79
8.6.4 调度与系统 ····· 79
8.7 能源、交通、医疗与科学发现 ····· 79
8.7.1 核聚变等离子体控制 ····· 79
8.7.2 电网与能源管理 ····· 79
8.7.3 交通信号与自动驾驶 ····· 79
8.7.4 医疗 ····· 79
8.7.5 科学发现 ····· 80
8.8 工业、金融与供应链 ····· 80
8.8.1 工业控制 ····· 80
8.8.2 金融 ····· 80
8.8.3 供应链与库存 ····· 80
8.9 效果汇总与证据分级 ····· 81
8.10 效应量的外推边界 ····· 82
8.10.1 从“基准”到“部署”的四个断层 ····· 82
8.10.2 “超人类”不等于“可部署” ····· 82
8.10.3 规模与成本 ····· 82

8.11 失败模式与负面案例 82
8.12 本章小结 83
第9章 当前挑战与未来方向：从“能学”到“可信地学” 84
9.1 样本效率：数据从哪里来 84
9.1.1 样本复杂度的现实 84
9.1.2 “经验时代” 84
9.2 奖励设计：代理目标的诅咒 85
9.2.1 奖励假设与它的边界 85
9.2.2 三类缓解思路 85
9.2.3 一个根本性的张力 86
9.3 分布偏移与泛化 86
9.3.1 三种分布偏移 86
9.3.2 泛化的层次 86
9.4 长程信用分配 86
9.4.1 为什么长程难 86
9.4.2 分层与选项 87
9.5 安全、对齐与可解释性 87
9.5.1 约束RL 与安全盾 87
9.5.2 对齐的深层困难 87
9.5.3 可解释性 88
9.6 评估危机 88
9.6.1 基准污染与饱和 88
9.6.2 动态评估与“能力边界” 88
9.7 RL Scaling：下一个Scaling Law? 88
9.7.1 从预训练Scaling 到RL Scaling 88
9.7.2 测试时Scaling 89
9.8 未来五年的研究议程 89
9.9 结语：强化学习的“第二曲线” 90
第A章 符号表与常用恒等式 91
第B章 算法速查表与研究复现清单 94

研究导读：如何阅读这份报告

一、问题意识

围绕强化学习的既有材料大致分为三类：教材（如Sutton 与Barto 的《Reinforcement Learning: An Introduction》）给出体系，但节奏偏慢；论文综述聚焦某一子领域，但缺少历史纵深；产业白皮书强调结果，却常常省略推导与适用边界。本报告试图填补三者之间的空隙：**用一条可验证的数学主线，解释算法为何在特定历史节点出现、解决了什么bottleneck、又在什么条件下失效**。

因此，全文围绕五个问题展开：

1. 为什么“延迟回报下的试错学习”需要一个独立的学科？它与管理学、控制论、监督学习的分界线在哪里？
2. 从Bellman 方程到策略梯度，数学上是如何一步步把“改进策略”化为可计算的优化的？
3. 每一次算法范式转移，究竟解决了前一代算法的哪一个具体失效模式？
4. 当前哪些应用已经产生可复核的效果？效应量有多大？证据强度如何？
5. 2026 年的前沿——RLVR、智能体强化学习、世界模型——在哪些方面延续了经典框架，在哪些方面提出了新的理论问题？

二、章节结构

第1 章 历史与思想起源。从最优控制、心理学与早期机器学习三条线索解释RL 的诞生，并给出“为什么是MDP”的论证。

第2 章 数学基础。严格定义MDP、回报、值函数与占用度量；证明贝尔曼算子的压缩性、最优策略的存在性、策略迭代与值迭代的收敛性；讨论函数逼近导致的“死亡三角”。

第3 章 发展历程。以五次范式转移为骨架，串联TD、Q-learning、策略梯度、DQN、AlphaGo、PPO、离线RL、RLHF、RLVR 等关键节点，并分析算法创新的五种机制。

第4 章 值函数方法的推导。TD(0)、TD(λ)、SARSA、Q-learning、Double Q-learning、DQN与分布型RL。

第5 章 策略优化方法的推导。策略梯度定理、REINFORCE、GAE、自然梯度与TRPO、PPO、DPG/DDPG、SAC、TD3。

第6章 模型基、搜索与离线RL。Dyna、MCTS/UCT、AlphaGo/AlphaZero/MuZero、BC-Q/CQL/IQL、Decision Transformer。

第7章 LLM时代的强化学习。RLHF的Bradley–Terry奖励模型、PPO目标、DPO闭式推导、GRPO、RLVR与DeepSeek-R1的训练逻辑。

第8章 应用版图与效果评估。游戏、机器人、LLM/智能体、推荐广告、芯片与系统、能源交通医疗科学、工业金融，逐域给出机制、代表工作、量化效果与证据等级。

第9章 挑战与展望。奖励劫持、样本效率、分布偏移、安全对齐、评估危机、RL scaling与“经验时代”。

附录A/B 符号表、算法速查表与时间线，便于查阅。

三、证据分级与引用原则

本报告对每一则关键经验事实标注证据等级，读者可据此判断结论的稳健性。

等级	来源类型	使用方式
A	同行评审论文、期刊版本、官方技术报告	可作为事实与效应量的主要依据；给出出处与实验条件。
B	权威机构博客、官方文档、基准榜单	数据具体但缺少独立复现；标注“官方口径”。
C	产业案例、媒体报道、二手转述	只用于说明方向与部署规模；不单独作为因果结论。
D	本报告的分析、推断与评论	明确标注为“本报告认为”，与事实陈述区分。

引用采用“作者—年份—出处”与编号参考文献并行的方式：正文中的方括号编号对应文末参考文献；关键论断在脚注中补充实验条件或原始数据口径。对于2025—2026年的新工作，优先引用arXiv预印本并注明版本日期，避免把尚未复现的结果当作定论。

四、符号约定

为减少歧义，全文采用以下约定（完整符号表见附录A）：

- 状态 $s \in \mathcal{S}$，动作 $a \in \mathcal{A}$，时间步 $t = 0, 1, 2, \ldots$；离散时间、有限或可数状态空间除非另行说明。
- 策略 $\pi(a \mid s)$ 表示在状态 s 下选择动作 a 的条件概率；确定性策略记为 $\mu(s)$。
- 回报 $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$，折扣因子 $\gamma \in [0, 1)$。
- 状态值函数 $V^\pi(s)$、动作值函数 $Q^\pi(s, a)$、优势函数 $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$。
- 贝尔曼期望算子 $\mathcal{T}^\pi$、贝尔曼最优算子 $\mathcal{T}^*$；$\mathcal{P}^\pi$ 为策略诱导的转移矩阵。

- 占用度量 $d^{\pi}(s) = (1-\gamma)\sum_{t=0}^{\infty}\gamma^{t}\mathbb{P}(s_t = s \mid \pi)$，另有状态—动作版本 $d^{\pi}(s,a)$。
- 参数化策略 π_θ，参数 $\theta \in \mathbb{R}^d$；ρ_θ 为策略诱导的轨迹分布。
- KL 散度 $\mathrm{KL}(\pi \,\|\, \pi_{\text{ref}}) = \mathbb{E}_{a\sim\pi}[\log \pi(a \mid s) - \log \pi_{\text{ref}}(a \mid s)]$。

五、范围与限制

本报告覆盖“基于交互信号优化决策”的广义强化学习，包括经典RL、深度RL、模型基RL、离线RL、RLHF/RLVR 与智能体RL。以下内容不在核心范围内，但在相关章节简要提及：纯监督的行为克隆、进化策略、贝叶斯优化、最优控制中的线性二次调节器（LQR）与模型预测控制（MPC）。这些方法与RL 共享问题结构，但假设与算法传统不同。

需要特别说明的是，产业界公开的“提升百分比”往往缺少对照组、置信区间与长期效应，且受选择偏差影响。本报告在引用此类数据时，尽量给出原始口径，并在第8 章单列“效应量与外推边界”一节。对于2026 年尚未经过同行评审的前沿工作，本报告将其定位为“值得关注的方向”，而非“已确立的结论”。

六、版本说明

本报告完成于2026 年10 月，文献检索截止于2026 年10 月中旬。强化学习是一个高速演进的领域，任何“当前”判断都有保质期；建议读者结合文末参考文献与附录时间线，自行追踪后续版本。

第 1 章

强化学习诞生的背景

1.1 一个思想史的提问

设想你要让一台机器学会下围棋。你可以把人类棋谱喂给它，让它模仿下一步；这是监督学习。你也可以写一套规则，枚举所有合法落子并用搜索评估；这是经典博弈程序。但如果你希望它在没有人告诉它每一步该下哪里的情况下，仅凭“这盘棋最终赢了还是输了”这样的结果信号，逐渐发现布局、劫争与官子的微妙权衡，那么你面对的就是强化学习（reinforcement learning, RL）的问题。

这个问题的困难不在于“学习”，而在于**信用分配**（credit assignment）：一盘棋只产生一个胜负信号，但棋盘上有数百手落子；究竟是哪一手导致了胜利？类似地，机器人摔倒只得到一个失败信号，但失败可能源于零点几秒前的某个关节力矩；推荐系统的一次点击只产生一个二值反馈，但用户此前已经浏览了十几个物品。监督学习假定每个输入都有明确的标签，因而可以把误差直接反传到对应输出；强化学习只有评价性（evaluative）而非指导性（instructive）的反馈，必须自行判断“哪一个动作应当为最终结果负责”。

正是这个“延迟、稀疏、评价性”的反馈结构，使强化学习成为一门独立的学科。它要回答三个相互纠缠的问题：

1. **预测**：给定一个策略，如何估计它将获得的长期回报？
2. **控制**：如何改进策略，使长期回报更大？
3. **探索**：在未知环境中，如何平衡“利用已知的好动作”与“尝试可能更好的新动作”？

第2 章将把这三个问题写成严格的数学对象；本章先回答一个更基本的问题：这套问题意识从哪里来？为什么它在1950—1980 年代逐渐汇聚成一个学科，而不是更早或更晚？

1.2 三条思想源流

强化学习并非某位学者在某一篇论文中“发明”的。它是三条相对独立的思想传统在20 世纪中期的交汇：最优控制与动态规划、心理学与神经科学中的效果律、以及早期人工智能中的机器自主学习。三条源流各自提供了不同的组件——数学形式、学习法则、以及“机器可以通过经验改善行为”的信念——缺一不可。

1.2.1 源流一：最优控制与动态规划

现代强化学习最直接的数学祖先是Bellman 的动态规划。1957 年，Richard Bellman 在处理多阶段决策问题时提出了**最优性原理**（principle of optimality）：一个最优策略具有这样的性质——无论初始状态和初始决策如何，剩余的决策对于由第一个决策所导致的状态而言，也必须构成一个最优策略[1]。用现代语言说，若 π^* 是最优策略，则在任意状态 s 下，先执行 π^* 给出的动作 a，再从后继状态 s' 开始继续执行 π^*，整体仍然最优。这一原理把“在整条轨迹上做全局优化”分解为“在每个状态上做局部递归优化”，从而导出了著名的Bellman 方程：

$$V^*(s) = \max_{a \in \mathcal{A}} \left\{ r(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s,a) V^*(s') \right\}. \tag{1.1}$$

这里 $V^*(s)$ 是从状态 s 出发、按最优策略行动所能获得的期望折扣回报，$r(s,a)$ 是即时奖励，$P(s' \mid s,a)$ 是转移概率，$\gamma \in [0,1)$ 是折扣因子。式(1.1) 的数学内容将在第2 章严格展开；此处只需注意它的哲学含义：**长期最优可以写成“即时收益+ 未来最优”的递归形式**。这为一切后续的值函数方法、时序差分学习和Q-learning 提供了骨架。

动态规划还提供了一个关键的方法论范式：*策略迭代*（policy iteration）与*值迭代*（value iteration）。前者在“评估当前策略”和“改进当前策略”之间交替，后者直接迭代Bellman 最优算子。两者都要求已知模型 P 与 r，因而属于“有模型”的情形。强化学习后来的核心问题正是：**当 P 与 r 未知、只能通过采样获得经验时，这套递归还成立吗？**答案是肯定的，但需要把“期望”替换为“样本”、把“精确求解”替换为“随机逼近”。这就是TD 与Q-learning 的历史位置。

1.2.2 源流二：心理学中的效果律与神经科学中的奖励预测误差

“通过结果来强化行为”并不是计算机科学的发明。Edward Thorndike 在1911 年提出的**效果律**（law of effect）指出：在相同情境下，带来满意结果的反应会被加强，带来不适结果的反应会被削弱[2]。这一表述虽然行为主义色彩浓厚，却已经包含了现代强化学习最核心的直觉：*行为由其后果塑造*。Skinner 的操作性条件反射进一步把“强化”操作化：奖励的时机、频率与schedule 会系统性地改变行为。

真正把心理学与计算连接起来的是“奖励预测误差”（reward prediction error, RPE）。1997 年，Schultz 等人记录了猴子中脑多巴胺神经元的放电活动，发现这些神经元并不单纯编码“奖励本身”，而是编码“实际奖励与预期奖励之差”：当奖励出乎意料地好时放电增强，当预期奖励落空时放电低于基线[3]。Montague、Dayan 与Sejnowski 在1996 年指出，这一信号与时序差分学习中的TD 误差在形式上高度一致[4]。换句话说，大脑可能使用了一种与TD 学习同构的算法来学习“什么线索预测奖励”。

这条源流的意义有两层。其一，它为强化学习提供了*生物学合理性*：RL 不只是工程技巧，而可能是智能体学习行为的一般机制。其二，它提出了一个至今仍在回响的问题：如果大脑的

奖励信号本身就是“预测误差”，那么当奖励被错误指定时，行为会如何系统性地偏离？这正是当代RLHF 中“奖励劫持”问题的远古版本。

1.2.3 源流三：早期人工智能中的机器自主学习

第三条源流来自人工智能的早期实践。1954 年，Marvin Minsky 在博士论文中设计了SNARC（Stochastic Neural Analog Reinforcement Calculator），用约40 个“神经元”组成的模拟电路模拟老鼠走迷宫的学习过程；它通过随机连接与强化信号来“记住”成功路径[5]。严格说，SNARC 的学习规则还很简单，但它提出了一个重要的方法论立场：*智能可以通过机器自身的经验来获得，而不必完全由程序员编码。*

更具里程碑意义的是Arthur Samuel 在1959 年发表的跳棋程序[6]。Samuel 的程序包含三个要素：对棋盘局面的线性评估函数、通过自我对弈产生的训练数据、以及“rote learning”与“generalization”两种学习机制。程序并不需要人类标注“这一步是好棋”；它通过最终胜负与搜索，反向调整评估函数的权重。用现代术语说，Samuel 已经同时触及了*策略评估*、*自博弈*与*函数逼近*三个概念。也正因如此，Sutton 与Barto 在经典教材中把Samuel 的工作视为强化学习的思想先驱之一[22]。

不过，早期AI 的这条线索在1970—1980 年代一度边缘化。原因并不难理解：当时的计算资源无法支撑大规模采样，感知与表示能力不足，且缺乏处理“延迟回报”的严格数学工具。直到Bellman 的动态规划、心理学中的TD 思想与机器学习中的函数逼近逐渐合流，强化学习才具备成为一个独立领域的条件。

1.3 问题形式化：智能体与环境的交互环路

三条源流最终汇聚为一个极简却强大的抽象：**智能体—环境交互环路**（agent–environment interaction loop）。在离散时间 $t = 0, 1, 2, \dots$ 上，智能体观察到一个状态 $S_t \in \mathcal{S}$，根据策略 π 选择一个动作 $A_t \in \mathcal{A}$；环境接收动作后，产生一个奖励 $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$ 与一个新的状态 S_{t+1}。这个过程不断重复，形成一条轨迹

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots). \tag{1.2}$$

智能体的目标不是最大化某一个即时奖励，而是最大化长期累积回报，例如

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \qquad \gamma \in [0, 1). \tag{1.3}$$

折扣因子 γ 既可以被解释为“未来的不确定性”，也可以被解释为“机会成本”：今天的1 单位奖励，等价于明天的 γ 单位奖励。它在数学上还保证了一个关键性质：当奖励有界时，无穷级数(1.3) 绝对收敛，从而值函数有良好定义。

这个环路看似简单，却已经把强化学习与其它学习范式区分开来：

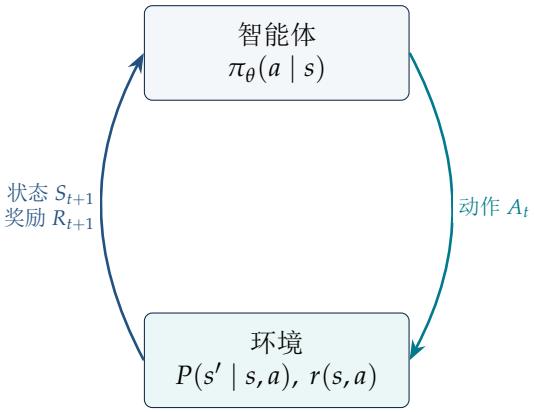


图 1.1 智能体—环境交互环路。智能体依据策略选择动作，环境返回奖励与下一状态；策略参数 θ 通过交互产生的轨迹更新。

- **没有“正确答案”**：环境不会告诉智能体“在状态 s 应该选动作 a”；它只给出一个标量奖励。智能体必须通过试错来推断动作的后果。
- **动作影响数据分布**：智能体选择什么动作，决定了它接下来看到什么状态、获得什么奖励。这与监督学习中“数据独立同分布”的假设根本不同，也是“探索—利用”张力的来源。
- **奖励延迟且稀疏**：最终结果可能只在轨迹末端出现，智能体必须把功劳或责任分配给此前的许多决策。
- **环境可能非平稳**：智能体自身的策略变化会改变它体验到的状态分布；在多智能体场景中，其他智能体也在学习，环境因而不断变化。

1.4 为什么是马尔可夫决策过程？

要把上述环路变成可分析的数学对象，必须对“状态”作出假设。最常用、也最有效的假设是**马尔可夫性**（Markov property）：

$$\mathbb{P}(S_{t+1}=s', R_{t+1}=r \mid S_0, A_0, \ldots, S_t, A_t) = \mathbb{P}(S_{t+1}=s', R_{t+1}=r \mid S_t, A_t). \tag{1.4}$$

也就是说，*给定当前状态与动作，未来与更早的历史条件独立*。用信息论的语言，当前状态 S_t 是决策所需的*充分统计量*：所有会影响未来的历史信息都被压缩进了 S_t。

为什么这个假设如此重要？至少有三层原因。

第一，它使**递归**成为可能。若未来只依赖当前状态，则“从 s 出发的长期回报”可以写成“即时奖励+ 从后继状态出发的长期回报”的期望，而不必把整条历史作为条件。这正是 Bellman 方程成立的前提。

第二，它使**值函数**成为有限维对象。若未来依赖全部历史，则值函数必须定义在历史序列上，其数量随长度指数增长；在马尔可夫假设下，值函数只需定义在状态空间 $\mathcal{S}$ 上。对于表格

型问题，这意味着可以用 $|\mathcal{S}|$ 个数值表示策略评估结果；对于函数逼近，这意味着神经网络只需以当前状态为输入。

第三，它使**采样估计**具有统计意义。智能体在状态 s 执行动作 a 后观察到的后继状态与奖励，是从条件分布 $P(\cdot \mid s,a)$ 中抽取的样本；反复采样即可逼近期望。若无马尔可夫性，同一条“状态—动作”对在不同历史下会对应不同的转移分布，采样估计将不再一致。

当然，马尔可夫性是一个理想化假设。真实世界中的观测往往是部分的：机器人只知道关节角度而不知道地面摩擦，推荐系统只知道用户点击而不知道其真实偏好，LLM 只知道当前上下文而不知道用户意图。这类问题应建模为**部分可观测马尔可夫决策过程**（POMDP），其严格处理需要引入信念状态（belief state）或记忆机制。第2 章会在扩展小节中给出POMDP 的形式化；此处只需强调：*经典RL 的全部结论都建立在MDP 假设之上；当假设失效时，算法不会自动失效，但其最优性保证会失效。*

1.5 与相邻学科的边界

强化学习常被误解为“监督学习的一种”或“最优控制的一个分支”。这两种说法都只说对了一半。为避免后续讨论中的概念混淆，这里给出四条边界。

与监督学习。 监督学习从带标签的样本 (x,y) 中学习映射 $f: x \mapsto y$，损失函数逐样本可分解；强化学习从轨迹中学习策略，奖励是轨迹级的、延迟的，且数据分布由策略自身决定。更重要的是，监督学习假设“老师知道正确答案”，而强化学习只假设“环境会给出评价”。这一差别在数学上表现为：监督学习的经验风险是独立同分布样本的平均，强化学习的目标是策略诱导分布下的期望，二者之间存在*分布偏移*。

与最优控制。 最优控制同样研究序贯决策，但其经典形式假设系统动力学已知（或可辨识），并用变分法、庞特里亚金最大值原理或Hamilton–Jacobi–Bellman 方程求解。强化学习则假设动力学未知，通过采样学习；它更关心“在有限样本与有限计算下如何逼近最优”。可以说，最优控制提供了“目标的形式”，强化学习提供了“学习的算法”。

与多臂老虎机。 多臂老虎机（multi-armed bandit）是MDP 的特例：状态空间只有一个元素，动作不影响未来状态，只影响即时奖励。老虎机已经包含了“探索—利用”的核心张力，但缺少“状态”与“延迟回报”两个维度。理解老虎机是理解完整RL 的第一步，但不能把老虎机的结论直接外推到MDP。

与进化算法。 进化策略通过种群层面的变异与选择来优化行为，不要求对个体动作做信用分配；强化学习则显式地估计“某个状态—动作对的长期价值”。在奖励稀疏、维度较低时，进化策略可能更稳健；在需要精细信用分配的任务中，RL 通常更样本有效。二者并非互斥，现代深度RL 也常把进化思想用于超参数搜索或探索。

1.6 历史必然性与偶然性

为什么强化学习在1950—1980 年代形成，而不是更早？一个常见的解释是“计算条件不具备”。这当然重要，但不完全。更深层的原因在于，强化学习需要同时具备四个条件：

1. **一个可分析的抽象**：MDP 与Bellman 方程提供了“延迟回报”的递归结构；
2. **一个可执行的学习法则**：TD 误差提供了仅凭采样就能更新的局部规则；
3. **一个可扩展的表示**：从表格到线性函数逼近，再到深度网络，表示能力决定了算法能处理多复杂的状态；
4. **一个可负担的交互环境**：从真实机器人到Atari 模拟器，再到LLM 的可验证题目生成器，环境成本决定了样本效率的经济性。

这四个条件在不同年代先后成熟：1950—1970 年代完成了(1)，1980 年代末完成了(2)，1990—2010 年代逐步完成(3)，2010 年代之后(4) 才真正规模化。于是我们看到了一个有趣的历史节奏：理论先行，算法随后，表示再后，应用最后。这也解释了为什么DQN（2015）看起来是“突然爆发”，实际上是卷积网络、GPU、Atari 平台与Q-learning 四者在同一时间窗口的汇合。

同样，2022 年之后的LLM 强化学习也遵循这一节奏：PPO（2017）、奖励建模（2017—2020）与Transformer（2017）早已就位，但直到“人类偏好数据+ 大规模分布式训练+ 可验证题目”三个条件同时具备，RLHF/RLVR 才从实验室走向主流程。历史不是简单的“算法英雄史”，而是抽象、算法、表示、环境四重条件的耦合史。

1.7 本章小结

强化学习的诞生，是三条思想源流在20 世纪中期的交汇：Bellman 的动态规划提供了“延迟回报的递归数学”，Thorndike 的效果律与多巴胺奖励预测误差提供了“行为由后果塑造”的生物学原型，Samuel 与Minsky 的早期程序提供了“机器可以通过经验改善行为”的工程信念。MDP 把三者压缩为一个可分析的对象，使信用分配从哲学问题变成期望问题。

但MDP 只是一个“问题模板”，它本身并不告诉我们如何在未知环境中学习。第2 章将给出这个模板的严格数学结构：回报、值函数、Bellman 算子、压缩映射、策略迭代与值迭代。只有把这些对象定义清楚，后续的TD、Q-learning、策略梯度与RLHF 才能被理解为同一个骨架上的不同“学习法则”。

第 2 章

数学基础：马尔可夫决策过程与贝尔曼算子

本章建立全文的数学语言。我们首先给出有限马尔可夫决策过程的形式定义，推导值函数的贝尔曼方程与占用度量表示；随后证明贝尔曼算子的压缩性、最优策略的存在性、策略迭代与值迭代的收敛性；最后讨论函数逼近带来的理论困难，以及部分可观测与平均奖励两个重要扩展。本章的结论是后续所有算法的共同地基，因而证明写得较为完整；不熟悉测度论的读者可把 $\mathcal{S}$ 与 $\mathcal{A}$ 理解为有限集合，所有期望均为有限和。

2.1 马尔可夫决策过程

定义 2.1 (折扣马尔可夫决策过程)**.** 一个折扣马尔可夫决策过程由六元组

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0) \tag{2.1}$$

构成，其中：

- $\mathcal{S}$ 是状态空间，$\mathcal{A}$ 是动作空间；本文默认二者为有限集，除非另行说明；
- $P : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ 是转移核，$P(s' \mid s, a)$ 表示在状态 s 执行动作 a 后到达 s' 的概率；
- $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ 是即时奖励函数，满足 $|r(s,a)| \le R_{\max} < \infty$；
- $\gamma \in [0, 1)$ 是折扣因子；
- $\rho_0 \in \Delta(\mathcal{S})$ 是初始状态分布。

定义 2.2 (策略)**.** 一个随机平稳策略是映射 $\pi : \mathcal{S} \to \Delta(\mathcal{A})$，其中 $\pi(a \mid s)$ 表示在状态 s 选择动作 a 的概率。若对每个 s 都存在 a 使 $\pi(a \mid s) = 1$，则称 π 为确定性策略，记作 $\pi(s)$。所有平稳策略的集合记为 Π。

在策略 π 下，轨迹 $\tau = (S_0, A_0, R_1, S_1, \dots)$ 的分布由初始分布、策略与转移核共同决定。对有限时域 T，

$$p_\pi(\tau) = \rho_0(S_0) \prod_{t=0}^{T-1} \pi(A_t \mid S_t) P(S_{t+1} \mid S_t, A_t), \tag{2.2}$$

奖励 $R_{t+1} = r(S_t, A_t)$ 由状态—动作对确定（若奖励本身随机，可把奖励并入状态或引入奖励核，结论不变）。式(2.2) 是后续“似然比技巧”的起点：策略只通过乘积中的 $\pi(A_t \mid S_t)$ 影响轨迹分布，因而对 θ 求导时，只有这些因子会产生非零贡献。

定义2.3 (回报与值函数)**.** 从时刻 t 开始的折扣回报定义为

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \tag{2.3}$$

给定策略 π，状态值函数与动作值函数分别为

$$V^\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right], \tag{2.4}$$

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right]. \tag{2.5}$$

优势函数定义为

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s). \tag{2.6}$$

由于 $|R_{t+k+1}| \le R_{\max}$ 且 $\gamma < 1$，回报满足 $|G_t| \le R_{\max}/(1-\gamma)$，故值函数有界，上述期望存在且有限。这个有界性是压缩映射论证中“无穷范数”良定义的前提。

2.1.1 占用度量与目标函数

强化学习的目标通常写成初始分布下的期望回报

$$J(\pi) = \mathbb{E}_{S_0 \sim \rho_0} \left[V^\pi(S_0) \right]. \tag{2.7}$$

直接对 J 求导需要处理轨迹分布，计算上不方便。一个更结构化的工具是**占用度量**（occupancy measure）。

定义2.4 (折扣占用度量)**.** 策略 π 的折扣状态占用度量与状态—动作占用度量分别定义为

$$d^\pi(s) = (1-\gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}^\pi(S_t = s), \tag{2.8}$$

$$d^\pi(s, a) = d^\pi(s) \pi(a \mid s). \tag{2.9}$$

由于 $\sum_{t=0}^{\infty} \gamma^t = 1/(1-\gamma)$ 且 $\sum_s \mathbb{P}^\pi(S_t = s) = 1$，有 $\sum_s d^\pi(s) = 1$，故 d^π 是一个概率分布。它把“在时间上按 γ 衰减的访问频率”重新归一化为一个静态分布，这一技巧在策略梯度与离线RL 中反复出现。

命题2.1 (目标函数的占用度量表示)**.** *对任意策略 π，*

$$J(\pi) = \frac{1}{1-\gamma} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} d^\pi(s, a)\, r(s, a). \tag{2.10}$$

证明 由期望的线性性与马尔可夫性，

$$
\begin{aligned}
J(\pi) &= \mathbb{E}_\pi\left[\sum_{t=0}^{\infty}\gamma^t R_{t+1}\right] = \sum_{t=0}^{\infty}\gamma^t \sum_{s,a}\mathbb{P}^\pi(S_t=s, A_t=a)\, r(s,a) \\
&= \sum_{s,a} r(s,a) \sum_{t=0}^{\infty}\gamma^t \mathbb{P}^\pi(S_t=s)\pi(a\mid s) \\
&= \frac{1}{1-\gamma}\sum_{s,a} r(s,a)\underbrace{(1-\gamma)\sum_{t=0}^{\infty}\gamma^t \mathbb{P}^\pi(S_t=s)}_{d^\pi(s)}\pi(a\mid s) \\
&= \frac{1}{1-\gamma}\sum_{s,a} d^\pi(s,a) r(s,a). \qquad (2.11)
\end{aligned}
$$

□

式(2.10) 揭示了一个重要事实：**策略对目标的影响完全通过它诱导的占用度量起作用**。若两个策略诱导相同的 d^π，它们的目标值相同；反之，策略优化可以看作在“可实现占用度量集合”上最大化线性函数。这一视角是镜像下降、相对熵策略搜索与对偶方法的共同起点。

2.2 贝尔曼方程

2.2.1 策略评估的贝尔曼方程

定理2.1 (贝尔曼期望方程). *对任意策略 π，值函数满足*

$$
V^\pi(s) = \sum_{a\in\mathcal{A}}\pi(a\mid s)\left[r(s,a) + \gamma\sum_{s'\in\mathcal{S}} P(s'\mid s,a)V^\pi(s')\right], \qquad (2.12)
$$

$$
Q^\pi(s,a) = r(s,a) + \gamma\sum_{s'\in\mathcal{S}} P(s'\mid s,a)\sum_{a'\in\mathcal{A}}\pi(a'\mid s')Q^\pi(s',a'). \qquad (2.13)
$$

证明 由回报的递归结构 $G_t = R_{t+1} + \gamma G_{t+1}$，取条件期望：

$$
\begin{aligned}
V^\pi(s) &= \mathbb{E}_\pi\left[R_{t+1} + \gamma G_{t+1} \mid S_t = s\right] \\
&= \mathbb{E}_\pi\left[R_{t+1}\mid S_t=s\right] + \gamma\,\mathbb{E}_\pi\left[\mathbb{E}_\pi\left[G_{t+1}\mid S_{t+1}, A_{t+1}, \ldots\right]\mid S_t=s\right] \\
&= \sum_a \pi(a\mid s) r(s,a) + \gamma\sum_a\pi(a\mid s)\sum_{s'}P(s'\mid s,a)\mathbb{E}_\pi\left[G_{t+1}\mid S_{t+1}=s'\right] \\
&= \sum_a \pi(a\mid s)\left[r(s,a) + \gamma\sum_{s'}P(s'\mid s,a)V^\pi(s')\right]. \qquad (2.14)
\end{aligned}
$$

第二步使用了全期望公式（tower property），第三步使用了马尔可夫性：给定 $S_{t+1}=s'$，未来与 S_t, A_t 条件独立。动作值函数的推导完全类似，只需把初始条件改为 $S_t=s, A_t=a$，并在第一步保留 $A_t=a$ 即可。 □

把式(2.12) 写成矩阵形式。令 $\boldsymbol{v}^\pi \in \mathbb{R}^{|\mathcal{S}|}$ 为值向量，$\boldsymbol{r}^\pi \in \mathbb{R}^{|\mathcal{S}|}$ 满足 $r^\pi(s) = \sum_a \pi(a \mid s) r(s,a)$，$\boldsymbol{P}^\pi \in \mathbb{R}^{|\mathcal{S}|\times|\mathcal{S}|}$ 满足 $P^\pi(s,s') = \sum_a \pi(a \mid s) P(s' \mid s,a)$，则

$$\boldsymbol{v}^\pi = \boldsymbol{r}^\pi + \gamma \boldsymbol{P}^\pi \boldsymbol{v}^\pi \quad \Longleftrightarrow \quad \boldsymbol{v}^\pi = \left(\boldsymbol{I} - \gamma \boldsymbol{P}^\pi\right)^{-1} \boldsymbol{r}^\pi. \tag{2.15}$$

由于 $\boldsymbol{P}^\pi$ 是随机矩阵，其谱半径不超过 1，而 $\gamma < 1$，故 $\boldsymbol{I} - \gamma \boldsymbol{P}^\pi$ 可逆，且

$$\left(\boldsymbol{I} - \gamma \boldsymbol{P}^\pi\right)^{-1} = \sum_{k=0}^{\infty} \gamma^k \left(\boldsymbol{P}^\pi\right)^k. \tag{2.16}$$

式(2.16) 是Neumann 级数，它把“无穷未来的折扣和”展开为“每一步转移的加权和”，在数值上对应迭代法，在概念上对应“值函数是转移算子的预解式”。

2.2.2 最优贝尔曼方程

定义2.5 (最优值函数). $V^*(s) = \sup_{\pi\in\Pi} V^\pi(s)$，$Q^*(s,a) = \sup_{\pi\in\Pi} Q^\pi(s,a)$。若策略 π^* 满足 $V^{\pi^*}(s) = V^*(s)$ 对所有 s 成立，则称其最优。

定理2.2 (贝尔曼最优方程). *在有限MDP 中，V^* 与 Q^* 满足*

$$V^*(s) = \max_{a\in\mathcal{A}} \left[r(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s' \mid s,a) V^*(s') \right], \tag{2.17}$$

$$Q^*(s,a) = r(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s' \mid s,a) \max_{a'\in\mathcal{A}} Q^*(s',a'). \tag{2.18}$$

并且存在一个确定性平稳最优策略 $\pi^*(s) \in \arg\max_a Q^*(s,a)$。

定理2.2 的证明依赖于“最大值与期望可交换”以及压缩映射的不动点性质。我们把它拆成两步：先证明最优算子有唯一不动点，再证明该不动点就是最优值函数。这样安排可以避免循环论证。

2.3 贝尔曼算子与压缩映射

定义2.6 (贝尔曼算子). 对任意有界函数 $V : \mathcal{S} \to \mathbb{R}$，定义贝尔曼期望算子 $\mathcal{T}^\pi$ 与贝尔曼最优算子 $\mathcal{T}^*$：

$$(\mathcal{T}^\pi V)(s) = \sum_a \pi(a \mid s) \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s') \right], \tag{2.19}$$

$$(\mathcal{T}^* V)(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s') \right]. \tag{2.20}$$

引理2.1 (压缩性). *在无穷范数 $\|V\|_\infty = \max_s |V(s)|$ 下，$\mathcal{T}^\pi$ 与 $\mathcal{T}^*$ 都是 γ-压缩映射：*

$$\|\mathcal{T}^\pi V_1 - \mathcal{T}^\pi V_2\|_\infty \le \gamma \|V_1 - V_2\|_\infty, \qquad \|\mathcal{T}^* V_1 - \mathcal{T}^* V_2\|_\infty \le \gamma \|V_1 - V_2\|_\infty. \tag{2.21}$$

证明 对 $\mathcal{T}^\pi$，固定 s，有

$$\begin{aligned} |(\mathcal{T}^\pi V_1)(s) - (\mathcal{T}^\pi V_2)(s)| &= \left| \gamma \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \left(V_1(s') - V_2(s') \right) \right| \\ &\le \gamma \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \left| V_1(s') - V_2(s') \right| \\ &\le \gamma \|V_1 - V_2\|_\infty \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \\ &= \gamma \|V_1 - V_2\|_\infty. \end{aligned} \tag{2.22}$$

对 $\mathcal{T}^*$，利用不等式 $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$，令 $f(a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s, a) V_1(s')$，$g(a)$ 同理以 V_2 替换，则

$$\begin{aligned} |(\mathcal{T}^* V_1)(s) - (\mathcal{T}^* V_2)(s)| &\le \max_a \left| \gamma \sum_{s'} P(s' \mid s, a) \left(V_1(s') - V_2(s') \right) \right| \\ &\le \gamma \|V_1 - V_2\|_\infty. \end{aligned} \tag{2.23}$$

对 s 取最大值即得式(2.21)。 □

定理2.3 (Banach 不动点定理). *设 $(\mathcal{X}, d)$ 是完备度量空间，$T : \mathcal{X} \to \mathcal{X}$ 满足 $d(Tx, Ty) \le \gamma d(x, y)$，其中 $0 \le \gamma < 1$。则 T 存在唯一不动点 x^*，且对任意 $x_0 \in \mathcal{X}$，迭代 $x_{k+1} = T x_k$ 满足*

$$d(x_k, x^*) \le \frac{\gamma^k}{1 - \gamma} d(x_1, x_0). \tag{2.24}$$

证明 唯一性：若 $Tx^* = x^*$ 且 $Ty^* = y^*$，则 $d(x^*, y^*) = d(Tx^*, Ty^*) \le \gamma d(x^*, y^*)$，因 $\gamma < 1$，只能 $d(x^*, y^*) = 0$。

存在性：对任意 $k \ge 1$，

$$d(x_{k+1}, x_k) = d(Tx_k, Tx_{k-1}) \le \gamma d(x_k, x_{k-1}) \le \cdots \le \gamma^k d(x_1, x_0). \tag{2.25}$$

于是对 $m > n$，由三角不等式与几何级数，

$$d(x_m, x_n) \le \sum_{k=n}^{m-1} d(x_{k+1}, x_k) \le d(x_1, x_0) \sum_{k=n}^{m-1} \gamma^k \le \frac{\gamma^n}{1 - \gamma} d(x_1, x_0). \tag{2.26}$$

故 $\{x_k\}$ 是Cauchy 列；由完备性，存在极限 x^*。由压缩性，T 连续，故 $Tx^* = T \lim_k x_k = \lim_k T x_k = \lim_k x_{k+1} = x^*$。令 $m \to \infty$ 即得式(2.24)。 □

推论2.1 (值函数的存在唯一性与迭代收敛). *对任意策略 π，$\mathcal{T}^\pi$ 在 $\mathbb{R}^{|\mathcal{S}|}$ 上有唯一不动点 V^π；$\mathcal{T}^*$ 有唯一不动点 V^*。并且从任意 V_0 出发，*

$$\|V_k^\pi - V^\pi\|_\infty \le \gamma^k \|V_0^\pi - V^\pi\|_\infty, \tag{2.27}$$

$$\|V_k^* - V^*\|_\infty \le \gamma^k \|V_0^* - V^*\|_\infty. \tag{2.28}$$

证明 $\mathbb{R}^{|\mathcal{S}|}$ 在无穷范数下完备；由引理2.1，两个算子均为压缩映射；直接应用定理2.3。不动点分别是贝尔曼期望方程与最优方程的解，而方程解的唯一性由压缩性保证。 □

定理2.4 (最优策略的存在性). *设 V^* 是 $\mathcal{T}^*$ 的唯一不动点。定义贪心策略*

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s') \right]. \tag{2.29}$$

则 π^* 是最优策略，且 $V^{\pi^*} = V^*$。

证明 由 π^* 的定义，对任意 s，

$$r(s,\pi^*(s)) + \gamma \sum_{s'} P(s' \mid s,\pi^*(s)) V^*(s') = \max_a \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s') \right] = V^*(s). \tag{2.30}$$

故 $V^* = \mathcal{T}^{\pi^*} V^*$。由 $\mathcal{T}^{\pi^*}$ 的不动点唯一性，$V^{\pi^*} = V^*$。于是 π^* 达到最优值。对任意 π，$V^\pi \le V^*$ 由定义；故 π^* 最优。 □

2.4 策略迭代与值迭代

2.4.1 策略改进定理

定理2.5 (策略改进). *设 π 与 π' 是任意两个策略。若对任意 $s \in \mathcal{S}$，*

$$Q^\pi(s, \pi'(s)) \ge V^\pi(s), \tag{2.31}$$

则 $V^{\pi'}(s) \ge V^\pi(s)$ 对所有 s 成立；若存在某个状态使不等式严格成立，则至少在该状态上 $V^{\pi'}(s) > V^\pi(s)$。

证明 由式(2.31)与贝尔曼方程，

$$\begin{aligned} V^{\pi}(s) &\le Q^{\pi}(s,\pi'(s)) \\ &= \mathbb{E}\left[R_{t+1}+\gamma V^{\pi}(S_{t+1}) \mid S_t=s, A_t=\pi'(s)\right] \\ &\le \mathbb{E}\left[R_{t+1}+\gamma Q^{\pi}(S_{t+1},\pi'(S_{t+1})) \mid S_t=s, A_t=\pi'(s)\right] \\ &= \mathbb{E}_{\pi'}\left[R_{t+1}+\gamma R_{t+2}+\gamma^2 V^{\pi}(S_{t+2}) \mid S_t=s\right]. \end{aligned} \tag{2.32}$$

重复展开 k 步，得到

$$V^{\pi}(s) \le \mathbb{E}_{\pi'}\left[\sum_{j=0}^{k-1}\gamma^j R_{t+j+1}+\gamma^k V^{\pi}(S_{t+k}) \mid S_t=s\right]. \tag{2.33}$$

令 $k\to\infty$。由于 $|V^{\pi}|\le R_{\max}/(1-\gamma)$，余项 $\gamma^k V^{\pi}(S_{t+k})$ 一致趋于零，故

$$V^{\pi}(s) \le \mathbb{E}_{\pi'}\left[\sum_{j=0}^{\infty}\gamma^j R_{t+j+1} \mid S_t=s\right] = V^{\pi'}(s). \tag{2.34}$$

严格不等式的情形由每一步的严格性沿正概率路径传播得到。 □

策略改进定理给出了一个非常实用的推论：若 $\pi'(s)\in\arg\max_a Q^{\pi}(s,a)$，则 $Q^{\pi}(s,\pi'(s))=\max_a Q^{\pi}(s,a)\ge V^{\pi}(s)$，故贪心策略不会变差。这正是策略迭代的合法性来源。

2.4.2 策略迭代

策略迭代（policy iteration）在“评估”与“改进”之间交替：

算法2.1（策略迭代）

1. 初始化策略 π_0；
2. 对 $k=0,1,2,\dots$：
(a) **策略评估**：解贝尔曼方程 $V^{\pi_k}=(\boldsymbol{I}-\gamma\boldsymbol{P}^{\pi_k})^{-1}\boldsymbol{r}^{\pi_k}$；
(b) **策略改进**：$\pi_{k+1}(s)\in\arg\max_a\left[r(s,a)+\gamma\sum_{s'}P(s'\mid s,a)V^{\pi_k}(s)\right]$；
(c) 若 $\pi_{k+1}=\pi_k$，停止并返回 π_k。

定理2.6 (策略迭代的收敛性)**.** *在有限MDP中，策略迭代在有限步内终止，且终止时得到的策略是最优策略。*

证明 由策略改进定理，$V^{\pi_{k+1}}\ge V^{\pi_k}$。若策略发生变化，则至少存在一个状态 s 使 $V^{\pi_{k+1}}(s)>V^{\pi_k}(s)$（否则新策略与旧策略在该状态的所有动作上等价，按确定性策略的取法可视为未变）。因此值向量严格上升。

有限MDP 的确定性平稳策略总数至多 $|\mathcal{A}|^{|\mathcal{S}|}$，而值向量严格上升意味着不会重复访问同一策略，故迭代必在有限步内停止。停止时 $\pi_{k+1}=\pi_k$，即

$$\pi_k(s)\in\arg\max_a\left[r(s,a)+\gamma\sum_{s'}P(s'\mid s,a)V^{\pi_k}(s')\right], \tag{2.35}$$

故 $V^{\pi_k}=\mathcal{T}^*V^{\pi_k}$。由 $\mathcal{T}^*$ 不动点唯一性，$V^{\pi_k}=V^*$，即 π_k 最优。 □

策略迭代的每一步评估需要解线性方程组，复杂度约为 $O(|\mathcal{S}|^3)$（或使用迭代法）；改进需要 $O(|\mathcal{S}|^2|\mathcal{A}|)$。它的收敛通常很快，但每轮成本高。

2.4.3 值迭代

值迭代（value iteration）直接迭代最优算子：

$$V_{k+1}(s)=\max_a\left[r(s,a)+\gamma\sum_{s'}P(s'\mid s,a)V_k(s')\right],\qquad V_0\equiv 0. \tag{2.36}$$

由推论2.1，$V_k\to V^*$ 几何收敛。实际中常用停止准则 $\|V_{k+1}-V_k\|_\infty<\varepsilon$，并用贪心策略抽取最终策略。

定理2.7 (值迭代的近似最优性)**.** *设 V_k 由式(2.36) 产生，π_k 为关于 V_k 的贪心策略。则*

$$\|V^*-V^{\pi_k}\|_\infty\le\frac{2\gamma}{1-\gamma}\|V^*-V_k\|_\infty\le\frac{2\gamma^{k+1}}{1-\gamma}\|V^*-V_0\|_\infty. \tag{2.37}$$

证明 记 $\epsilon=\|V^*-V_k\|_\infty$。由 π_k 的贪心性，$\mathcal{T}^{\pi_k}V_k=\mathcal{T}^*V_k$。于是

$$\begin{aligned}V^*-V^{\pi_k}&=\mathcal{T}^*V^*-\mathcal{T}^{\pi_k}V^{\pi_k}\\&=(\mathcal{T}^*V^*-\mathcal{T}^*V_k)+(\mathcal{T}^{\pi_k}V_k-\mathcal{T}^{\pi_k}V^{\pi_k})\\&\le\gamma\|V^*-V_k\|_\infty\mathbf{1}+\gamma\|V_k-V^{\pi_k}\|_\infty\mathbf{1},\end{aligned} \tag{2.38}$$

其中 $\mathbf{1}$ 为全 1 向量，不等式逐分量成立。取无穷范数：

$$\|V^*-V^{\pi_k}\|_\infty\le\gamma\epsilon+\gamma\|V_k-V^{\pi_k}\|_\infty\le\gamma\epsilon+\gamma\left(\epsilon+\|V^*-V^{\pi_k}\|_\infty\right). \tag{2.39}$$

移项得 $(1-\gamma)\|V^*-V^{\pi_k}\|_\infty\le 2\gamma\epsilon$，即第一个不等式。第二个不等式由推论2.1 代入 $\epsilon\le\gamma^k\|V^*-V_0\|_\infty$ 得到。 □

这个界说明：值迭代的“值误差”以 γ^k 衰减，而“策略误差”只以 $2\gamma/(1-\gamma)$ 放大。当 γ 接近 1 时，放大因子趋于无穷；这解释了为什么长时程任务中“值函数近似得很好”并不自动意味着“策略很好”。

2.5 探索的数学：从老虎机到UCB

在进入无模型算法之前，先处理一个最纯粹的探索问题。多臂老虎机（multi-armed bandit）是状态空间只有单个元素的MDP：智能体在 K 个动作（臂）中选择，动作 a 的奖励服从未知分布，均值为 μ_a，最优均值为 $\mu^* = \max_a \mu_a$。执行 T 次后，**遗憾**（regret）定义为

$$R_T = T\mu^* - \sum_{t=1}^{T} \mu_{a_t}. \tag{2.40}$$

目标是使 $\mathbb{E}[R_T]$ 随 T 增长尽可能慢。若已知各臂均值，最优策略是始终选择最优臂，遗憾为零；未知时，必须通过采样估计均值，同时承担“选错”的代价。

UCB1 算法。 对每个臂 a，记录被选择的次数 $n_a(t)$ 与经验均值 $\bar{\mu}_a(t)$。在第 t 步选择

$$a_t \in \arg\max_a \left[\bar{\mu}_a(t) + \sqrt{\frac{2\ln t}{n_a(t)}} \right]. \tag{2.41}$$

第二项是“探索奖励”：选择次数越少，置信上界越高。它的形式来自Hoeffding 不等式：对 n 个 $[0,1]$ 奖励样本，均值以至少 $1-\delta$ 的概率落在 $\bar{\mu} \pm \sqrt{\ln(1/\delta)/(2n)}$ 内。取 $\delta = t^{-4}$ 并用 union bound 即得式(2.41) 的常数。

定理2.8 (UCB1 的遗憾界)**.** *若所有奖励位于 $[0,1]$，UCB1 在 T 步后的期望遗憾满足*

$$\mathbb{E}[R_T] \le 8 \sum_{a:\mu_a<\mu^*} \frac{\ln T}{\Delta_a} + O(K), \qquad \Delta_a = \mu^* - \mu_a. \tag{2.42}$$

证明思路 把“次优臂 a 在第 t 步被选中”这一事件分解为两个互斥事件：要么 a 的置信上界高于 μ^*（探索项过大），要么最优臂的置信下界低于 μ^*（低估）。由Hoeffding 不等式与union bound，两者发生的概率之和不超过 $2t^{-4}$。对 t 求和，并利用 Δ_a 把“被选次数”换算为遗憾，即得式(2.42)。完整常数与 $O(K)$ 项见Auer 等人的原始论文[17]。 □

UCB 的意义不止于老虎机。它给出了“乐观面对不确定性”（optimism in the face of uncertainty）这一探索原则的严格版本：在不确定的地方给予奖励，直到不确定性被数据消除。这一原则后来被推广到MDP 的UCBVI、Euler、UCB-MCTS 等算法，成为样本复杂度分析的核心工具。

2.6 函数逼近与“死亡三角”

2.6.1 从表格到函数逼近

当状态空间很大或连续时，表格型值函数无法存储。代之以为值函数指定一个参数化族 $V_\theta(s)$，例如线性模型 $V_\theta(s)=\phi(s)^\top\theta$ 或神经网络。目标是最小化投影误差

$$\mathcal{L}(\theta)=\sum_{s\in\mathcal{S}} d(s)\left(V_\theta(s)-V^\pi(s)\right)^2, \tag{2.43}$$

其中 d 是状态分布。若用梯度下降直接最小化(2.43)，需要知道 $V^\pi(s)$，这正是策略评估要解决的问题；于是形成“用估计目标估计估计”的循环。

2.6.2 半梯度TD 与投影

TD(0) 的线性函数逼近更新为

$$\theta_{t+1}=\theta_t+\alpha\left[R_{t+1}+\gamma\phi(S_{t+1})^\top\theta_t-\phi(S_t)^\top\theta_t\right]\phi(S_t). \tag{2.44}$$

它并不是(2.43) 的真实梯度，因为目标 $R_{t+1}+\gamma\phi(S_{t+1})^\top\theta_t$ 也依赖 θ_t；若把目标视为常数，得到的“半梯度”为

$$-\frac{1}{2}\nabla_\theta\left(R_{t+1}+\gamma V_\theta(S_{t+1})-V_\theta(S_t)\right)^2=\delta_t\phi(S_t), \tag{2.45}$$

其中 $\delta_t=R_{t+1}+\gamma V_\theta(S_{t+1})-V_\theta(S_t)$。半梯度方法在表格型情形下等价于TD(0)，但在函数逼近下可能不收敛于投影不动点。

2.6.3 死亡三角

Sutton 与Barto 把“函数逼近+ 自举（bootstrapping）+ 离策略训练”称为**死亡三角**（deadly triad）[22]。三者单独出现时通常是安全的：

- 函数逼近+ 在策略+ 不自举：监督学习式的回归，收敛；
- 表格+ 自举+ 离策略：Q-learning，在适当条件下收敛；
- 表格+ 在策略+ 自举：TD(0)，收敛。

但三者同时出现时，更新算子可能不再是压缩映射，值函数可能发散。经典的Baird 反例给出了一个具体的发散构造[19]。其直觉是：离策略分布 d 与目标策略 π 的占用分布不匹配，函数逼近的投影在 d 下进行，而自举目标却在 π 下定义；两者之间的“度量错配”使半梯度方向不再指向任何稳定不动点。

2.6.4 深度RL 的工程补救

DQN 用两个工程手段缓解死亡三角：**经验回放**（experience replay）打破样本的时间相关性，使更新更接近独立同分布；**目标网络**（target network）冻结自举目标，使半梯度更新在一段时间内成为“真正的”回归。二者都不是理论上的根治，但在Atari 等任务上极其有效。后

续的Double DQN、优先经验回放、分布式价值、NoisyNet 等改进，本质上都在同一个“稳定化”框架内做文章。

从理论上看，深度RL 的收敛性仍然是开放问题。我们能够证明的是：在表格型、或线性近似+ 在策略+ 适当步长的条件下，TD 收敛；对于非线性网络+ 离策略+ 自举，我们只有局部稳定性和经验证据。这一“理论—实践”落差是强化学习最迷人的地方之一，也是第9 章讨论“可复现性危机”的背景。

2.7 两个重要扩展

2.7.1 部分可观测马尔可夫决策过程

POMDP 在MDP 的基础上增加了观测空间 Ω 与观测核 $O(o \mid s, a)$：

$$\mathcal{M}_{\text{POMDP}} = (\mathcal{S}, \mathcal{A}, P, r, \Omega, O, \gamma, \rho_0)\,. \tag{2.46}$$

智能体看不到 S_t，只能看到 $O_t \sim O(\cdot \mid S_t, A_{t-1})$。它的信息状态是信念 $b_t(s) = \mathbb{P}(S_t = s \mid h_t)$，其中 $h_t = (O_0, A_0, \ldots, O_t)$。贝叶斯更新为

$$b_{t+1}(s') = \frac{O(o_{t+1} \mid s', a) \sum_s P(s' \mid s, a) b_t(s)}{\sum_{s''} O(o_{t+1} \mid s'', a) \sum_s P(s'' \mid s, a) b_t(s)}. \tag{2.47}$$

信念空间是单纯形，连续且维度为 $|\mathcal{S}| - 1$；POMDP 因而等价于一个信念MDP，但其状态空间连续，精确求解通常不可行。工程上常用循环网络（RNN/Transformer）把历史 h_t 编码为隐状态，作为近似的信念表示；这正是LLM 智能体与部分可观测机器人控制的共同结构。

2.7.2 平均奖励准则

当任务没有自然折扣、或 $\gamma \to 1$ 时，常用平均奖励准则：

$$g^\pi = \lim_{T \to \infty} \frac{1}{T} \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} R_{t+1} \right]. \tag{2.48}$$

在连通MDP 中，g^π 与初始状态无关。定义偏差函数 $h^\pi(s)$（bias），它满足平均奖励贝尔曼方程

$$g^\pi + h^\pi(s) = r^\pi(s) + \sum_{s'} P^\pi(s, s') h^\pi(s'). \tag{2.49}$$

平均奖励准则在排队、调度、通信网络等长期运行系统中更自然；它与折扣准则的关系可以通过“折扣因子趋于1”的极限来理解，但极限与优化的交换需要额外条件。第6 章的MCTS 与第8 章的数据中心冷却都更接近平均奖励而非折扣回报。

2.8 本章小结

本章建立了强化学习的数学骨架：

1. MDP 用六元组刻画“状态—动作—奖励—转移”，马尔可夫性使值函数成为状态上的函数，使贝尔曼递归成立；
2. 贝尔曼期望方程把策略评估写成线性方程，其解可由Neumann 级数展开；贝尔曼最优方程把最优控制写成逐状态的最大化；
3. 贝尔曼算子在本章所定义的无穷范数下是 γ-压缩映射，由Banach 不动点定理，值函数唯一且值迭代几何收敛；
4. 策略改进定理保证“对 Q^{π} 贪心”不会使策略变差，策略迭代因此在有限MDP 中有限步收敛；值迭代的近似策略误差由 $2\gamma/(1-\gamma)$ 放大；
5. 函数逼近破坏了压缩性，形成“函数逼近+ 自举+ 离策略”的死亡三角；DQN 的经验回放与目标网络是工程补救，而非理论根治；
6. POMDP 与平均奖励是两个必要的扩展：前者把“状态”换成“信念”，后者把“折扣和”换成“长期平均”。

有了这套语言，我们就可以进入算法史。第3 章将说明：TD、Q-learning、策略梯度、DQN、PPO、RLHF 等看似不同的算法，分别是在这张骨架上替换了哪一个组件——是“期望”还是“样本”，是“值函数”还是“策略”，是“在策略”还是“离策略”，是“标量奖励”还是“人类偏好”。

第3章

发展历程：五次范式转移与关键算法

3.1 为什么要用“范式转移”而不是“年表”

强化学习的教科书式历史常被写成一条年表：1957 年Bellman，1988 年TD，1989 年Q-learning，2015 年DQN，2016 年AlphaGo，2022 年RLHF，2025 年RLVR。年表有用，但它掩盖了一个更重要的问题：*为什么是这些算法在那些年份出现？它们各自解决了前一代算法的哪一个具体失效模式？*

本章把历史组织为五次**范式转移**（paradigm shift）。这里的“范式”不是Kuhn 意义上的不可通约的世界观，而是一组“默认假设+ 标准工具+ 典型失败模式”的集合。每一次转移都保留了前一代的数学骨架（贝尔曼方程或策略梯度），但改变了以下四个组件中的一个或多个：

1. **表示**：值函数或策略用表格、线性模型、深度网络还是序列模型表示；
2. **数据**：在策略采样、离策略回放、离线数据、仿真器、人类偏好还是可验证奖励；
3. **更新**：精确动态规划、随机逼近、策略梯度、信任域、最大熵或偏好优化；
4. **目标**：折扣回报、平均奖励、约束优化、KL 正则回报或可验证正确性。

用这四个组件来读历史，会发现一个反复出现的节奏：*表示能力提升 → 数据分布偏移加剧 → 更新规则需要更强的稳定性约束 → 新的目标函数把约束吸收进去*。DQN 的经验回放、TRPO 的KL 约束、PPO 的裁剪、RLHF 的KL 惩罚、DPO 的闭式消元，都是这条节奏在不同年代的回声。

3.2 史前史与奠基（1930s—1957）

在Bellman 之前，“序贯决策”已经是经济学、控制论与统计学的共同问题。von Neumann 与Morgenstern 的博弈论（1944）给出了策略互动的形式化；Wiener 的控制论（1948）把“反馈”确立为智能行为的核心隐喻；Shannon 的信息论则提供了“不确定性”的度量。但真正为强化学习提供可计算骨架的，是Bellman 对多阶段决策的递归分解。

Bellman 的贡献常被简化为“一个方程”，实际上包含三个层次。第一，他证明了在相当一般的条件下，多阶段决策问题可以分解为逐阶段的递归问题（最优性原理）。第二，他给出了求解该递归的数值方法（值迭代与策略迭代）。第三，他把“折扣”引入目标函数，使无穷时域问题在数学上良定义。这三层共同构成了第2 章的数学基础。

与此同时，心理学与早期AI 提供了另外两块拼图。Thorndike 的效果律（1911）与Skinner 的操作性条件反射给出了“行为由后果塑造”的实证传统；Minsky 的SNARC（1954）与Samuel 的跳棋程序（1959）则证明了机器可以通过自我对弈与强化信号改善行为。值得注意的是，Samuel 的程序已经包含了“用最终胜负作为评价信号”的思想，这与现代RL 的“延迟奖励”高度一致；但它仍依赖人工设计的局面特征，因而没有触及“表示学习”这一维度。

3.3 第一范式：表格型时序差分与收敛性（1957—1989）

第一范式的核心问题是：**在模型未知、状态空间可枚举的条件下，如何保证学习算法收敛到最优？**这一时期的算法以表格型值函数为主，理论上追求“几乎必然收敛”，工程上则受限于状态维度。

3.3.1 从动态规划到时序差分

动态规划要求已知 P 与 r。若未知，最直接的做法是“采样—平均”：用蒙特卡洛回报 G_t 估计 $V^\pi(s)$。蒙特卡洛估计无偏，但方差大，且必须等到回合结束才能更新。TD(0) 用“即时奖励+ 下一状态估计”替代完整回报：

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right]. \tag{3.1}$$

括号中的量就是TD 误差 δ_t。它是有偏的（因为 $V(S_{t+1})$ 本身是估计），但方差远小于蒙特卡洛，且可以在线更新。Sutton 在1988 年系统分析了TD 的预测性质，并证明在适当条件下它收敛到 V^π[8]。TD 的提出标志着强化学习从“动态规划的采样版”变成了一个独立的学习范式：*用估计更新估计*（bootstrapping）。

3.3.2 Q-learning 与离策略控制

TD(0) 只解决“策略评估”。要把评估变成控制，需要比较不同动作的价值。Watkins 在1989 年的博士论文中提出了Q-learning[9]：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right]. \tag{3.2}$$

它与TD(0) 的关键差别在于 $\max_{a'}$：更新目标直接使用下一状态的最优动作价值，而不是当前策略的动作价值。这使得Q-learning 成为**离策略**（off-policy）算法：行为策略可以是任意探索策略（如 ε-贪心），而学习目标是 π^*。Watkins 与Dayan 在1992 年给出了在适当步长与遍历性条件下的收敛性证明[10]。

同一时期，SARSA 作为在策略版本出现：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]. \tag{3.3}$$

SARSA 与Q-learning 的差别看似只是一个 max，实则决定了算法在“悬崖行走”等风险任务中的行为：Q-learning 会学到沿悬崖边缘的最优路径（因为它的目标策略是最优策略），而 SARSA 会学到更保守的路径（因为它评估的是带探索的行为策略）。这一差异是“离策略偏差”最早的直观案例。

3.3.3 Actor–Critic 与REINFORCE

值函数方法之外，另一条线索是直接优化策略。Barto、Sutton 与Anderson 在1983 年提出了actor–critic 结构：actor 更新策略参数，critic 估计值函数并提供TD 误差作为“优势”信号[7]。这一结构把“策略”与“评价”分离，成为后来A2C、PPO、SAC 的共同祖先。

Williams 在1992 年提出了REINFORCE 算法[12]，它用似然比技巧估计策略梯度：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, G_t\right]. \tag{3.4}$$

REINFORCE 的优雅之处在于它不需要环境可微，也不需要值函数；但它的方差极大，且对步长敏感。降低方差成为此后二十年策略梯度研究的主线：基线、优势函数、资格迹、自然梯度、信任域，都是围绕这一目标展开的。

3.3.4 TD-Gammon：第一次“深度”强化学习

1992 年，Tesauro 的TD-Gammon 用TD(λ) 训练一个多层感知机下西洋双陆棋，达到了接近人类专家的水平[13]。它常被视为深度强化学习的先驱，因为它同时使用了非线性函数逼近、自博弈与TD 学习。但TD-Gammon 的成功很大程度上依赖于西洋双陆棋的随机性与“局面评估”的可学习性；同样的方法在象棋、围棋上并未复现，原因包括状态空间更大、奖励更稀疏、以及函数逼近的不稳定。这一“局部成功、难以推广”的经验，预示了第二范式的核心矛盾。

3.4 第二范式：函数逼近与策略梯度（1989—2012）

第二范式的核心问题是：**当状态连续或维度很高、表格无法存储时，如何用函数逼近保持学习稳定？**这一时期的主题从“收敛性证明”转向“近似误差、方差与稳定性”。

3.4.1 函数逼近的理论困难

把TD 与线性函数逼近结合，会得到半梯度更新（第2 章式(2.44)）。Tsitsiklis 与Van Roy 在1997 年证明：在在策略采样、线性逼近、步长满足Robbins–Monro 条件的假设下，TD 收敛到投影贝尔曼方程的解[21]。但若采样分布与目标策略不匹配（离策略），更新可能发散。Baird 的反例与“死亡三角”的提出，使研究者意识到：*函数逼近不是把表格算法“套上”一个模型那么简单，它改变了算子的几何性质。*

这一时期出现了多种稳定化思路：残差梯度（residual gradient）直接对贝尔曼残差做梯度下降，避免了半梯度的目标依赖；最小二乘TD（LSTD）用闭式解替代迭代；梯度TD（GTD）用双时间尺度更新保证收敛。这些方法在理论上优雅，但在高维非线性场景中计算昂贵，未能成为主流。

3.4.2 策略梯度定理与自然梯度

策略梯度方向的另一条主线在2000 年前后成熟。Sutton、McAllester、Singh 与Mansour 在2000 年给出了策略梯度定理的一般形式[15]：

$$\nabla_\theta J(\theta) = \mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, Q^{\pi_\theta}(s,a)\right]. \tag{3.5}$$

这一定理把“对轨迹分布求导”转化为“对动作对数概率求导”，使策略梯度可以在线估计。它的证明将在第5 章完整给出。

2001 年，Kakade 把Amari 的自然梯度引入策略搜索[16]。普通梯度在参数空间取欧氏度量，而自然梯度在分布空间取Fisher 信息度量：

$$\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1}\nabla_\theta J(\theta), \qquad F(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s)^\top\right]. \tag{3.6}$$

自然梯度对策略参数化具有不变性：重新参数化策略不会改变更新方向在分布空间中的几何意义。这一思想后来直接催生了TRPO 与PPO 的“信赖域”视角。

3.4.3 探索、选项与搜索

同一时期，探索与分层也是热点。UCB（第2 章）为老虎机提供了乐观探索的严格框架；E3、R-max 等算法为MDP 的样本复杂度提供了理论保证；Sutton、Precup 与Singh 在1999 年提出“选项”（options）框架，把时序抽象引入RL[14]。这些工作在当时更多是理论兴趣，但它们在2010 年代以“分层RL”“技能发现”“选项批判”等形式重新回到主流。

搜索方面，UCT 在2006 年把UCB 与蒙特卡洛树搜索结合[18]，为高分支因子的博弈提供了可行的在线规划算法。UCT 后来成为AlphaGo 的骨架之一，但在2006 年，它还没有与深度学习结合。

3.4.4 第二范式的遗产与局限

第二范式留下了三项遗产：策略梯度定理、自然梯度/信赖域思想、以及“探索需要显式机制”的认识。它的局限也很明显：

- 线性函数逼近与手工特征无法处理原始像素、语音、文本等高维感知输入；
- 策略梯度的方差随horizon 增长，样本效率低；
- 离策略+ 非线性逼近的稳定性缺乏理论保证；

- 大多数成功案例仍依赖领域知识设计的特征与奖励。

这些局限在2013 年之后被深度表示、大规模并行采样与更强的稳定性算法逐一突破。

3.5 第三范式：深度强化学习与稳定性（2013—2017）

第三范式的核心问题是：**如何让RL 直接处理高维感知输入，并在非线性函数逼近下保持稳定？**这一时期的标志性事件是DQN 在Atari 上的成功，以及AlphaGo 在围棋上的突破。

3.5.1 DQN：经验回放与目标网络

2013 年，Mnih 等人用深度Q 网络（DQN）在Atari 2600 的多个游戏上达到人类水平[23]；2015 年的Nature 版本进一步展示了49 个游戏上的系统性结果[24]。DQN 的核心不是“用神经网络替代表格”，而是两个稳定化技巧：

1. **经验回放**：把转移 $(S_t, A_t, R_{t+1}, S_{t+1})$ 存入缓冲区，训练时随机采样小批量。这打破了样本的时间相关性，使SGD 的独立同分布假设近似成立，并提高了数据复用率。
2. **目标网络**：用一份延迟更新的参数 θ^- 计算自举目标

$$y_t = R_{t+1} + \gamma \max_{a'} Q_{\theta^-}(S_{t+1}, a'), \tag{3.7}$$

从而在若干步内把“目标”固定，避免“追逐自己尾巴”的发散。

DQN 的损失为

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\right)^2\right]. \tag{3.8}$$

它在49 个Atari 游戏上以同一套网络结构与超参数达到了接近或超过人类专家的得分，但这一比较忽略了样本效率：DQN 需要数千万帧，而人类只需几分钟。这一“得分超越、效率落后”的对比，成为后续研究的核心驱动力之一。

3.5.2 从A3C 到PPO：策略优化的工程化

DQN 之后，策略梯度方法迅速工程化。2016 年，Mnih 等人提出A3C（Asynchronous Advantage Actor–Critic），用多线程异步采样打破相关性，并在连续动作任务上取得了良好效果[25]。A3C 的同步版本A2C 更易复现，成为后来的基线。

2015 年，Schulman 等人提出TRPO（Trust Region Policy Optimization）[26]，用KL 散度约束策略更新：

$$\max_\theta \ \mathbb{E}\left[\frac{\pi_\theta(a \mid s)}{\pi_{\theta_{\text{old}}}(a \mid s)} A^{\pi_{\theta_{\text{old}}}}(s, a)\right] \quad \text{s.t.} \quad \mathbb{E}\left[\mathrm{KL}\left(\pi_{\theta_{\text{old}}} \,\|\, \pi_\theta\right)\right] \le \delta. \tag{3.9}$$

TRPO 的动机是“单调改进下界”：只要新旧策略的KL 足够小，surrogate 目标的提升就能保证真实目标的提升。它的理论优雅，但实现复杂（需要共轭梯度与线搜索）。2017 年，PPO 用裁剪目标替代显式KL 约束[27]：

$$\mathcal{L}^{\mathrm{CLIP}}(\theta) = \mathbb{E}\left[\min\left(\rho_t(\theta)A_t,\ \mathrm{clip}\left(\rho_t(\theta), 1-\epsilon, 1+\epsilon\right)A_t\right)\right], \tag{3.10}$$

其中 $\rho_t(\theta) = \pi_\theta(A_t \mid S_t)/\pi_{\theta_{\mathrm{old}}}(A_t \mid S_t)$。PPO 以极低的实现成本获得了接近TRPO 的稳定性，成为此后多年最常用的策略优化算法，也是RLHF 的第一代主力。

3.5.3 AlphaGo 与AlphaZero：搜索与学习的结合

2016 年，AlphaGo 以4:1 击败李世石[35]。它的核心是“策略网络+ 价值网络+ 蒙特卡洛树搜索”：策略网络缩小搜索宽度，价值网络评估叶节点，MCTS 在两者之上做在线规划。2017 年的AlphaGo Zero 去掉了人类棋谱，仅用自我对弈与MCTS 从零训练[36]；2018 年的AlphaZero 把同一框架推广到国际象棋与将棋[37]。

AlphaGo 的意义超出了围棋。它证明了*学习与搜索可以互补*：学习提供先验与评估，搜索提供在线修正；两者结合可以在巨状态空间中达到超人类水平。这一思想后来被MuZero（学习环境模型）、AlphaTensor（发现矩阵乘法算法）、AlphaDev（发现排序算法）继承，并在2024—2025 年的“推理模型”中以“测试时搜索/采样”的形式再次出现。

3.6 第四范式：模型基、离线学习与对齐（2017—2021）

第四范式的核心问题是：**如何提高样本效率、利用历史数据、并把RL 扩展到“奖励难以手写”的任务？**这一时期的四条主线——模型基RL、离线RL、分布型RL、RLHF——看似分散，实则都在回答“数据从哪来、如何复用”。

3.6.1 模型基RL 与世界模型

模型基RL 先学习环境模型 $\hat{P}(s' \mid s,a)$ 与 $\hat{r}(s,a)$，再用模型做规划或生成虚拟样本。Dyna（1990）已经提出了“真实经验+ 模型模拟”的混合框架[11]；PILCO（2011）用高斯过程建模并做概率规划；Dreamer 系列（2019—2023）用隐空间世界模型训练策略[32]。2020 年，MuZero 在没有显式游戏规则的情况下，学习“隐式模型”并在围棋、象棋、将棋与Atari 上达到超人类水平[34]。

模型基方法的优势是样本效率；风险是“模型偏差”（model bias）：策略会利用模型的错误，即“模型劫持”。DreamerV3 在2025 年登上Nature，展示了单一超参数在150 多个任务上的稳健性[33]，这是模型基RL 走向通用性的重要一步。

3.6.2 离线RL：从历史数据中学习

离线RL（offline RL）假设只能访问固定数据集 $\mathcal{D}$，不能与环境交互。这消除了试错成本，但也带来了**分布偏移**：策略会倾向于选择数据集中不存在、但价值估计被高估的动作。BCQ（2019）、CQL（2020）、IQL（2021）是三条代表性路线[45, 46, 47]：

- **BCQ**：限制策略只选择数据集中出现过的动作，或用生成模型逼近行为策略的支持集；
- **CQL**：在损失中加入保守正则项，压低数据分布外动作的Q 值；
- **IQL**：完全避免显式评估OOD 动作，用分位数回归拟合“在数据分布内的最优值”。

离线RL 在医疗、机器人、推荐等“在线试错代价高”的领域有天然吸引力。但它对数据覆盖、保守程度与评估方法极为敏感；一个在离线基准上表现良好的算法，可能在部署时因分布偏移而崩溃。这一“离线—在线”落差是第9 章讨论的重点之一。

3.6.3 分布型RL 与最大熵RL

分布型RL 不再只估计回报的期望，而是估计回报的完整分布 $Z^\pi(s,a)$，其贝尔曼方程在分布层面成立。C51（2017）、QR-DQN（2018）、IQN（2018）等算法在Atari 上取得了稳定提升。分布型视角的价值在于：它区分了“均值相同但风险不同”的策略，为风险敏感决策提供了工具。

最大熵RL 则从另一侧改进了探索。SAC（2018）最大化“回报+ 策略熵”：

$$J(\pi) = \mathbb{E}\left[\sum_t \gamma^t \left(R_{t+1} + \alpha \mathcal{H}\left(\pi(\cdot \mid S_t)\right)\right)\right], \tag{3.11}$$

并用软策略迭代求解[31]。SAC 在连续控制基准上样本效率高、稳定性好，成为机器人控制的主流基线之一。TD3（2018）则通过双critic、延迟更新与目标策略平滑，缓解了DDPG 的Q 值过估计[30]。

3.6.4 RLHF：把人类偏好变成奖励

2017 年，Christiano 等人提出了“从人类偏好中学习奖励”的框架[48]：让人类比较两条轨迹，训练一个奖励模型，再用RL 优化策略。这一框架在2020 年的摘要任务与2022 年的InstructGPT 中被系统化[49, 50]。InstructGPT 的关键结果是：一个1.3B 参数的RLHF 模型，其输出在人类偏好评估中优于175B 的GPT-3[50]。这一结果震动了业界，因为它说明对齐数据与优化目标可以在参数量小两个数量级的情况下显著改变模型的有用性。

RLHF 把强化学习的目标从“环境奖励”扩展为“人类偏好”，从而把RL 的应用边界从游戏、机器人推向了语言。但它也引入了新的问题：奖励模型是对偏好的近似，策略会利用近似误差（reward hacking）；人类偏好本身有噪声、有偏、且随群体而异。第7 章将给出RLHF 的完整数学推导，并讨论这些问题的缓解方法。

3.7 第五范式：偏好、可验证奖励与推理智能（2022—2026）

第五范式的核心问题是：**当奖励来自人类偏好或可验证规则、当动作空间是自然语言、当任务需要长程推理时，经典RL 算法需要怎样改造？**这一时期的三个关键词是RLHF、RLVR 与Agentic RL。

3.7.1 RLHF 的规模化与DPO

2022 年11 月ChatGPT 的发布，使RLHF 从论文走向产品。其标准流程是：监督微调（SFT）→ 奖励建模（RM）→ PPO 优化。2023 年，Rafailov 等人提出DPO（Direct Preference Optimization），证明了KL 正则RLHF 目标的最优策略可以用奖励模型闭式表示，从而把“训练奖励模型+ PPO”简化为“直接在偏好对上做分类”[51]。DPO 的损失为

$$\mathcal{L}_{\mathrm{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w\mid x)}{\pi_{\mathrm{ref}}(y_w\mid x)} - \beta\log\frac{\pi_\theta(y_l\mid x)}{\pi_{\mathrm{ref}}(y_l\mid x)}\right)\right]. \tag{3.12}$$

DPO 去掉了显式奖励模型与在线采样，工程上更简单、更稳定，迅速成为开源模型对齐的主流方法。但DPO 也有代价：它依赖离线偏好数据，缺少在线探索；当偏好数据覆盖不足时，DPO 可能学到“表面偏好”而非“真实能力”。这一局限在2024—2025 年催生了KTO、ORPO、SimPO 等变体，以及“在线DPO”等混合方法。

3.7.2 GRPO 与RLVR

2024 年，DeepSeek 团队在DeepSeekMath 中提出GRPO（Group Relative Policy Optimization）[52]。GRPO 去掉了PPO 的价值网络，改用同一问题下多个采样回答的组内相对奖励作为优势：

$$\hat{A}_i = \frac{r_i - \mathrm{mean}(r_1,\ldots,r_G)}{\mathrm{std}(r_1,\ldots,r_G)}, \tag{3.13}$$

其中 r_i 是第 i 个回答的奖励。这一设计大幅降低了LLM 训练的内存开销（无需与策略同规模的价值网络），并使“可验证奖励”（verifiable reward）成为可能：数学题可以检查最终答案，代码题可以运行单元测试，逻辑题可以比对形式化证明。

2025 年1 月，DeepSeek-R1 展示了“纯RL 激励推理”的路径[53]：R1-Zero 直接从基座模型出发，用规则奖励（答案正确性+ 格式）做GRPO，模型自发出现了长链推理、自我验证与反思；R1 则在此基础上加入冷启动数据与多阶段训练，进一步提升了可读性与稳定性。R1 在AIME 2024 上达到79.8% 的pass@1，在MATH-500 上达到97.3%，在Codeforces 上达到2029 的评分[53]。这项工作在2025 年9 月以封面文章形式发表于Nature[54]，成为RLVR 的标志性事件。

RLVR 的核心创新不是“一个新的策略梯度公式”，而是**奖励来源的结构性变化**：奖励不再是人类偏好模型，而是可自动验证的规则。这带来了三个结果：奖励更客观、可大规模生成；训练可以完全在线、无需人类标注；模型的行为更容易被“正确性”而非“讨喜度”驱动。但它

也带来了新的风险：模型可能学会“迎合验证器”而非“真正推理”（reward hacking 的变体），且在不可验证的开放任务上泛化有限。

3.7.3 推理模型与测试时扩展

2024 年9 月，OpenAI 发布o1 系列，展示了“长思维链+ 强化学习”在数学、代码与科学问题上的显著提升；2024 年12 月，o3 在ARC-AGI 上取得87.5% 的成绩（高计算预算下）[61]。这一路线与DeepSeek-R1 相互印证：*RL 可以教会模型“如何思考”，而不仅仅是“说什么”。*

与之相伴的是“测试时计算”（test-time compute）的兴起：在推理阶段采样多条轨迹、用搜索或验证器选择最优答案。Snell 等人证明，在固定计算预算下，“更长的采样”与“更好的验证器”可以显著优于单纯增大模型[60]。这与AlphaGo 的“学习+ 搜索”结构高度同构：训练时学习先验，推理时用搜索/采样修正。

3.7.4 Agentic RL：从单步生成到长程决策

2025—2026 年，强化学习的应用重心从“单轮问答”转向“智能体”：模型需要调用工具、浏览网页、操作软件、在多轮交互中完成长程任务。Agentic RL 把LLM 的策略建模为部分可观测马尔可夫决策过程（POMDP）：状态是环境与对话历史，动作是工具调用或文本输出，奖励是任务完成度、工具返回结果或人类反馈[55]。

这一转向带来了新的技术问题：

- **长程信用分配**：一条轨迹可能包含数十次工具调用，最终成功/失败如何归因到具体步骤？
- **环境规模**：需要可复现、可验证、足够多样化的交互环境（浏览器、操作系统、代码仓库、游戏）；
- **奖励设计**：稀疏的成功信号、密集的过程奖励、以及“过程奖励被劫持”的风险；
- **训练稳定性**：多轮rollout 的方差、工具调用的不可微、以及策略与环境的共同演化。

2026 年的代表性工作开始从“单步RLVR”走向“步骤级/回合级”优化：StepPO 等工作把信用分配细化到步骤级别[56]；课程RL 试图让模型在训练中逐步接触更难的任务[57]；大规模“零RL”实验则探索纯RL 在万亿参数模型上的可扩展性[58]。这些工作尚未形成统一范式，但它们共同指向一个结论：*智能体RL 的瓶颈不在策略梯度公式，而在环境、奖励与信用分配的基础设施。*

3.8 算法进步的五个机制

回顾五次范式转移，可以提炼出五个反复出现的创新机制。理解这五个机制，比记住算法名字更有用。

机制一：偏差—方差权衡。 从蒙特卡洛（无偏、高方差）到TD（有偏、低方差），从REINFORCE（高方差）到actor–critic（用critic 降低方差），从单步TD 到TD(λ)/GAE（用资格迹/指数加权在偏差与方差之间连续调节），再到GRPO（用组内基线替代critic），每一次都在调整“用多少未来信息来估计当前价值”。第4、5 章将给出这些方法的统一推导。

机制二：策略更新的稳定性。 从普通梯度到自然梯度，从自然梯度到TRPO 的KL 约束，从TRPO 到PPO 的裁剪，从PPO 到RLHF 的KL 惩罚，核心都是“限制一步更新走多远”。这一机制的理论基础是“策略改进下界”：只有当新旧策略足够接近时，surrogate 目标的提升才能转化为真实目标的提升。

机制三：数据复用与分布校正。 从在策略采样到经验回放，从离策略Q-learning 到Retrace/IMPALA，从离线RL 的保守正则到DPO 的闭式偏好优化，核心都是“如何用旧数据/别人的数据学习目标策略”。数据复用提高了样本效率，但引入了分布偏移；所有校正方法都在“利用旧数据”与“避免外推错误”之间权衡。

机制四：搜索与学习的结合。 从UCT 到AlphaGo/AlphaZero，从MuZero 到测试时采样，核心都是“用搜索弥补先验的不足，用学习缩小搜索的宽度”。这一机制在2024—2026 年的推理模型中被重新发现：训练时的RL 教会模型“往哪里想”，推理时的采样/验证教会模型“多想几步”。

机制五：奖励来源的扩展。 从手工奖励到人类偏好，从人类偏好到可验证规则，从规则到环境反馈，核心都是“如何定义一个既真实又可优化的目标”。每一次扩展都打开了新的应用空间，也带来了新的劫持风险。第7 章将说明：RLHF 的Bradley–Terry 模型、DPO 的闭式解与GRPO 的组内优势，本质上都是在“奖励不可直接观测”的条件下做逆问题求解。

3.9 里程碑年表

表3.1 给出1957—2026 年的关键节点。表中“创新”一列强调该工作改变了哪一个组件，“局限/后续”一列强调它留下的问题；这两列比“年份—算法”更有信息量。

表 3.1 强化学习关键里程碑（1957—2026）

年份	算法/事件	关键创新	局限或后续
1957	动态规划 / MDP	最优性原理、Bellman 方程、值迭代与策略迭代	需要已知模型；状态空间需可枚举
1959	Samuel 跳棋	自我对弈、线性评估、rote learning	依赖人工特征；无收敛保证

年份	算法/事件	关键创新	局限或后续
1983	Actor–Critic	策略与价值分离，TD 误差作为优势信号	非线性逼近下不稳定
1988	TD(0)	用自举目标在线估计值函数	仅策略评估；方差—偏差需调节
1989/1992	Q-learning	离策略、直接逼近最优动作价值（1989 博士论文提出，1992 期刊版并给出收敛性证明）	最大化偏差；函数逼近下发散
1992	REINFORCE	似然比策略梯度，无需环境模型	方差极大；样本效率低
1992/1995	TD-Gammon	非线性函数逼近+ 自博弈+ TD(λ)	难以推广到围棋等任务
1999	Options 框架	时序抽象、技能复用	选项发现困难；理论复杂
2000	策略梯度定理	将轨迹导数化为动作对数概率导数	仍需大量样本；方差高
2001	自然梯度 / 策略搜索	Fisher 信息度量、参数化不变性	计算Fisher 矩阵昂贵
2006	UCT / MCTS	UCB 与树搜索结合，在线规划	依赖模拟器；高维感知弱
2013/2015	DQN	经验回放、目标网络、端到端像素输入	样本效率低；离策略不稳定
2015	TRPO	KL 约束下的单调改进下界	实现复杂；二阶优化昂贵
2015	GAE	指数加权优势估计，偏差—方差连续调节	需要critic；超参数敏感
2016	A3C / A2C	异步/同步并行采样	大规模分布式工程复杂
2016	AlphaGo	策略/价值网络+ MCTS + 自我对弈	依赖人类棋谱与领域知识
2017	PPO	裁剪替代KL 约束，简单稳定	非单调保证；裁剪机制有争议
2017	AlphaZero	无人类数据，纯自我对弈+ MCTS	需要完美模拟器；计算昂贵
2017	RLHF（Christiano）	人类偏好训练奖励模型	奖励劫持；偏好噪声
2018	SAC / TD3	最大熵、双critic、延迟更新	连续控制为主；离散动作需适配

年份	算法/事件	关键创新	局限或后续
2019	AlphaStar	多智能体、联盟训练、星际争霸	训练规模巨大；泛化有限
2019/2020	BCQ / CQL / IQL	离线RL 的保守正则与OOD 动作抑制	数据覆盖敏感；评估困难
2020	MuZero	学习隐式模型+ 树搜索	模型偏差；训练复杂
2021	Decision Transformer	把RL 建模为条件序列生成	非真正在线决策；长期信用分配弱
2022	InstructGPT / ChatGPT	RLHF 规模化，1.3B 优于175B GPT-3	人类偏好昂贵；对齐税
2023	DPO	闭式消去奖励模型，偏好对直接优化	离线数据；缺少在线探索
2024	GRPO / DeepSeek-Math	组内相对优势，去掉critic	组大小敏感；奖励可验证性限制
2024	o1 / o3	长思维链+ RL，测试时计算扩展	推理成本高；可解释性差
2025	DeepSeek-R1	纯RL 激励推理，RLVR 范式	开放性任务泛化有限；奖励劫持
2025/2026	Agentic RL	POMDP、工具调用、长程信用分配	环境/奖励基础设施不成熟

3.10 本章小结

强化学习的历史不是一串孤立的算法，而是一部“表示—数据—更新—目标”四组件协同演化的历史。第一范式用表格与TD 解决了“无模型收敛”；第二范式用函数逼近与策略梯度解决了“连续状态”；第三范式用深度表示与稳定化技巧解决了“高维感知”；第四范式用模型、离线数据与人类偏好解决了“样本效率与目标定义”；第五范式用可验证奖励与智能体环境把RL 推向了“推理与行动”。

每一次转移都留下了未解问题：第一范式的维度灾难、第二范式的死亡三角、第三范式的样本效率、第四范式的分布偏移、第五范式的奖励劫持与环境规模。这些问题将在第4—7 章的技术推导与第8—9 章的应用评估中反复出现。下一章先从值函数方法开始，给出TD、SARSA、Q-learning、Double Q-learning、DQN 与分布型RL 的严格推导。

第 4 章

值函数方法的严格推导：从 TD 到分布式 RL

本章从贝尔曼期望方程出发，推导表格型与时序差分方法，随后进入深度值函数逼近。贯穿全章的问题是：**当我们只有采样、没有模型时，如何构造一个收敛到贝尔曼不动点的随机逼近算法？**我们将看到，TD、SARSA、Q-learning 的差别可以用“估计哪个算子、用哪个策略采样、是否对目标取最大”三个维度完全刻画；DQN 则是把Q-learning 的更新放进“经验回放+目标网络”的工程稳定器；分布式RL 则把“期望”升级为“分布”。

4.1 TD(0)：贝尔曼期望方程的随机逼近

4.1.1 从贝尔曼方程到更新规则

固定策略 π，贝尔曼期望方程为

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[R_{t+1} + \gamma V^{\pi}(S_{t+1}) \mid S_t = s\right]. \tag{4.1}$$

若已知模型，可用动态规划迭代 $V_{k+1} = \mathcal{T}^{\pi} V_k$。若未知模型，但可以采样，则最自然的想法是：用一次观测 $R_{t+1} + \gamma V(S_{t+1})$ 作为方程右端的无偏样本，把“精确期望”替换为“随机样本”，把“同步更新所有状态”替换为“只更新被访问的状态”。

设当前估计为 V，定义TD 误差

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t). \tag{4.2}$$

TD(0) 更新为

$$V(S_t) \leftarrow V(S_t) + \alpha\, \delta_t, \tag{4.3}$$

其中 $\alpha \in (0, 1]$ 是步长。若把 V 看作参数向量 θ（表格型时 $\theta_s = V(s)$），则上式等价于

$$\theta \leftarrow \theta + \alpha\, \delta_t \nabla_{\theta} V_{\theta}(S_t). \tag{4.4}$$

对表格型，$\nabla_{\theta} V_{\theta}(S_t)$ 是第 S_t 个分量为 1、其余为 0 的基向量，故只有被访问状态被更新。

4.1.2 它优化的是什么目标?

TD(0) 并不是任何固定目标函数的精确梯度。若定义均方TD 误差

$$\mathcal{L}(\theta) = \frac{1}{2}\mathbb{E}\left[\left(R_{t+1} + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right)^2\right], \tag{4.5}$$

对 θ 求导时，目标项 $R_{t+1} + \gamma V_\theta(S_{t+1})$ 也依赖 θ。若把目标视为常数（stop-gradient），得到

$$\nabla_\theta \mathcal{L} \approx -\mathbb{E}\left[\delta_t \nabla_\theta V_\theta(S_t)\right], \tag{4.6}$$

故式(4.4) 是半梯度下降。这个“不完整”的梯度正是TD 的特点：它把自举目标当作“临时的真值”，从而避免了需要完整回报的等待。

定理4.1 (表格型TD(0) 的收敛性)**.** *设每个状态被访问无穷多次，步长满足*

$$\sum_{t=0}^{\infty} \alpha_t = \infty, \qquad \sum_{t=0}^{\infty} \alpha_t^2 < \infty, \tag{4.7}$$

奖励有界。则表格型TD(0) 以概率 1 收敛到 V^π。

证明思路 把更新写成随机逼近形式

$$V_{t+1}(s) = V_t(s) + \alpha_t\left[\mathcal{T}^\pi V_t(s) - V_t(s) + \xi_t(s)\right], \tag{4.8}$$

其中 $\xi_t(s)$ 是采样噪声，条件期望为零。对应的常微分方程为

$$\dot{V}(s) = \mathcal{T}^\pi V(s) - V(s). \tag{4.9}$$

由于 $\mathcal{T}^\pi$ 是 γ-压缩映射，V^π 是唯一全局吸引子：取Lyapunov 函数 $\|V - V^\pi\|_\infty$，有

$$\frac{\mathrm{d}}{\mathrm{d}t}\|V - V^\pi\|_\infty \le -(1-\gamma)\|V - V^\pi\|_\infty < 0. \tag{4.10}$$

由随机逼近理论（Robbins–Monro 条件保证噪声被平均掉），$V_t \to V^\pi$ 几乎必然。完整证明需验证噪声的鞅差性质与有界性，见Tsitsiklis[20]。 □

这个证明揭示了一个重要事实：TD 的收敛性来自贝尔曼算子的压缩性，而随机逼近只负责“把期望换为样本”。只要算子仍是压缩的，算法就有稳定的漂移方向；步长条件保证漂移不被噪声淹没。

4.1.3 偏差与方差：TD 与蒙特卡洛的对偶

蒙特卡洛估计用完整回报 G_t：

$$V(S_t) \leftarrow V(S_t) + \alpha\,(G_t - V(S_t))\,. \tag{4.11}$$

它无偏（$\mathbb{E}[G_t \mid S_t = s] = V^\pi(s)$），但方差随回合长度增长。TD(0) 用 $R_{t+1} + \gamma V(S_{t+1})$，方差小，但有偏（因为 $V(S_{t+1})$ 是估计）。n-步回报提供了连续的折中：

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n}). \tag{4.12}$$

$n = 1$ 为TD(0)，$n \to \infty$ 为蒙特卡洛。偏差随 n 减小，方差随 n 增大。这一“偏差—方差前沿”是第5 章GAE 与TD(λ) 的共同基础。

4.2 TD(λ) 与资格迹

4.2.1 前向视角：λ-回报

把不同 n 的 n-步回报做几何加权，得到 λ-回报：

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}, \qquad \lambda \in [0,1]. \tag{4.13}$$

权重 $(1-\lambda)\lambda^{n-1}$ 归一化为 1。当 $\lambda = 0$ 时只剩 $G_t^{(1)}$（TD(0)）；当 $\lambda = 1$ 时极限为蒙特卡洛回报。λ 因而是一个连续的“记忆长度”旋钮。

前向视角的更新为

$$V(S_t) \leftarrow V(S_t) + \alpha \left(G_t^\lambda - V(S_t)\right). \tag{4.14}$$

它的概念清晰，但需要等到未来很多步才能计算 G_t^λ，不适合在线学习。

4.2.2 后向视角：资格迹

资格迹（eligibility trace）为每个状态维护一个“最近被访问的程度”：

$$E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbf{1}\{S_t = s\}, \qquad E_{-1}(s) = 0. \tag{4.15}$$

每一步用当前TD 误差更新所有状态：

$$V(s) \leftarrow V(s) + \alpha\,\delta_t\,E_t(s), \qquad \forall s \in \mathcal{S}. \tag{4.16}$$

直觉是：当前TD 误差不仅归因于当前状态，也归因于最近访问过、且与当前状态“时序接近”的状态；λ 控制这种回溯的长度。

定理4.2 (前向—后向等价). *在回合式任务中，若所有更新按时间顺序累积，且 V 在回合内不变，则后向更新(4.16) 的总和等于前向更新(4.14) 的总和。*

证明思路 设回合在 T 结束。把后向更新中状态 s 的总改变量写成

$$\sum_{t=0}^{T-1} \alpha\, \delta_t\, E_t(s). \tag{4.17}$$

展开 $E_t(s) = \sum_{k=0}^{t} (\gamma\lambda)^{t-k} \mathbf{1}\{S_k = s\}$，交换求和次序：

$$\sum_{k:S_k=s} \sum_{t=k}^{T-1} \alpha\, \delta_t (\gamma\lambda)^{t-k}. \tag{4.18}$$

括号内的和正是 λ-回报与当前估计之差（对 T 截断的版本）：

$$\sum_{t=k}^{T-1} \delta_t (\gamma\lambda)^{t-k} = G_k^{\lambda} - V(S_k). \tag{4.19}$$

故总更新等于 $\sum_{k:S_k=s} \alpha (G_k^{\lambda} - V(S_k))$，即前向更新。截断误差随 T 指数衰减。 □

TD(λ) 的意义在于：它把“看多远”从算法结构变成了一个连续超参数，并以 $O(|\mathcal{S}|)$ 的额外内存实现在线计算。这一思想后来在GAE 中以“优势函数的指数加权”形式重现。

4.3 SARSA：在策略控制

TD(0) 只做策略评估。要把评估变成控制，需要把状态值换成动作值。SARSA 的更新为

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right], \tag{4.20}$$

其中 A_{t+1} 由当前行为策略（如 ε-贪心）采样。名字SARSA 来自五元组 $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$。

SARSA 优化的是当前行为策略 π 的动作值 Q^{π}，因此是**在策略**（on-policy）算法。若策略是 ε-贪心且 ε 随训练衰减（GLIE：greedy in the limit with infinite exploration），则在适当步长下SARSA 收敛到最优策略。其收敛性证明与TD(0) 类似，只是算子变为 $\mathcal{T}^{\pi}$ 的动作值版本。

SARSA 的一个有趣性质是“风险敏感”：在悬崖行走任务中，它倾向于绕开悬崖，因为它的更新使用了带探索的行为策略；而Q-learning 会学到贴着悬崖的最优路径。这不是bug，而是目标不同：SARSA 评估的是“你实际执行的策略”，Q-learning 评估的是“最优策略”。

4.4 Q-learning：离策略最优控制

4.4.1 算法与算子

Q-learning 的更新为

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q(S_{t+1}, a') - Q(S_t, A_t) \right]. \tag{4.21}$$

它直接逼近贝尔曼最优算子 $\mathcal{T}^*$ 的不动点 Q^*。与SARSA 的差别仅在于目标中的 $Q(S_{t+1}, A_{t+1})$ 被替换为 $\max_{a'} Q(S_{t+1}, a')$。这一替换使行为策略与目标策略解耦：无论用什么策略采样（只要覆盖所有状态—动作对），学习目标都是最优策略。这就是**离策略**（off-policy）的含义。

定理4.3 (Q-learning 的收敛性). *设每个状态—动作对被访问无穷多次，步长满足式(4.7)，奖励有界。则表格型Q-learning 以概率 1 收敛到 Q^*。*

证明思路 写成随机逼近形式

$$Q_{t+1}(s,a) = Q_t(s,a) + \alpha_t \left[\mathcal{T}^* Q_t(s,a) - Q_t(s,a) + \xi_t(s,a) \right]. \tag{4.22}$$

对应ODE 为 $\dot{Q} = \mathcal{T}^* Q - Q$。由 $\mathcal{T}^*$ 的 γ-压缩性，Q^* 是唯一全局吸引子；Lyapunov 函数 $\|Q - Q^*\|_\infty$ 满足

$$\frac{\mathrm{d}}{\mathrm{d}t} \|Q - Q^*\|_\infty \le -(1-\gamma) \|Q - Q^*\|_\infty. \tag{4.23}$$

在遍历性与步长条件下，随机逼近收敛到ODE 的吸引子，即 Q^*。完整证明见Watkins 与 Dayan[10]。 □

4.4.2 最大化偏差

Q-learning 的目标含 max，这在统计上不是无害的。设对每个动作 a，$Q_t(s,a)$ 是 $Q^*(s,a)$ 的噪声估计。由Jensen 不等式（max 是凸函数），

$$\mathbb{E}\left[\max_a Q_t(s,a) \right] \ge \max_a \mathbb{E}\left[Q_t(s,a) \right] = \max_a Q^*(s,a). \tag{4.24}$$

因此即使每个动作的估计都是无偏的，取最大后也会**系统性高估**。高估会通过自举传播，导致策略偏好“被高估的动作”，在噪声大的环境中尤其严重。

4.4.3 Double Q-learning

Double Q-learning 用两个独立的估计 Q_1, Q_2 解耦“选动作”与“评估动作”：

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q_2\left(S_{t+1}, \arg\max_a Q_1(S_{t+1}, a) \right) - Q_1(S_t, A_t) \right], \tag{4.25}$$

并以 1/2 的概率交换 Q_1, Q_2 的角色。其直觉是：用 Q_1 选动作（可能仍偏向高估），但用独立的 Q_2 评估该动作，从而切断“同一个噪声既选又评”的闭环。可以证明，在适当条件下Double Q-learning 收敛到 Q^*，且高估显著减小。

命题4.1 (Double Q-learning 的无偏性直觉). *若 Q_1, Q_2 相互独立且对每个 (s,a) 无偏，则*

$$\mathbb{E}\left[Q_2\left(s, \arg\max_a Q_1(s,a)\right)\right] \tag{4.26}$$

不一定等于 $\max_a Q^*(s,a)$，但其高估程度通常小于 $\mathbb{E}[\max_a Q_1(s,a)] - Q^*(s,a)$。

严格地说，Double Q-learning 并非完全无偏：选择动作的 Q_1 仍有噪声，Q_2 在该动作上的值也可能偏离。但在实践与多数理论分析中，它把“最大化偏差”从一阶降到了可接受的范围。这一思想后来被Double DQN 直接继承。

4.5 DQN：深度值函数逼近

4.5.1 损失与梯度

DQN 用参数为 θ 的卷积网络 $Q_\theta(s,a)$ 逼近动作值，用参数为 θ^- 的目标网络计算自举目标：

$$y_t = R_{t+1} + \gamma \max_{a'} Q_{\theta^-}(S_{t+1}, a'). \tag{4.27}$$

损失为

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y_t - Q_\theta(s,a)\right)^2\right]. \tag{4.28}$$

对 θ 求梯度（y_t 视为常数）：

$$\nabla_\theta \mathcal{L}(\theta) = -2\,\mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y_t - Q_\theta(s,a)\right)\nabla_\theta Q_\theta(s,a)\right]. \tag{4.29}$$

这就是“半梯度”在深度网络上的形式。目标网络的参数不是通过梯度更新，而是每隔 C 步从 θ 复制，或使用Polyak 平均

$$\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-, \qquad \tau \in (0,1). \tag{4.30}$$

4.5.2 经验回放为什么有效?

设数据由行为策略 μ 产生，而目标是 π。若直接用在线样本做SGD，样本高度相关，且分布随策略变化。经验回放把转移存入缓冲区 $\mathcal{D}$，训练时均匀采样。这带来三个效果：

1. **去相关**：随机小批量近似独立同分布，使SGD 的方差估计更可靠；
2. **数据复用**：一个转移可以被多次使用，提高样本效率；

3. **分布混合**：缓冲区中的样本来自多个历史策略，降低了“追逐当前策略”的非平稳性。

但回放也引入了**分布偏移**：缓冲区分布 $d^{\mathcal{D}}$ 与目标策略的占用分布 d^{π} 不一致。Q-learning 的离策略性质允许这种偏移，但函数逼近下的收敛性不再有保证。这一矛盾是DQN 之后所有“离线/批量RL”研究的起点。

4.5.3 Double DQN

Double DQN 把Double Q-learning 的思想搬到深度网络：用在线网络选动作，用目标网络评估：

$$y_t = R_{t+1} + \gamma\, Q_{\theta^-}\left(S_{t+1}, \arg\max_{a'} Q_{\theta}(S_{t+1}, a')\right). \tag{4.31}$$

与式(4.27) 相比，动作选择与价值评估使用了不同的参数，从而缓解了 max 带来的高估。这个改动几乎不增加计算成本，却在Atari 上带来了稳定提升。

4.5.4 优先经验回放与重要性采样

优先经验回放（prioritized replay）按TD 误差的大小采样：

$$p_i = \frac{|\delta_i|^{\alpha}}{\sum_j |\delta_j|^{\alpha}}, \qquad \alpha \geq 0. \tag{4.32}$$

由于采样分布不再是均匀分布，需要重要性采样权重来保持无偏：

$$w_i = \left(\frac{1}{N p_i}\right)^{\beta}, \tag{4.33}$$

损失变为

$$\mathcal{L}(\theta) = \mathbb{E}_{i\sim p}\left[w_i \left(y_i - Q_{\theta}(s_i, a_i)\right)^2\right]. \tag{4.34}$$

β 从初始值退火到 1，以平衡偏差与方差。优先回放的本质是“把有限的更新预算花在TD 误差大的样本上”，这与主动学习的思想相通。

4.5.5 Dueling 与噪声网络

Dueling 结构把Q 分解为状态价值与优势：

$$Q_{\theta}(s,a) = V_{\theta}(s) + A_{\theta}(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A_{\theta}(s,a'). \tag{4.35}$$

减去均值是为了可辨识性：否则 V 与 A 可以同时加减一个常数而不改变Q。Dueling 让网络在“动作选择不重要”的状态上更专注于学习 V，从而提升样本效率。

NoisyNet 则把探索从动作层（ε-贪心）移到参数层：在网络的权重上注入可学习的噪声，使探索与状态相关。这些改进都属于“表示与探索”的工程创新，不改变Q-learning 的数学骨架。

4.6 分布式RL：从期望到分布

4.6.1 为什么需要分布?

Q-learning 只估计 $\mathbb{E}[G_t]$。但在风险敏感的场景中，两个策略可能有相同的均值、截然不同的尾部风险。分布式RL 估计回报的完整分布

$$Z^\pi(s,a) \overset{D}{=} R(s,a) + \gamma Z^\pi(S',A'), \qquad S' \sim P(\cdot \mid s,a),\ A' \sim \pi(\cdot \mid S'), \tag{4.36}$$

其中 $\overset{D}{=}$ 表示同分布。若我们能精确表示 Z^π，则 $\mathbb{E}[Z^\pi] = Q^\pi$，且可以获得方差、分位数、CVaR 等风险度量。

4.6.2 C51：分类分布与投影

C51 把回报分布限制在固定支撑

$$z_1 < z_2 < \cdots < z_N, \qquad z_i = z_{\min} + (i-1)\Delta z, \quad \Delta z = \frac{z_{\max} - z_{\min}}{N-1} \tag{4.37}$$

上，用一个分类分布 $\{p_i(s,a)\}$ 表示。贝尔曼更新后的支撑点 $r + \gamma z_j$ 可能落在网格之间，因此需要投影到相邻两个格点：

$$\big(\Phi \hat{Z}(s,a)\big)_i = \sum_{j=1}^{N} \hat{p}_j(s,a) \left[1 - \frac{\left| \big[r + \gamma z_j \big]_{z_{\min}}^{z_{\max}} - z_i \right|}{\Delta z} \right]_+, \tag{4.38}$$

其中 $[x]_a^b = \min\{\max\{x,a\},b\}$，$[\cdot]_+ = \max\{\cdot,0\}$。损失为两个分类分布之间的KL 散度：

$$\mathcal{L}(\theta) = \mathbb{E}\left[\sum_{i=1}^{N} \big(\Phi \hat{Z}_{\theta^-}(s',a')\big)_i \log \frac{\big(\Phi \hat{Z}_{\theta^-}(s',a')\big)_i}{p_{\theta,i}(s,a)} \right]. \tag{4.39}$$

投影(4.38) 是“把连续分布的质量按线性插值分配到最近的两个格点”，它保证了更新后的分布仍在同一支撑上。

4.6.3 QR-DQN：分位数回归

QR-DQN 不学习分类分布，而是学习 N 个分位数 $\{z_i\}$，对应分位点 $\tau_i = (i-0.5)/N$。它最小化分位数回归损失

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \mathbb{E}\left[\rho_{\tau_i}\left(\delta_{ij}\right)\right], \qquad \rho_\tau(\delta) = |\tau - \mathbf{1}\{\delta < 0\}|\,|\delta|, \tag{4.40}$$

其中 $\delta_{ij} = r + \gamma z_j(s', a') - z_i(s, a)$。分位数回归的梯度只依赖误差的符号与分位点，因而对异常值稳健。IQN 进一步把 τ 作为输入，学习连续的分位数函数，提升了表达能力。

分布式RL 的理论价值在于：它把贝尔曼方程从“标量期望”提升为“分布算子”，并证明了分布贝尔曼算子也是压缩的（在Wasserstein 或Cramér 度量下）。这一视角在风险敏感控制、离线RL 的不确定性估计与探索中都有应用。

4.7 本章小结

本章从贝尔曼期望方程出发，构造了值函数方法的完整谱系：

1. TD(0) 用一次采样替代期望，用半梯度替代真梯度，其收敛性来自贝尔曼算子的压缩性与 Robbins–Monro 步长条件；
2. TD(λ) 用资格迹在“看得远”与“更新快”之间连续调节，前向与后向视角在回合内等价；
3. SARSA 评估行为策略，Q-learning 直接逼近最优算子；二者在风险任务中的行为差异，是“在策略/离策略”的最直观体现；
4. Q-learning 的 max 带来最大化偏差，Double Q-learning 用两个独立估计解耦“选”与“评”；
5. DQN 用经验回放与目标网络把Q-learning 稳定地扩展到像素输入，但分布偏移与收敛性问题并未根治；
6. 分布式RL 把“期望”升级为“分布”，C51 的投影与QR-DQN 的分位数回归是两种代表性的分布表示。

值函数方法的核心是“先估计价值，再贪心改进”。但当动作空间连续、或策略本身需要复杂表示时，先估计价值再取 arg max 可能不可行。第5 章转向直接优化策略：策略梯度、自然梯度、TRPO、PPO、DDPG、SAC。我们将看到，这些方法并不是值函数方法的替代品，而是把“价值估计”降级为“辅助信号”（critic/baseline），把优化对象直接提升为策略参数。

第 5 章

策略优化方法的严格推导：从策略梯度到 SAC

值函数方法先估计“每个动作值多少”，再按价值选动作。当动作空间连续、或最优策略是随状态变化的复杂分布时，“先取 $\arg\max$ 再执行”既不现实也不必要。策略优化方法直接把策略参数化，用样本估计目标的梯度，再沿梯度方向改进。本章给出策略梯度定理的完整证明，推导REINFORCE、actor–critic、GAE、TRPO、PPO、DPG/DDPG、SAC 与TD3，并说明它们各自的偏差来源与适用边界。

5.1 策略梯度定理

5.1.1 问题设定与轨迹似然

设策略由参数 $\theta \in \mathbb{R}^d$ 参数化：$\pi_\theta(a \mid s)$。目标是最大化初始分布下的期望折扣回报

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right], \tag{5.1}$$

其中轨迹 $\tau = (S_0, A_0, R_1, S_1, \dots)$ 的分布为

$$p_\theta(\tau) = \rho_0(S_0) \prod_{t=0}^{\infty} \pi_\theta(A_t \mid S_t) P(S_{t+1} \mid S_t, A_t). \tag{5.2}$$

注意 P 与 ρ_0 不依赖 θ，只有策略因子依赖 θ。这一结构是似然比技巧能够成立的根本原因。

引理5.1 (轨迹似然比). *若 $\pi_\theta(a \mid s) > 0$ 且对 θ 可微，则*

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(A_t \mid S_t). \tag{5.3}$$

证明 对式(5.2) 取对数：

$$\log p_\theta(\tau) = \log \rho_0(S_0) + \sum_{t=0}^{\infty} \log \pi_\theta(A_t \mid S_t) + \sum_{t=0}^{\infty} \log P(S_{t+1} \mid S_t, A_t). \tag{5.4}$$

对 θ 求导，只有第二项非零：

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(A_t \mid S_t). \tag{5.5}$$

两边乘以 $p_\theta(\tau)$ 即得。 □

5.1.2 主定理

定理5.1 (策略梯度定理). 设策略 π_θ 可微、且 $\pi_\theta(a \mid s) > 0$，则

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{\infty} \gamma^t \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, G_t\right], \tag{5.6}$$

其中 $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$。等价地，令 d^{π_θ} 为式(2.8)定义的归一化占用度量，则

$$\nabla_\theta J(\theta) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi_\theta}, a \sim \pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, Q^{\pi_\theta}(s,a)\right]. \tag{5.7}$$

证明 由引理5.1，

$$\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \int p_\theta(\tau) G(\tau)\, \mathrm{d}\tau = \int \nabla_\theta p_\theta(\tau) G(\tau)\, \mathrm{d}\tau \\
&= \int p_\theta(\tau) \left(\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(A_t \mid S_t)\right) G(\tau)\, \mathrm{d}\tau \\
&= \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, G(\tau)\right],
\end{aligned} \tag{5.8}$$

其中 $G(\tau) = \sum_{t'=0}^{\infty} \gamma^{t'} R_{t'+1}$。

接下来利用因果性。把 $G(\tau)$ 拆成 t 之前与 t 之后的奖励：

$$G(\tau) = \sum_{t'=0}^{t-1} \gamma^{t'} R_{t'+1} + \sum_{t'=t}^{\infty} \gamma^{t'} R_{t'+1}. \tag{5.9}$$

对第一部分的每一项，给定 S_t 后，$\nabla_\theta \log \pi_\theta(A_t \mid S_t)$ 的条件期望为零：

$$\mathbb{E}\left[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \mid S_t = s\right] = \sum_{a \in \mathcal{A}} \nabla_\theta \pi_\theta(a \mid s) = \nabla_\theta \sum_{a \in \mathcal{A}} \pi_\theta(a \mid s) = \nabla_\theta 1 = 0. \tag{5.10}$$

因此这些项在期望中消失。于是

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \nabla_\theta \log \pi_\theta(A_t \mid S_t) \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}\right] = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \nabla_\theta \log \pi_\theta(A_t \mid S_t) G_t\right]. \tag{5.11}$$

这里把 γ^t 提出来，是因为 $G_t=\sum_{k=0}^{\infty}\gamma^k R_{t+k+1}$ 对应的是从 t 开始的折扣回报；原式中的 $\gamma^{t'}$ 在 $t'=t+k$ 时分解为 $\gamma^t\gamma^k$。

最后证明与占用度量形式(5.7) 的等价性。对给定 $S_t=s,A_t=a$，有 $\mathbb{E}[G_t\mid S_t=s,A_t=a]=Q^{\pi_\theta}(s,a)$。于是

$$\begin{aligned}\nabla_\theta J(\theta)&=\sum_{t=0}^{\infty}\gamma^t\mathbb{E}\left[\nabla_\theta\log\pi_\theta(A_t\mid S_t)G_t\right]\\&=\sum_{t=0}^{\infty}\gamma^t\sum_{s,a}\mathbb{P}(S_t=s,A_t=a)\nabla_\theta\log\pi_\theta(a\mid s)Q^{\pi_\theta}(s,a)\\&=\sum_{s,a}\left(\sum_{t=0}^{\infty}\gamma^t\mathbb{P}(S_t=s)\right)\pi_\theta(a\mid s)\nabla_\theta\log\pi_\theta(a\mid s)Q^{\pi_\theta}(s,a)\\&=\frac{1}{1-\gamma}\sum_{s,a}d^{\pi_\theta}(s,a)\nabla_\theta\log\pi_\theta(a\mid s)Q^{\pi_\theta}(s,a).\end{aligned}\tag{5.12}$$

其中最后一步用了 $d^{\pi_\theta}(s,a)=(1-\gamma)\sum_t\gamma^t\mathbb{P}(S_t=s)\pi_\theta(a\mid s)$。这正是式(5.7)。 □

定理5.1 有两个值得强调的推论。第一，策略梯度*不依赖于转移核的可微性*：环境可以是黑箱、离散、甚至不可导。第二，策略梯度只通过 $\log\pi_\theta(A_t\mid S_t)$ 依赖参数，因而可以用任何可计算的策略分布（Softmax、高斯、分类分布）实现。

常见误区：把 G_t 换成 Q 是否等价?

在期望意义上，$\mathbb{E}[G_t\mid S_t,A_t]=Q^\pi(S_t,A_t)$，故用 Q 替代 G_t 不改变梯度。但在单样本估计中，G_t 是 Q 的无偏样本，方差更大；用critic 估计 Q 会引入偏差但降低方差。这一权衡是actor–critic 的核心。

5.2 REINFORCE 与基线

5.2.1 REINFORCE 估计量

用蒙特卡洛样本近似式(5.6)，得到REINFORCE 估计量

$$\hat{g}_{\mathrm{RF}}=\sum_{t=0}^{T-1}\gamma^t G_t\nabla_\theta\log\pi_\theta(A_t\mid S_t).\tag{5.13}$$

它是 $\nabla_\theta J(\theta)$ 的无偏估计：

$$\mathbb{E}[\hat{g}_{\mathrm{RF}}]=\nabla_\theta J(\theta).\tag{5.14}$$

证明只需把上面的定理反过来读：每一项的条件期望正好是 $\gamma^t\nabla\log\pi\,Q$。无偏性不保证低方差；实际上 G_t 的方差随回合长度增长，REINFORCE 的梯度估计常常噪声极大。

5.2.2 基线为什么不改变期望?

引入一个只依赖状态的基线 $b(s)$：

$$\hat{g}_b = \sum_{t=0}^{T-1} \gamma^t \left(G_t - b(S_t)\right) \nabla_\theta \log \pi_\theta(A_t \mid S_t). \tag{5.15}$$

命题5.1 (基线不变性). 对任意 $b : \mathcal{S} \to \mathbb{R}$,

$$\mathbb{E}\left[(G_t - b(S_t)) \nabla_\theta \log \pi_\theta(A_t \mid S_t) \mid S_t = s\right] = \mathbb{E}\left[G_t \nabla_\theta \log \pi_\theta(A_t \mid S_t) \mid S_t = s\right]. \tag{5.16}$$

证明 由式(5.10)，

$$\mathbb{E}\left[b(S_t) \nabla_\theta \log \pi_\theta(A_t \mid S_t) \mid S_t = s\right] = b(s) \sum_a \nabla_\theta \pi_\theta(a \mid s) = b(s) \nabla_\theta 1 = 0. \tag{5.17}$$

故减去基线不改变条件期望。 □

基线的作用是降低方差。若把 $\hat{g}$ 的方差写成

$$\mathrm{Var}(\hat{g}) = \mathbb{E}\left[\|\hat{g}\|^2\right] - \|\mathbb{E}[\hat{g}]\|^2, \tag{5.18}$$

则最优基线可以通过最小化 $\mathbb{E}[\|(G - b)\nabla \log \pi\|^2]$ 对 b 求导得到：

$$b^*(s) = \frac{\mathbb{E}\left[\|\nabla_\theta \log \pi_\theta(A \mid s)\|^2 G \mid S_t = s\right]}{\mathbb{E}\left[\|\nabla_\theta \log \pi_\theta(A \mid s)\|^2 \mid S_t = s\right]}. \tag{5.19}$$

它是以“梯度范数平方”为权重的 G 的加权平均。实践中常用 $V^\pi(s)$ 作为基线，因为它容易用 critic 估计，且近似满足“好动作正优势、坏动作负优势”的直觉。

5.2.3 优势函数与actor–critic

定义优势 $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$。由于 $V^\pi(s)$ 是与动作无关的基线，策略梯度可以写成

$$\nabla_\theta J(\theta) = \frac{1}{1-\gamma} \mathbb{E}_{s\sim d^\pi, a\sim\pi}\left[\nabla_\theta \log \pi_\theta(a \mid s) A^\pi(s,a)\right]. \tag{5.20}$$

优势的符号直接告诉我们“这个动作比平均好还是差”：正优势应提高概率，负优势应降低概率。这比用原始 Q 更稳定，因为 Q 的绝对尺度可能很大，而优势围绕零波动。

actor–critic 用参数化的critic $V_\phi(s)$ 估计 $V^\pi(s)$，用TD 误差作为优势的样本：

$$\delta_t = R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t). \tag{5.21}$$

命题5.2 (TD 误差是优势的无偏样本)**.** *若 $V_\phi = V^\pi$，则*

$$\mathbb{E}\left[\delta_t \mid S_t = s, A_t = a\right] = A^\pi(s,a). \tag{5.22}$$

证明 由贝尔曼方程，

$$\mathbb{E}\left[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s, A_t = a\right] = Q^\pi(s,a). \tag{5.23}$$

两边减去 $V^\pi(s)$ 即得。 □

实践中 V_ϕ 并不精确，故TD 优势有偏；但它的方差远小于蒙特卡洛回报。A2C 的损失通常写成

$$\mathcal{L}_{\text{A2C}}(\theta,\phi) = -\mathbb{E}\left[\log \pi_\theta(A_t \mid S_t)\hat{A}_t\right] + \frac{1}{2}\mathbb{E}\left[\left(V_\phi(S_t) - \hat{R}_t\right)^2\right] - \beta\mathbb{E}\left[\mathcal{H}\left(\pi_\theta(\cdot \mid S_t)\right)\right], \tag{5.24}$$

其中 $\hat{A}_t$ 是优势估计，$\hat{R}_t$ 是回报目标，$\mathcal{H}$ 是策略熵（用于鼓励探索）。

5.3 广义优势估计（GAE）

5.3.1 从 n 步优势到指数加权

定义TD 误差 $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$。$n$ 步优势估计为

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}) - V(S_t). \tag{5.25}$$

它与 n 步回报的关系是 $\hat{A}_t^{(n)} = G_t^{(n)} - V(S_t)$，并约定 $\hat{A}_t^{(0)} = 0$。n 越小，偏差越大、方差越小；n 越大，偏差越小、方差越大。

GAE 对所有 n 做几何加权：

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1}\hat{A}_t^{(n)}, \qquad \lambda \in [0,1]. \tag{5.26}$$

命题5.3 (GAE 的递归形式)**.** *式(5.26) 等价于*

$$\hat{A}_t^{\text{GAE}} = \delta_t + \gamma\lambda\, \hat{A}_{t+1}^{\text{GAE}}. \tag{5.27}$$

证明 把 $\hat{A}_t^{(n)} = \delta_t + \gamma \hat{A}_{t+1}^{(n-1)}$ 代入(5.26)：

$$\begin{aligned}
\hat{A}_t^{\mathrm{GAE}} &= (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \left(\delta_t + \gamma \hat{A}_{t+1}^{(n-1)} \right) \\
&= \delta_t + \gamma(1-\lambda) \sum_{m=0}^{\infty} \lambda^m \hat{A}_{t+1}^{(m)} \\
&= \delta_t + \gamma\lambda(1-\lambda) \sum_{m=1}^{\infty} \lambda^{m-1} \hat{A}_{t+1}^{(m)} \\
&= \delta_t + \gamma\lambda\, \hat{A}_{t+1}^{\mathrm{GAE}}.
\end{aligned} \tag{5.28}$$

边界条件 $\hat{A}_T^{\mathrm{GAE}} = 0$（回合结束）。 □

GAE 的两个极端值得记住：

- $\lambda = 0$：$\hat{A}_t = \delta_t$，单步TD 优势，偏差最大、方差最小；
- $\lambda = 1$：$\hat{A}_t = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} = G_t - V(S_t)$，蒙特卡洛优势，偏差最小、方差最大。

因此 λ 是一个可调的“偏差—方差旋钮”，也是PPO 实现中最重要的超参数之一。

5.4 自然梯度与TRPO

5.4.1 策略改进下界

TRPO 的出发点不是“梯度下降”，而是“每一步最多改进多少”。先给出策略性能差引理。

引理5.2 (性能差引理)**.** *对任意两个策略 π, π'，*

$$J(\pi') - J(\pi) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi'}, a \sim \pi'} \left[A^{\pi}(s,a) \right]. \tag{5.29}$$

证明 从 $J(\pi')$ 中减去 $J(\pi)$，并在每一步插入 $V^{\pi}(S_t)$：

$$\begin{aligned}
J(\pi') - J(\pi) = \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] - \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t V^{\pi}(S_t) \right] \\
+ \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t V^{\pi}(S_t) \right] - J(\pi).
\end{aligned} \tag{5.30}$$

第一行用到了 $J(\pi) = \mathbb{E}_{\pi'}[\sum_t \gamma^t V^{\pi}(S_t)]$？这并不成立：$J(\pi)$ 是在 π 下定义的。正确做法是利用Telescoping：

$$J(\pi') = \mathbb{E}_{\pi'} \left[\sum_{t=0}^{\infty} \gamma^t \left(R_{t+1} + \gamma V^{\pi}(S_{t+1}) - V^{\pi}(S_t) \right) \right] + V^{\pi}(S_0). \tag{5.31}$$

因为中间项望远镜相消：

$$\sum_{t=0}^{T-1} \gamma^t \left(\gamma V^\pi(S_{t+1}) - V^\pi(S_t)\right) = \gamma^T V^\pi(S_T) - V^\pi(S_0) \to -V^\pi(S_0). \tag{5.32}$$

故

$$J(\pi') = J(\pi) + \mathbb{E}_{\pi'}\left[\sum_{t=0}^{\infty} \gamma^t A^\pi(S_t, A_t)\right]. \tag{5.33}$$

把期望按占用度量展开，即得式(5.29)。 □

性能差引理把“新策略比旧策略好多少”写成“新策略占用分布下旧策略优势的期望”。但 $d^{\pi'}$ 依赖新策略，无法直接优化。TRPO 用旧策略的占用分布 d^π 做重要性采样：

$$J(\pi') - J(\pi) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^\pi, a\sim\pi}\left[\frac{d^{\pi'}(s)}{d^\pi(s)}\frac{\pi'(a\mid s)}{\pi(a\mid s)} A^\pi(s,a)\right]. \tag{5.34}$$

若假设 $d^{\pi'} \approx d^\pi$，就得到surrogate 目标

$$L_\pi(\pi') = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^\pi, a\sim\pi}\left[\frac{\pi'(a\mid s)}{\pi(a\mid s)} A^\pi(s,a)\right]. \tag{5.35}$$

$L_\pi(\pi)$ 的一阶导数等于 J 在 π 处的一阶导数，因此它是 J 的局部近似。

定理5.2 (策略改进下界)**.** *设 $\epsilon = \max_{s,a}|A^\pi(s,a)|$。则存在常数 $C \geq 2$，使得*

$$J(\pi') \geq L_\pi(\pi') - \frac{C\gamma\epsilon}{(1-\gamma)^2}\max_s D_{\mathrm{KL}}\left(\pi'(\cdot\mid s)\,\|\,\pi(\cdot\mid s)\right). \tag{5.36}$$

不同证明给出的常数 C 在 2 与 4 之间，取决于用全变差还是KL 放缩。

证明思路 把式(5.34) 与 L_π 的差写成

$$\left|J(\pi') - J(\pi) - L_\pi(\pi')\right| \leq \frac{\epsilon}{1-\gamma}\mathbb{E}_{s\sim d^\pi}\left|\frac{d^{\pi'}(s)}{d^\pi(s)} - 1\right|. \tag{5.37}$$

占用度量的差异由策略差异累积而来。利用

$$\sum_s \left|d^{\pi'}(s) - d^\pi(s)\right| \leq \frac{2\gamma}{1-\gamma}\max_s D_{\mathrm{TV}}\left(\pi'(\cdot\mid s), \pi(\cdot\mid s)\right), \tag{5.38}$$

以及Pinsker 不等式 $D_{\mathrm{TV}} \leq \sqrt{D_{\mathrm{KL}}/2}$，再对 $\sqrt{D_{\mathrm{KL}}}$ 用 $2\sqrt{x} \leq x+1$ 放缩，即得式(5.36)。常数 C 的具体取值来自这些不等式的松紧程度。 □

5.4.2 从下界到信赖域

由定理5.2，最大化 $J(\pi')$ 的一个保守代理是

$$\max_{\theta} \; L_{\pi_{\text{old}}}(\pi_\theta) \quad \text{s.t.} \quad \max_{s} D_{\text{KL}}\left(\pi_\theta \,\|\, \pi_{\text{old}}\right) \le \delta. \tag{5.39}$$

用样本估计 L，用平均KL 替代最大KL，得到实用形式

$$\max_{\theta} \; \mathbb{E}\left[\frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)} A^{\pi_{\text{old}}}(s,a)\right] \quad \text{s.t.} \quad \mathbb{E}\left[D_{\text{KL}}\left(\pi_{\text{old}} \,\|\, \pi_\theta\right)\right] \le \delta. \tag{5.40}$$

对小步长做二阶近似。令 $g = \nabla_\theta L|_{\theta_{\text{old}}}$，$F$ 为Fisher 信息矩阵

$$F = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s)^\top\right], \tag{5.41}$$

则 $L \approx g^\top(\theta - \theta_{\text{old}})$，$D_{\text{KL}} \approx \frac{1}{2}(\theta - \theta_{\text{old}})^\top F(\theta - \theta_{\text{old}})$。于是问题化为

$$\max_{d} \; g^\top d \quad \text{s.t.} \quad \frac{1}{2} d^\top F d \le \delta. \tag{5.42}$$

构造拉格朗日函数

$$\mathcal{L}(d, \nu) = g^\top d - \frac{\nu}{2}\left(d^\top F d - 2\delta\right). \tag{5.43}$$

由一阶条件 $g - \nu F d = 0$ 得 $d = \frac{1}{\nu} F^{-1} g$；代入约束得

$$\nu = \sqrt{\frac{g^\top F^{-1} g}{2\delta}}, \qquad d = \sqrt{\frac{2\delta}{g^\top F^{-1} g}}\, F^{-1} g. \tag{5.44}$$

这就是**自然梯度**步：方向是 $F^{-1}g$，步长由信赖域半径 δ 与梯度在Fisher 度量下的范数共同决定。

TRPO 的实现用共轭梯度求 $F^{-1}g$（避免显式构造 F），用线搜索保证KL 约束与目标提升。它的理论优雅，但工程复杂，且对批量大小与KL 估计敏感。

5.5 PPO：用裁剪替代约束

5.5.1 裁剪目标

PPO 用概率比

$$r_t(\theta) = \frac{\pi_\theta(A_t \mid S_t)}{\pi_{\theta_{\text{old}}}(A_t \mid S_t)} \tag{5.45}$$

构造裁剪目标

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta) A_t,\; \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right) A_t\right)\right]. \tag{5.46}$$

其中 A_t 是优势估计（通常用GAE），ϵ 是裁剪范围（常用 0.2）。

命题5.4 (裁剪的梯度行为). 对单个样本，令 $L_t = \min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t)$。则

$$\frac{\partial L_t}{\partial \theta} = \begin{cases} A_t \nabla_\theta r_t(\theta), & A_t > 0,\ r_t < 1+\epsilon, \\ A_t \nabla_\theta r_t(\theta), & A_t < 0,\ r_t > 1-\epsilon, \\ 0, & \text{否则}. \end{cases} \tag{5.47}$$

证明 分四种情形。若 $A_t > 0$，未裁剪项 $r_t A_t$ 随 r_t 增大；当 $r_t < 1+\epsilon$ 时，min 取未裁剪项，梯度为 $A_t \nabla r_t$；当 $r_t \geq 1+\epsilon$ 时，裁剪项为常数 $(1+\epsilon)A_t$，min 取裁剪项，梯度为零。若 $A_t < 0$，未裁剪项随 r_t 增大而减小；当 $r_t > 1-\epsilon$ 时，min 取未裁剪项（更负），梯度为 $A_t \nabla r_t$；当 $r_t \leq 1-\epsilon$ 时，裁剪项为常数，梯度为零。其余边界情形由次梯度处理。 □

这个梯度行为说明：PPO 不是“硬约束”，而是“在目标函数里取消越界方向的奖励”。当优势为正时，它不允许策略把 r_t 推到 $1+\epsilon$ 以上继续获益；当优势为负时，它不允许把 r_t 压到 $1-\epsilon$ 以下继续获益。越界后梯度为零，策略被“冻结”在边界附近。与TRPO 相比，PPO 只用了一阶信息，实现简单、并行友好，但不提供单调改进的严格保证。

5.5.2 PPO 的完整损失

实践中PPO 通常同时优化策略、价值与熵：

$$\mathcal{L}(\theta, \phi) = -\mathbb{E}_t \left[L_t^{\text{CLIP}}(\theta) \right] + c_1 \mathbb{E}_t \left[\left(V_\phi(S_t) - \hat{R}_t \right)^2 \right] - c_2 \mathbb{E}_t \left[\mathcal{H}\left(\pi_\theta(\cdot \mid S_t) \right) \right], \tag{5.48}$$

其中 $\hat{R}_t$ 是回报目标（可用GAE 反推），c_1, c_2 是权重。常见技巧包括：

- 优势归一化：在一个批量内对 $\hat{A}_t$ 做零均值、单位方差归一化；
- 价值裁剪：把 V_ϕ 的更新限制在旧值附近，防止critic 发散；
- 梯度裁剪：限制全局梯度范数；
- 学习率衰减与KL 早停：当近似KL 超过阈值时提前停止该轮更新。

PPO 是2017—2023 年最常用的策略优化算法，也是RLHF 第一代流程的主力。但它在LLM 场景中有一个固有缺陷：critic 需要与策略同规模，显存开销大；且长序列的credit assignment 使critic 难以准确。这一问题直接催生了GRPO（第7 章）。

5.6 确定性策略梯度：DPG 与DDPG

5.6.1 DPG 定理

当动作空间连续时，随机策略的梯度需要对动作积分；若策略是确定性的 $a = \mu_\theta(s)$，积分消失，梯度变得更简单。Silver 等人在2014 年提出确定性策略梯度（DPG）[29]：

$$\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s\sim d^{\mu_\theta}}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q^{\mu_\theta}(s,a)\big|_{a=\mu_\theta(s)}\right]. \tag{5.49}$$

定理5.3 (DPG 定理)**.** *在标准正则条件下，式*(5.49) *成立。*

证明思路 把确定性策略看作随机策略的极限：令 $\pi_\theta(a \mid s) = \mathcal{N}(a; \mu_\theta(s), \sigma^2 I)$，当 $\sigma \to 0$ 时，策略梯度定理中的 $\nabla_\theta \log \pi_\theta(a \mid s)$ 为

$$\nabla_\theta \log \pi_\theta(a \mid s) = \frac{1}{\sigma^2}\,(a - \mu_\theta(s))^\top \nabla_\theta \mu_\theta(s). \tag{5.50}$$

代入策略梯度，并对 a 积分。当 $\sigma \to 0$ 时，a 集中在 $\mu_\theta(s)$ 附近，$\frac{1}{\sigma^2}\mathbb{E}[(a-\mu)\cdot]$ 的极限化为 $\nabla_a Q$ 在 $a = \mu_\theta(s)$ 处的值，即得式(5.49)。严格证明需对 $\sigma \to 0$ 与期望交换极限，并要求 Q 对 a 可微。 □

DPG 的直观含义是：actor 沿着“critic 对动作的梯度”方向调整动作，而critic 告诉actor“在这个状态下，动作往哪个方向变会更好”。这与策略梯度中“提高好动作概率”不同：DPG 直接移动动作本身。

5.6.2 DDPG 的实现

DDPG（Deep Deterministic Policy Gradient）把DPG 与DQN 的稳定化技巧结合[28]：

$$y_t = R_{t+1} + \gamma Q_{\theta^-}\left(S_{t+1}, \mu_{\theta^-}(S_{t+1})\right), \tag{5.51}$$

$$\mathcal{L}_Q(\theta) = \mathbb{E}\left[\left(y_t - Q_\theta(S_t, A_t)\right)^2\right], \tag{5.52}$$

$$\mathcal{L}_\mu(\theta) = -\mathbb{E}\left[Q_\theta\left(S_t, \mu_\theta(S_t)\right)\right]. \tag{5.53}$$

目标网络用Polyak 平均更新。探索通过在动作上加噪声实现：

$$A_t = \mu_\theta(S_t) + \mathcal{N}_t, \tag{5.54}$$

早期常用Ornstein–Uhlenbeck 过程，后来发现简单的高斯噪声在多数任务上足够。

DDPG 的问题是Q 值过估计与超参数敏感。TD3 从三个方面改进：

1. **截断双Q**：用两个critic，取较小值作为目标

$$y_t = R_{t+1} + \gamma \min_{i=1,2} Q_{\theta_i^-}\left(S_{t+1}, \tilde{A}_{t+1}\right); \tag{5.55}$$

2. **目标策略平滑**：在目标动作上加截断噪声

$$\tilde{A}_{t+1} = \mu_{\theta^-}(S_{t+1}) + \mathrm{clip}\,(\epsilon, -c, c)\,, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2); \tag{5.56}$$

3. **延迟策略更新**：critic 更新多次后，actor 才更新一次。

TD3 的动机是：双Q 缓解高估，目标平滑防止actor 利用critic 的尖峰误差，延迟更新降低actor 与critic 的耦合振荡。

5.7 最大熵RL：SAC

5.7.1 最大熵目标

SAC（Soft Actor–Critic）在回报中加入策略熵[31]：

$$J(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t \left(R_{t+1} + \alpha\, \mathcal{H}\left(\pi(\cdot \mid S_t)\right)\right)\right], \tag{5.57}$$

其中 $\mathcal{H}(\pi(\cdot \mid s)) = -\mathbb{E}_{a\sim\pi}[\log \pi(a \mid s)]$，$\alpha > 0$ 是温度。熵项鼓励策略在多个好动作之间保持多样性，从而提升探索与鲁棒性。

5.7.2 软贝尔曼方程

定义软价值函数

$$Q^\pi(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s'\sim P}\left[V^\pi(s')\right], \tag{5.58}$$

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}\left[Q^\pi(s,a) - \alpha \log \pi(a \mid s)\right]. \tag{5.59}$$

把式(5.59) 代入(5.58)，得到软Q 的贝尔曼方程：

$$Q^\pi(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s'\sim P,\, a'\sim\pi}\left[Q^\pi(s',a') - \alpha \log \pi(a' \mid s')\right]. \tag{5.60}$$

软贝尔曼最优方程为

$$Q^*(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s'\sim P}\left[\alpha \log \sum_{a'} \exp\left(\frac{Q^*(s',a')}{\alpha}\right)\right], \tag{5.61}$$

其中对数—求和—指数（log-sum-exp）是“软最大”算子。当 $\alpha \to 0$ 时，soft max 退化为硬max，SAC 退化为普通Q-learning。

5.7.3 软策略迭代

定理5.4 (软策略改进)**.** 对固定 Q，最大化“期望Q + 熵”的策略为

$$\pi_{\text{new}}(a \mid s) = \frac{\exp\left(Q(s,a)/\alpha\right)}{Z(s)}, \qquad Z(s) = \sum_{a} \exp\left(Q(s,a)/\alpha\right). \tag{5.62}$$

证明 要最大化的目标为

$$\mathbb{E}_{a\sim\pi'}\left[Q(s,a)\right] + \alpha\mathcal{H}(\pi'(\cdot \mid s)) = \mathbb{E}_{a\sim\pi'}\left[Q(s,a) - \alpha \log \pi'(a \mid s)\right]. \tag{5.63}$$

考虑KL 散度

$$D_{\text{KL}}\left(\pi'(\cdot \mid s) \,\|\, \frac{\exp(Q(s,\cdot)/\alpha)}{Z(s)}\right) = \mathbb{E}_{\pi'}\left[\log \pi'(a \mid s) - \frac{Q(s,a)}{\alpha} + \log Z(s)\right]. \tag{5.64}$$

因此

$$\mathbb{E}_{\pi'}\left[Q(s,a) - \alpha \log \pi'(a \mid s)\right] = -\alpha D_{\text{KL}}\left(\pi' \,\|\, \pi_{\text{new}}\right) + \alpha \log Z(s). \tag{5.65}$$

右端第一项在 $\pi' = \pi_{\text{new}}$ 时取最小值 0，故目标最大。π_{new} 即为式(5.62)。 □

SAC 的实践版本用两个Q 网络、一个策略网络与一个温度参数。critic 损失为

$$\mathcal{L}_Q(\theta_i) = \mathbb{E}\left[\left(Q_{\theta_i}(s,a) - \left(r + \gamma\left(\min_{j=1,2} Q_{\theta_j^-}(s',a') - \alpha \log \pi_\theta(a' \mid s')\right)\right)\right)^2\right], \tag{5.66}$$

其中 $a' \sim \pi_\theta(\cdot \mid s')$。策略损失为

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{s\sim\mathcal{D},\, a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_\theta(s,a)\right]. \tag{5.67}$$

连续动作通过重参数化 $a = \tanh(\mu_\theta(s) + \sigma_\theta(s) \odot \xi)$，$\xi \sim \mathcal{N}(0, I)$，使梯度可以穿过采样。温度 α 可以固定，也可以自动调节以维持目标熵 $\bar{\mathcal{H}}$：

$$\mathcal{L}_\alpha = \mathbb{E}\left[-\alpha\left(\log \pi_\theta(a \mid s) + \bar{\mathcal{H}}\right)\right]. \tag{5.68}$$

SAC 的核心优势是样本效率与稳定性：最大熵目标使策略在早期保持随机，避免过早收敛到局部最优；软策略迭代提供了清晰的收敛框架。它在连续控制（机器人、自动驾驶、工业控制）中是默认基线之一。

5.8 算法谱系与选择

表5.1 总结本章算法的关键属性。选择算法时，先问三个问题：动作空间是离散还是连续？能否在线采样？是否需要离策略数据复用？答案会迅速缩小候选集。

表 5.1 策略优化算法对比

算法	策略类型	在/离策略	核心机制与适用场景
REINFORCE	随机	在策略	蒙特卡洛策略梯度；简单、方差大；适合教学与小规模任务
A2C/A3C	随机	在策略	actor–critic + 优势；并行采样；适合离散/连续控制
TRPO	随机	在策略	KL 信赖域+ 自然梯度；理论稳健、实现复杂
PPO	随机	在策略	裁剪目标；简单稳定；RLHF 与游戏的主流
DPG/DDPG	确定性	离策略	确定性策略梯度+ critic；连续控制；易过估计
TD3	确定性	离策略	双Q + 目标平滑+ 延迟更新；连续控制强基线
SAC	随机	离策略	最大熵+ 软策略迭代；样本效率高、探索好

5.9 本章小结

本章从策略梯度定理出发，构建了策略优化方法的完整谱系：

1. 策略梯度定理把“对轨迹分布求导”化为“对动作对数概率求导”，其证明只依赖似然比与因果性，不要求环境可微；
2. REINFORCE 是无偏但高方差的估计；基线不改变期望，只降低方差；TD 误差在critic 精确时是优势的无偏样本；
3. GAE 用指数加权在 n 步优势之间连续调节，递归形式 $\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}$ 是PPO 实现的核心；
4. TRPO 从性能差引理出发，用KL 约束把策略改进转化为信赖域问题，其二次近似给出自然梯度步；
5. PPO 用裁剪替代显式KL 约束，以牺牲单调性保证换取实现简单与并行友好；
6. DPG/DDPG 把策略梯度推广到确定性策略，TD3 用双Q、目标平滑与延迟更新稳定critic；
7. SAC 用最大熵目标与软策略迭代，把“探索”写进目标函数，在连续控制中表现优异。

这些算法构成了2015—2023 年深度RL 的主干。但它们的共同假设是“奖励是标量、环境可交互、动作空间是低维向量”。当动作变成自然语言、奖励变成人类偏好或可验证规则时，这些假设需要被重新审视。第6 章先处理模型基、搜索与离线RL，第7 章再进入LLM 时代的RLHF/RLVR。

第 6 章

模型基、搜索与离线强化学习

前三章的方法都假设智能体可以与环境交互，并直接用真实转移更新价值或策略。本章转向三个“非标准”但日益重要的设定：**模型基RL** 先学习环境模型，再用模型做规划或生成虚拟样本；**搜索**在决策时用树展开替代完整策略；**离线RL** 只能使用固定数据集，不能在线试错。三者的共同主题是：*当真实交互昂贵、危险或不可用时，如何用“代理”获得经验。*代理分别是学习到的模型、搜索树与历史数据；它们都能提高样本效率，也都引入了新的偏差。

6.1 模型基强化学习

6.1.1 基本框架

模型基RL（model-based RL, MBRL）显式或隐式地学习环境模型

$$\hat{P}_\phi(s' \mid s,a) \approx P(s' \mid s,a), \qquad \hat{r}_\psi(s,a) \approx r(s,a), \tag{6.1}$$

然后用模型做三件事之一：

1. **规划**：在模型上做动态规划、树搜索或轨迹优化，直接得到动作；
2. **数据增强**：用模型生成虚拟转移，与真实转移混合训练无模型算法；
3. **表示学习**：用模型学习紧凑的隐空间，再在隐空间中训练策略。

MBRL 的吸引力在于样本效率：一次真实交互可以被模型“放大”为多次虚拟更新。它的风险在于**模型偏差**：策略会主动寻找模型预测错误的地方，因为那里往往有“被高估的奖励”。这与第7 章的奖励劫持同构：*优化器总会利用代理目标的漏洞。*

6.1.2 Dyna：真实经验与模拟经验的混合

Dyna-Q 是最早的MBRL 框架之一[11]。它维护一个经验表 $\mathcal{M}$，每一步：

1. 用真实转移 (S, A, R, S') 做一次Q-learning 更新；
2. 把该转移写入 $\mathcal{M}$；
3. 从 $\mathcal{M}$ 中随机抽取 k 个转移，重复Q-learning 更新。

用公式表示，真实更新为

$$Q(S,A) \leftarrow Q(S,A) + \alpha \left[R + \gamma \max_{a'} Q(S',a') - Q(S,A) \right], \tag{6.2}$$

模拟更新为，对 $i = 1, \ldots, k$，

$$Q(\tilde{S},\tilde{A}) \leftarrow Q(\tilde{S},\tilde{A}) + \alpha \left[\tilde{R} + \gamma \max_{a'} Q(\tilde{S}',a') - Q(\tilde{S},\tilde{A}) \right], \quad (\tilde{S},\tilde{A},\tilde{R},\tilde{S}') \sim \mathcal{M}. \tag{6.3}$$

Dyna 的意义在于它把“模型”降级为“经验表”：不需要显式的概率模型，只要有可重放的转移，就能获得类似规划的效果。现代的回放缓冲区、世界模型与离线RL 都可以看作Dyna 的变体。

6.1.3 世界模型：Dreamer 与隐空间想象

当状态是高维图像时，显式预测像素既昂贵又充满无关细节。Dreamer 系列用**隐空间世界模型**（latent world model）解决这一问题[32, 33]。其核心是循环状态空间模型（RSSM）：

$$h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}), \tag{6.4}$$

$$z_t \sim q_\phi(z_t \mid h_t, o_t), \tag{6.5}$$

$$\hat{z}_t \sim p_\phi(z_t \mid h_t), \tag{6.6}$$

$$\hat{o}_t \sim p_\phi(o_t \mid h_t, z_t), \qquad \hat{r}_t \sim p_\phi(r_t \mid h_t, z_t). \tag{6.7}$$

其中 h_t 是确定性隐状态，z_t 是随机隐状态，o_t 是观测。世界模型的损失由重建项、奖励项与KL 项组成：

$$\mathcal{L}_{\text{WM}}(\phi) = \mathbb{E}\left[\sum_t \left(\|\hat{o}_t - o_t\|^2 + \|\hat{r}_t - r_t\|^2 + \beta\, D_{\text{KL}}\left(q_\phi(z_t \mid h_t, o_t) \,\|\, p_\phi(z_t \mid h_t) \right) \right) \right]. \tag{6.8}$$

训练好世界模型后，策略在“想象”中优化：从真实轨迹的后验状态出发，用先验模型滚动 H 步，得到虚拟轨迹，再用actor–critic 更新。这样，策略的每一次更新都不需要真实环境交互。

DreamerV3 的关键贡献是*稳健性*：它用symlog 变换、归一化与自适应KL 平衡，使同一套超参数在150 多个任务（连续控制、Atari、Minecraft）上工作[33]。这被视为通用世界模型的重要一步。

6.1.4 MuZero：学习“足够用于规划”的模型

MuZero 不预测观测，也不预测完整状态，而是学习一个“隐式模型”，只保证规划所需的三件事准确[34]：

$$s^0 = h_\theta(o_1, \ldots, o_t), \tag{6.9}$$

$$r^k, s^k = g_\theta(s^{k-1}, a_{t+k}), \tag{6.10}$$

$$p^k, v^k = f_\theta(s^k), \tag{6.11}$$

其中 h 是表示函数，g 是动力学函数，f 是预测函数（策略与价值）。训练目标是在 K 步展开上匹配真实的目标：

$$\mathcal{L}(\theta) = \sum_{k=0}^{K} \left[\ell^p \left(\pi_{t+k}, p_t^k \right) + \ell^v \left(z_{t+k}, v_t^k \right) + \ell^r \left(u_{t+k}, r_t^k \right) \right], \tag{6.12}$$

其中 π_{t+k} 是MCTS 的访问分布，z_{t+k} 是真实回报（或n 步bootstrap），u_{t+k} 是真实奖励。MuZero 的妙处在于：它不要求隐状态可解释，只要求它在规划中“有用”。这与“预测未来观测”的世界模型是两种哲学：前者为决策服务，后者为重建服务。

6.2 搜索：从UCT 到AlphaZero

6.2.1 蒙特卡洛树搜索与UCT

给定一个生成模型（真实环境或模拟器），我们可以在决策时展开一棵搜索树。每个节点 s 记录访问次数 $N(s)$、动作价值 $Q(s,a)$ 与先验 $P(s,a)$。UCT 用UCB 在选择与探索之间平衡[18]：

$$a_t = \arg\max_{a \in \mathcal{A}} \left[Q(s,a) + c \sqrt{\frac{\ln N(s)}{N(s,a)}} \right]. \tag{6.13}$$

第一项利用当前估计，第二项鼓励访问少的动作。每次模拟从根节点出发，按UCT 选到叶节点，用随机rollout 或价值网络评估叶节点，再把结果回传，更新路径上的 N 与 Q。

定理6.1 (UCT 的收敛性（简述）). *在有限MDP、奖励有界、每个动作被无限次访问的条件下，UCT 的根节点价值估计以概率 1 收敛到最优值；若叶节点评估无偏，则选择的最优动作概率趋于 1。*

证明的核心是把UCB1 的遗憾界应用到树的每一层，并对深度做归纳；由于树宽随深度指数增长，实际收敛速度受“有效分支因子”限制。围棋的宽度约 250，深度约 150，朴素UCT 无法收敛；AlphaGo 的贡献是用策略网络把宽度缩小到可管理的范围。

6.2.2 AlphaGo：先验、价值与搜索

AlphaGo 的搜索使用PUCT（predictor + UCT）：

$$a_t = \arg\max_a \left[Q(s,a) + c_{\text{puct}}\, P_\theta(s,a) \frac{\sqrt{N(s)}}{1+N(s,a)} \right], \tag{6.14}$$

其中 $P_\theta(s,a)$ 是策略网络的先验。与UCT 相比，PUCT 用先验 P 替代了均匀的探索项：先验高的动作在访问次数少时获得更大的加成，从而把搜索集中到有希望的分支。

AlphaGo 的价值网络 $v_\theta(s)$ 用于评估叶节点，代替随机rollout；策略网络用于缩小候选动作。训练分为三个阶段：监督学习（人类棋谱）→ 自我对弈强化学习（REINFORCE/策略梯度）→ 价值网络回归。2016 年的版本还使用了快速rollout 策略与输赢分类器；2017 年的AlphaGo Zero 去掉了人类数据与rollout，完全依赖MCTS 与自我对弈[36]。

6.2.3 AlphaZero 的目标与训练

AlphaZero 把搜索得到的访问分布作为策略目标，把游戏结果作为价值目标。对每个局面 s_t：

$$\pi_t(a) = \frac{N(s_t,a)^{1/\tau}}{\sum_b N(s_t,b)^{1/\tau}}, \tag{6.15}$$

$$z_t = \text{最终胜负（从当前玩家视角）}, \tag{6.16}$$

损失为

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_t,\pi_t,z_t)} \left[(z_t - v_\theta(s_t))^2 - \sum_a \pi_t(a) \log p_\theta(a \mid s_t) + c\|\theta\|^2 \right]. \tag{6.17}$$

第一项让价值网络预测胜负，第二项让策略网络模仿搜索分布，第三项是权重衰减。注意这里没有显式的“策略梯度”：策略学习是*蒸馏搜索*，而不是直接对回报求导。这一“搜索—蒸馏”循环是AlphaZero 的核心：搜索产生比网络更强的策略，网络再把这个策略泛化到未见局面。

6.2.4 从AlphaZero 到推理模型

AlphaZero 的结构与2024—2026 年的推理模型高度同构：

- **先验**：策略网络（或语言模型）给出“往哪里想”的分布；
- **搜索/采样**：MCTS（或多次采样、树搜索）展开候选推理路径；
- **评估**：价值网络（或验证器/奖励模型）判断路径的好坏；
- **蒸馏**：把搜索得到的好路径用于训练策略（或直接作为RL 的轨迹）。

区别在于：围棋的模拟器完美、奖励明确；语言推理的环境是“思维链”，奖励是“答案是否正确”，搜索空间是自然语言。这一差异使推理模型的“搜索”更多依赖采样与验证器，而非严格的MCTS；但“学习先验+ 测试时搜索”的骨架是一样的。

6.3 离线强化学习

6.3.1 问题设定与分布偏移

离线RL 给定一个由行为策略 π_β 采集的数据集

$$\mathcal{D} = \left\{(s_i, a_i, r_i, s_i')\right\}_{i=1}^{N}, \qquad (s_i, a_i) \sim d^{\pi_\beta},\ s_i' \sim P(\cdot \mid s_i, a_i), \tag{6.18}$$

要求在不与环境交互的前提下学习一个策略 π，使其在线评估时表现良好。

核心困难是**分布偏移**：策略 π 可能选择数据集中从未出现过的动作 a，而Q 函数在这些动作上的值是没有数据支撑的外推。贝尔曼备份会把这种外推误差放大：

$$Q(s,a) \leftarrow r + \gamma \max_{a'} Q(s', a'), \tag{6.19}$$

若 a' 是OOD 动作且 $Q(s', a')$ 被高估，这个高估会沿轨迹回传，使策略越来越偏好OOD 动作。离线RL 的所有方法，本质上都在回答：如何在不查询环境的情况下，判断一个动作是否“可信”？

6.3.2 理论直觉：集中度系数

在表格型线性MDP 中，离线策略评估的误差可以写成

$$\left|J(\pi) - \hat{J}(\pi)\right| \lesssim \frac{C^\pi}{(1-\gamma)^2}\sqrt{\frac{\log(1/\delta)}{N}}, \tag{6.20}$$

其中 C^π 是**集中度系数**（concentrability coefficient）

$$C^\pi = \max_{s,a} \frac{d^\pi(s,a)}{d^{\pi_\beta}(s,a)}. \tag{6.21}$$

它衡量目标策略的占用分布相对行为策略的“覆盖程度”。若目标策略访问了行为策略几乎不访问的区域，C^π 很大，误差界失效。这一系数解释了为什么离线RL 在数据覆盖好的任务上有效，在覆盖差的任务上脆弱。

6.3.3 BCQ：限制动作支持集

BCQ（Batch-Constrained deep Q-learning）的思路是：只允许策略选择“行为策略支持集内”的动作[45]。它用一个生成模型 $G_\omega(s)$（如VAE）建模行为策略的动作分布，采样 n 个

候选动作，再用扰动网络 ξ_ϕ 做小幅修正，最后选Q 值最大的：

$$\pi(s) = \underset{a_i+\xi_\phi(s,a_i)}{\arg\max}\, Q_\theta\left(s, a_i + \xi_\phi(s,a_i)\right), \qquad a_i \sim G_\omega(s),\ i = 1,\ldots,n. \tag{6.22}$$

约束“候选动作来自行为分布”等价于约束策略的支持集，从而减少OOD 外推。BCQ 的局限是：生成模型可能遗漏数据中的稀有动作；扰动范围需要调参；在高维动作空间中采样效率低。

6.3.4 CQL：保守Q 学习

CQL（Conservative Q-Learning）不显式限制动作，而是在损失中加入保守正则[46]：

$$\begin{aligned}\mathcal{L}_{\mathrm{CQL}}(\theta) = \alpha\, \mathbb{E}_{s\sim\mathcal{D}}\left[\log \sum_a \exp Q_\theta(s,a) - \mathbb{E}_{a\sim\pi_\beta(\cdot|s)}\left[Q_\theta(s,a)\right]\right] \\ + \frac{1}{2}\mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(Q_\theta(s,a) - \mathcal{B}^\pi\hat{Q}(s,a)\right)^2\right].\end{aligned} \tag{6.23}$$

第一项（log-sum-exp）把所有动作的Q 值往上推，第二项把数据动作的Q 值往下拉；两者相减，净效果是*压低OOD 动作、保留数据动作*。可以证明，这个正则给出了真实Q 值的一个下界（在适当条件下），从而抑制高估。

CQL 的优点是实现简单、在离散动作与连续动作上都能用；缺点是保守程度 α 需要调参：太大导致过度悲观，太小无法抑制OOD。后续的Cal-QL、IQL 等工作试图用更自适应的方式平衡保守与效率。

6.3.5 IQL：隐式Q 学习

IQL（Implicit Q-Learning）完全避免查询OOD 动作[47]。它不直接最大化Q，而是用expectile 回归拟合“数据分布内的最优值”。定义expectile 损失

$$L_2^\tau(u) = |\tau - \mathbf{1}\{u<0\}|\, u^2, \qquad \tau \in (0,1). \tag{6.24}$$

值函数通过

$$\mathcal{L}_V(\psi) = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[L_2^\tau\left(Q_{\hat{\theta}}(s,a) - V_\psi(s)\right)\right] \tag{6.25}$$

学习，其中 $\tau > 0.5$ 使 V 逼近Q 的上分位数（而不是均值）。Q 函数通过

$$\mathcal{L}_Q(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma V_\psi(s') - Q_\theta(s,a)\right)^2\right] \tag{6.26}$$

学习。注意Q 的目标只用 $V_\psi(s')$，不需要对 a' 取max，因此完全避开了OOD 动作。策略通过优势加权回归（AWR）抽取：

$$\mathcal{L}_\pi(\phi) = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\exp\left(\beta\left(Q_{\hat{\theta}}(s,a) - V_\psi(s)\right)\right)\log\pi_\phi(a\mid s)\right]. \tag{6.27}$$

IQL 的优雅之处在于：它把“保守”转化为“分位数”，不需要显式约束动作支持集，也不需要对抗训练。它在D4RL 等离线基准上表现强劲，成为离线RL 的重要基线。

6.3.6 Decision Transformer：把RL 变成序列建模

Decision Transformer（DT）把轨迹写成序列

$$\tau = (R_0, s_0, a_0, R_1, s_1, a_1, \dots), \qquad R_t = \sum_{t'=t}^{T} r_{t'}, \tag{6.28}$$

其中 R_t 是“回报-to-go”。策略用因果Transformer 建模

$$\pi_\theta(a_t \mid R_t, s_t, \text{历史}), \tag{6.29}$$

训练目标是对数据中的动作做最大似然：

$$\mathcal{L}_{\text{DT}}(\theta) = -\mathbb{E}_{\tau\sim\mathcal{D}}\left[\sum_t \log \pi_\theta(a_t \mid R_t, s_t, \text{历史})\right]. \tag{6.30}$$

DT 的吸引力在于：它完全避开了Bellman 备份与OOD 动作问题，把RL 变成了监督学习；在推理时，给定一个目标回报 R_0，模型会生成“相应的”动作序列。但DT 也有明显局限：它不能做动态规划，不能“拼接”次优轨迹中的好片段，对长期信用分配与随机环境敏感。它更像是一种“轨迹条件的行为克隆”，而不是真正的值函数学习。

6.4 三条路线的比较

表 6.1 模型基、搜索与离线RL 的比较

路线	优势	核心风险
模型基RL	样本效率高；可用于规划与想象；适合机器人、控制	模型偏差；策略利用模型错误；长时程误差累积
搜索（MCTS/采样）	决策时可修正；不需要完美全局策略；适合博弈与推理	计算成本高；依赖模拟器/验证器；分支因子大时难扩展
离线RL	无需在线试错；可利用历史数据；适合医疗、推荐、工业	分布偏移；OOD 高估；评估困难；对数据覆盖敏感

三者的边界正在模糊。现代系统常把三者结合：用世界模型生成想象数据（模型基），在想象中做树搜索（搜索），再用离线数据做预训练（离线）。AlphaZero 是“搜索+ 学习”的典范；MuZero 是“模型+ 搜索+ 学习”的典范；Dreamer 是“模型+ 想象+ actor–critic”的典范；离线RL 则是“数据+ 保守正则”的典范。

6.5 本章小结

本章讨论了三种“代理经验”的方法：

1. 模型基RL 用学习到的模型替代真实环境，Dyna 用经验表做规划，Dreamer 用隐空间世界模型做想象，MuZero 学习“足够用于规划”的隐式模型；
2. 搜索用在线树展开替代完整策略，UCT 用UCB 平衡探索，AlphaGo/AlphaZero 用先验与价值网络缩小搜索宽度，PUCT 与“搜索—蒸馏”是核心；
3. 离线RL 用固定数据集替代在线交互，BCQ 限制支持集，CQL 用保守正则压低OOD 动作，IQL 用expectile 回归避开OOD 查询，Decision Transformer 把RL 变成序列建模。

三者的共同教训是：*代理越强，偏差越隐蔽*。模型基方法会利用模型误差，搜索会利用价值网络的盲区，离线RL 会利用数据分布的空洞。第7 章将看到，LLM 时代的RLHF/RLVR 也遵循同一逻辑：奖励模型是代理，验证器是代理，策略总会找到代理的漏洞。

第 7 章

从人类反馈到可验证奖励：LLM 时代的强化学习

前六章的强化学习以“低维动作、标量奖励、可反复试错”为默认设定。大语言模型（LLM）打破了这三个假设：动作是长度为数百到数万token 的序列，奖励往往只能由人类偏好或程序化验证器给出，而在线试错的成本极高。本章说明，经典RL 的数学骨架并未失效，但需要三处改造：**把奖励变成偏好模型或验证器、把KL 正则写进目标、把策略优化从“价值网络”改为“组内相对优势”或“闭式偏好损失”**。我们将完整推导RLHF、DPO、GRPO 与RLVR，并讨论2025—2026 年的争议与前沿。

7.1 RLHF 的问题设定

7.1.1 从“给奖励”到“学奖励”

在经典RL 中，奖励函数 $r(s,a)$ 由环境给出。对LLM 而言，我们很难为“一段回答是否helpful、harmless、honest”写一个可微的标量函数。RLHF（Reinforcement Learning from Human Feedback）的做法是：让人类比较两条回答，用比较数据训练一个奖励模型 $r_\phi(x,y)$，再用RL 优化策略 $\pi_\theta(y \mid x)$ 以最大化奖励，同时约束它不要偏离参考策略太远。

形式化地，给定提示分布 $x \sim \mathcal{D}$、参考策略 π_{ref}（通常是SFT 模型），目标是

$$\max_\theta \; \mathcal{J}(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\left[r_\phi(x,y)\right] - \beta\, \mathbb{E}_{x\sim\mathcal{D}}\left[D_{\text{KL}}\left(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)\right)\right], \tag{7.1}$$

其中 $\beta > 0$ 控制“奖励”与“不离题”之间的权衡。KL 项有两个作用：一是防止策略退化到只会输出高奖励的短句，二是让优化在参考策略附近进行，避免奖励模型的外推误差被放大。

7.1.2 KL 正则目标的闭式最优策略

式(7.1) 有一个非常漂亮的闭式解，它是DPO 推导的起点。

定理7.1 (KL 正则RL 的最优策略)**.** *对固定奖励 $r(x,y)$，式(7.1) 的最优策略为*

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\, \pi_{\text{ref}}(y \mid x) \exp\left(\frac{r(x,y)}{\beta}\right), \tag{7.2}$$

其中配分函数

$$Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\frac{r(x,y)}{\beta}\right). \tag{7.3}$$

证明 对每个固定的 x，把目标写成对 y 的求和：

$$\mathcal{J}_x(\pi) = \sum_y \pi(y \mid x) r(x,y) - \beta \sum_y \pi(y \mid x) \log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)}. \tag{7.4}$$

这是一个带归一化约束 $\sum_y \pi(y \mid x) = 1$ 的凸优化问题。构造拉格朗日函数

$$\mathcal{L}(\pi, \lambda) = \sum_y \pi(y \mid x) \left(r(x,y) - \beta \log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)}\right) + \lambda \left(1 - \sum_y \pi(y \mid x)\right). \tag{7.5}$$

对 $\pi(y \mid x)$ 求偏导并令其为零：

$$r(x,y) - \beta \log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)} - \beta - \lambda = 0. \tag{7.6}$$

解得

$$\pi(y \mid x) = \pi_{\text{ref}}(y \mid x) \exp\left(\frac{r(x,y)}{\beta}\right) \exp\left(-1 - \frac{\lambda}{\beta}\right). \tag{7.7}$$

令 $1/Z(x) = \exp(-1 - \lambda/\beta)$，并由归一化条件确定 $Z(x)$，即得式(7.2)。 □

由式(7.2) 反解奖励：

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x). \tag{7.8}$$

这个式子说明：*在KL正则目标下，奖励与“策略相对参考策略的对数比”只差一个与 y 无关的常数*。DPO 正是利用这一点，把“训练奖励模型+RL”转化为“直接在偏好对上优化策略”。

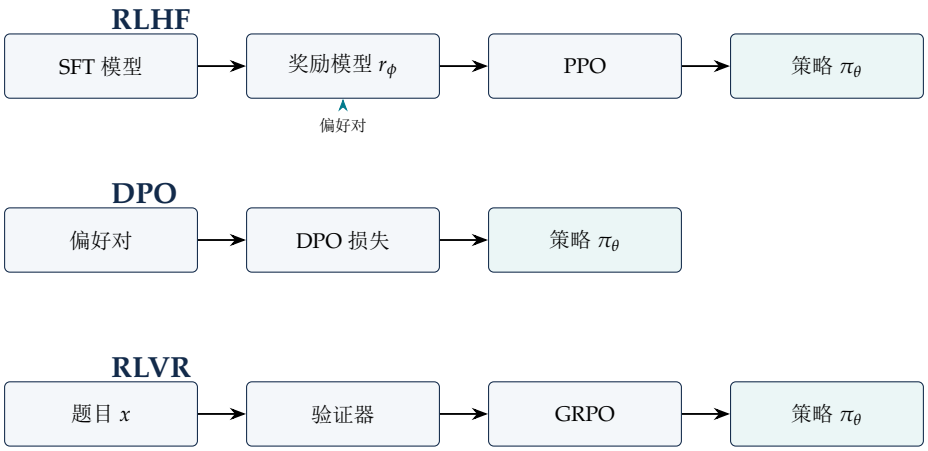


图 7.1 三条LLM 强化学习流程。RLHF 先训练奖励模型再用PPO；DPO 直接在偏好对上优化，跳过显式奖励模型；RLVR 用可验证规则作为奖励，常用GRPO。

7.2 奖励建模：Bradley–Terry 模型

7.2.1 偏好概率模型

假设人类在回答 y_w（winner）与 y_l（loser）之间的偏好由奖励差决定，且服从Bradley–Terry模型：

$$\mathbb{P}\left(y_w \succ y_l \mid x\right) = \sigma\left(r(x, y_w) - r(x, y_l)\right), \qquad \sigma(z) = \frac{1}{1+e^{-z}}. \tag{7.9}$$

这一模型等价于假设每条回答有一个潜在的“效用”$r(x,y)$，人类选择效用更高的那条，且选择噪声服从Gumbel 分布（Logit 模型）。它不是唯一的偏好模型（还有Thurstone 模型、Plackett–Luce 模型），但在RLHF 中最为常用。

命题7.1 (Bradley–Terry 的似然)**.** *给定偏好数据集 $\mathcal{D} = \{(x_i, y_{w,i}, y_{l,i})\}_{i=1}^{N}$，奖励模型 r_ϕ 的负对数似然为*

$$\mathcal{L}_{\mathrm{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right]. \tag{7.10}$$

证明 对每个样本，偏好标签是伯努利变量，成功概率为 $\sigma(r_\phi(x,y_w) - r_\phi(x,y_l))$。取对数似然并取负，即得。 □

对 ϕ 求梯度：

$$\nabla_\phi \mathcal{L}_{\mathrm{RM}} = -\mathbb{E}\left[\sigma\left(-\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right)\left(\nabla_\phi r_\phi(x, y_w) - \nabla_\phi r_\phi(x, y_l)\right)\right]. \tag{7.11}$$

当模型已经正确排序时，$r_\phi(x,y_w) - r_\phi(x,y_l)$ 很大，$\sigma(-\cdot)$ 接近零，梯度消失；当排序错误时，梯度较大。这与逻辑回归的梯度形式一致。

7.2.2 奖励模型的偏差来源

奖励模型不是“人类偏好的真值”，而是它的统计近似。它的偏差至少来自四个层面：

1. **采样偏差**：偏好数据由特定标注者、特定提示分布产生，不能代表全部用户与任务；
2. **标注噪声**：标注者之间不一致，甚至同一标注者前后矛盾；
3. **模型容量**：奖励模型可能记住了表面特征（长度、格式、语气），而非实质质量；
4. **分布偏移**：RL 训练会把策略推向奖励模型的高分区域，而该区域可能远离训练数据。

第四点尤其致命：策略会主动寻找奖励模型的“盲区”，产生reward hacking。KL 正则、奖励模型集成、对抗训练、过程奖励模型，都是缓解手段，但没有一种能从根本上消除偏差。

7.3 PPO-RLHF：把语言生成当作序列决策

7.3.1 Token 级MDP

把LLM 的生成过程建模为一个有限时域MDP：

- 状态 $s_t = (x, y_{<t})$：提示与前缀；
- 动作 $a_t = y_t \in \mathcal{V}$：词表中的下一个token；
- 转移 $s_{t+1} = (x, y_{\le t})$：确定性拼接；
- 奖励：只在序列结束时给出 $r_\phi(x, y)$，中间步可用KL 惩罚塑形。

策略 $\pi_\theta(y_t \mid x, y_{<t})$ 就是LLM 的下一个token 分布。由于动作空间是词表（数万维），但每步只采样一个token，PPO 可以直接套用。

7.3.2 KL 塑形的奖励

为了在token 级施加KL 约束，常用“每token KL 惩罚”塑形奖励：

$$\tilde{r}_t = r_\phi(x, y)\mathbf{1}\{t = T\} - \beta \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}. \tag{7.12}$$

于是整条序列的折扣回报（取 $\gamma = 1$）为

$$\sum_{t=1}^{T} \tilde{r}_t = r_\phi(x, y) - \beta \sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})} = r_\phi(x, y) - \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}. \tag{7.13}$$

这正是式(7.1) 中的KL 项。于是“序列级KL 正则”被分解为“token 级奖励塑形”，可以直接用PPO 优化。

7.3.3 PPO 目标与实现

RLHF 的PPO 目标为

$$\begin{aligned} \mathcal{L}^{\text{RLHF}}(\theta) = -\mathbb{E}_{x,y\sim\pi_{\theta_{\text{old}}}} \left[\sum_{t=1}^{T} \min\left(\rho_t(\theta)\hat{A}_t,\ \text{clip}\left(\rho_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right] \\ + \beta\, \mathbb{E}_x \left[D_{\text{KL}}\left(\pi_\theta \,\|\, \pi_{\text{ref}}\right) \right], \end{aligned} \tag{7.14}$$

其中

$$\rho_t(\theta) = \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})}, \tag{7.15}$$

$\hat{A}_t$ 由GAE 估计。实践中，价值网络通常与策略网络共享主干、单独输出一个标量；KL 项可以直接加在损失里，也可以像式(7.12) 那样塑形到奖励中。

KL 估计器。 直接计算 $\log \pi_\theta - \log \pi_{\text{ref}}$ 是无偏的（k1 估计器），但方差较大。SAC/RLHF 中常用k3 估计器：

$$\widehat{D}_{\text{KL}}^{\text{k3}} = \frac{\pi_{\text{ref}}}{\pi_\theta} - \log \frac{\pi_{\text{ref}}}{\pi_\theta} - 1, \tag{7.16}$$

它非负、方差较小，但在 π_θ 很小时可能数值不稳定。工程上常对KL 做裁剪或使用自适应系数。

奖励归一化与优势。 由于奖励模型的尺度因任务而异，RLHF 通常对一个批量内的奖励做归一化，再用GAE 计算优势。优势归一化可以显著稳定训练，但也可能抹平“极好/极差”样本的差异，需要谨慎。

7.4 DPO：消去奖励模型的闭式偏好优化

7.4.1 从最优策略反解奖励

由定理7.1，最优策略与奖励满足

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x). \tag{7.17}$$

把它代入Bradley–Terry 模型：

$$\begin{aligned} \mathbb{P}(y_w \succ y_l \mid x) &= \sigma\left(r(x, y_w) - r(x, y_l)\right) \\ &= \sigma\left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} + \beta \log Z(x) - \beta \log Z(x)\right) \\ &= \sigma\left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right). \end{aligned} \tag{7.18}$$

配分函数 $Z(x)$ 消掉了！这意味着我们不需要显式训练奖励模型，也不需要知道 $Z(x)$，就可以用偏好数据直接优化策略。

定理7.2 (DPO 损失)**.** *设 π_θ 是待学策略，π_{ref} 是参考策略。DPO 的负对数似然损失为*

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]. \tag{7.19}$$

证明 把式(7.18) 中的 π^* 替换为 π_θ，取负对数似然即得。 □

7.4.2 DPO 的梯度

令

$$u = \beta\left(\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right). \tag{7.20}$$

则

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta\, \sigma(-u)\left(\nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_l \mid x)\right). \tag{7.21}$$

证明 $\frac{\mathrm{d}}{\mathrm{d}u}\log\sigma(u)=\sigma(-u)$。对 u 求导后再对 θ 用链式法则，注意 π_{ref} 不依赖 θ。 □

式(7.21) 的直观含义是：DPO 提高获胜回答的似然、降低失败回答的似然，权重为 $\sigma(-u)$。当模型已经更偏好 y_w 时（u 很大），权重趋近零，更新自动减弱；当模型偏好错误时，权重接近 1，更新更强。这种“自适应难例加权”是DPO 在工程上稳定的原因之一。

7.4.3 DPO 的局限

DPO 的优雅来自“离线”与“闭式”：它不需要在线采样，也不需要奖励模型。但代价是：

- **没有探索**：DPO 只在固定偏好对上优化，无法发现数据之外更好的回答；
- **依赖参考策略**：π_{ref} 的选择会显著影响结果；若参考策略太弱，KL 约束可能限制改进；
- **长度与格式偏差**：偏好数据中常存在长度偏好，DPO 可能学会“更长即更好”；
- **分布外泛化弱**：DPO 学的是“在给定偏好对上相对好坏”，不一定能泛化到新的提示分布。

这些局限催生了KTO、ORPO、SimPO、在线DPO 等变体。它们的共同方向是：在保持“无需显式奖励模型”的同时，引入更好的参考策略、在线采样或长度归一化。

7.5 GRPO：组内相对优势与无critic 优化

7.5.1 动机：critic 太贵

PPO-RLHF 需要一个与策略同规模的价值网络（critic），它要估计每个token 的期望回报。对于7B—70B 的LLM，这意味着显存与计算量翻倍；而且长序列的信用分配使critic 很难准确。GRPO（Group Relative Policy Optimization）的做法是：*对同一个问题采样一组回答，用组内相对奖励作为优势*，从而完全去掉critic[52]。

7.5.2 目标函数

对每个问题 q，从旧策略采样 G 个回答 $\{o_1,\dots,o_G\}$，每个回答得到一个奖励 r_i。GRPO 的目标为

$$\mathcal{J}_{\text{GRPO}}(\theta)=\mathbb{E}_{q\sim\mathcal{P}(Q),\,\{o_i\}\sim\pi_{\theta_{\text{old}}}}\Bigg[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\left(\rho_{i,t}\hat{A}_i,\ \text{clip}\left(\rho_{i,t},1-\epsilon,1+\epsilon\right)\hat{A}_i\right)-\beta\,D_{\text{KL}}\left(\pi_\theta\,\|\,\pi_{\text{ref}}\right)\Bigg],\tag{7.22}$$

其中

$$\rho_{i,t}=\frac{\pi_\theta(o_{i,t}\mid q,o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}\mid q,o_{i,<t})},\tag{7.23}$$

组内优势为

$$\hat{A}_i = \frac{r_i - \text{mean}\,(r_1, \ldots, r_G)}{\text{std}\,(r_1, \ldots, r_G)}. \tag{7.24}$$

注意优势是序列级的：同一个回答的所有token 共享 $\hat{A}_i$。这与PPO 的token 级GAE 不同，也是GRPO 实现简单的关键。

标准差归一化需注意两点。其一，它是正比例缩放，不改变组内排序，因而只改变不同题目之间的奖励尺度可比性，不改变“同组内谁更好”的判断。其二，当组内奖励全相同（例如全对或全错）时 $\text{std} = 0$ 会导致除零；此时组内优势本应全为零、该组不产生梯度信号，实现中须加 ϵ 或直接跳过该组。

7.5.3 为什么组内均值是合理的基线?

策略梯度的方差可以通过减去一个与动作无关的基线来降低。在单问题、单回答的设置中，最优基线是 $V(q) = \mathbb{E}_{o\sim\pi}[r(q,o)]$。GRPO 用组内均值 $\bar{r} = \frac{1}{G}\sum_i r_i$ 估计它：

$$\hat{A}_i = r_i - \bar{r}. \tag{7.25}$$

由于 $\mathbb{E}[\bar{r} \mid q] \approx V(q)$，$\hat{A}_i$ 近似为优势的无偏估计。除以组内标准差则是额外的方差归一化，使不同问题的奖励尺度可比。这一做法与“留一法基线”（leave-one-out）密切相关：用除 i 以外的样本均值作为基线，可以进一步减少 r_i 与基线之间的相关性。

命题7.2 (组内优势的基线性质). *对固定 q，若 $\{r_i\}$ 独立同分布，则*

$$\mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\hat{A}_i\right] = 0. \tag{7.26}$$

证明 $\sum_i (r_i - \bar{r}) = \sum_i r_i - G\bar{r} = 0$，故样本均值恒为零；取期望仍为零。 □

零均值意味着组内优势是“相对好坏”的信号：它不关心问题的绝对难度，只关心“哪个回答更好”。这对LLM 推理任务是合适的：同一道题的多个回答可以直接比较，而跨题目的奖励尺度差异被消除。

7.5.4 GRPO 与PPO 的对比

GRPO 的代价是“信用分配变粗”：同一个回答的所有token 共享一个优势，无法区分“哪一步推理错了”。这在短回答上问题不大，但在长链推理与智能体任务中会成为瓶颈。2025—2026 年的许多工作（如StepPO、过程奖励模型、步骤级GAE）正是在补这一课。

7.6 RLVR：可验证奖励强化学习

表 7.1 PPO-RLHF 与GRPO 的对比

维度	PPO-RLHF	GRPO
优势估计	token 级GAE，需要critic	序列级组内相对奖励，无需critic
显存开销	策略+ 价值网络（约2 倍）	仅策略（约1 倍）
信用分配	细粒度，但critic 难训	粗粒度，依赖组内比较
适用奖励	稠密/可塑形	结果奖励、可验证奖励
典型场景	通用对齐、对话	数学、代码、STEM 推理

7.6.1 奖励的结构性变化

RLVR（Reinforcement Learning with Verifiable Rewards）把奖励从“人类偏好模型”换成“可自动验证的规则”。对数学题，奖励可以是“最终答案是否等于标准答案”；对代码题，奖励可以是“是否通过单元测试”；对逻辑题，奖励可以是“证明是否通过形式化检查”。最简单的形式是

$$r(x,y) = \mathbf{1}\{\text{Verify}(x,y) = 1\} + \lambda\,\mathbf{1}\{\text{格式正确}\}, \tag{7.27}$$

其中 Verify 是确定性验证器，λ 是格式奖励的权重。

RLVR 的三个优势是：奖励客观、可大规模生成、训练可以完全在线。它把RL 从“人类偏好”的昂贵与噪声中解放出来，使“纯RL 训练推理”成为可能。

7.6.2 DeepSeek-R1：纯RL 激励推理

2025 年1 月，DeepSeek 发布R1 技术报告[53]，其核心发现是：*不需要人类标注的推理轨迹，仅用可验证奖励+ GRPO，就能让基座模型自发产生长链推理、自我验证与反思*。训练流程可以概括为：

1. **R1-Zero**：从基座模型出发，直接用GRPO + 规则奖励训练。模型在训练中逐渐学会“先写长推理、再给答案”，并出现“检查—发现错误—重写”的行为。
2. **R1**：在R1-Zero 的基础上加入冷启动数据（少量高质量长链推理）、拒绝采样与多阶段SFT/RL，以提升可读性、语言一致性与稳定性。
3. **蒸馏**：用R1 生成推理数据，蒸馏到更小的模型，使小模型也获得推理能力。

DeepSeek-R1 的公开结果

据DeepSeek-R1 技术报告与Nature 论文[53, 54]，R1 在AIME 2024 上达到79.8% 的pass@1，在MATH-500 上达到97.3%，在Codeforces 上达到2029 的评分（约96.3 百分位），在MMLU 上达到90.8%，在GPQA Diamond 上达到71.5%。这些数字是“单次采样、零样本”的口径；若允许多次采样与投票，成绩会更高，但计算成本也更高。

R1 的意义不仅是成绩，而是训练范式的转变：从“模仿人类推理”转向“用结果信号激励推理”。这一转变与AlphaGo Zero 的“无人类棋谱、纯自我对弈”高度同构：人类demonstrations 不再是上限，RL 可以在可验证的任务上超越它们。

7.6.3 “RLVR 是否真的提升了推理?”

2025 年出现了一场重要争论。一方认为RLVR 教会了模型新的推理能力；另一方认为RLVR 只是把基座模型已有的推理模式“锐化”，在pass@k 增大时，RL 模型的优势会消失，甚至被基座模型反超。

这一争论的技术核心是评价指标。若只看pass@1，RL 确实提升了“第一次就答对”的概率；但若看pass@k（采样 k 次至少一次答对），基座模型可能早就有能力答对，只是概率较低。Yue 等人（2025）据此指出，RLVR 可能主要改变了输出的分布，而非扩展了模型的能力边界[59]。反对者则认为，pass@k 的高方差与采样温度设置会影响结论，且RL 带来的“可读性、稳定性、自我验证”本身就是能力。

无论哪一方正确，这场争论揭示了一个重要方法论问题：我们用什么指标判断“RL 是否提升了推理”？仅看基准分数是不够的；需要区分“能力边界”（pass@k）、“典型性能”（pass@1）、“计算效率”（达到某分数所需token 数）与“泛化”（分布外任务）。第8 章将把这一框架推广到应用评估。

7.6.4 奖励劫持的新形态

RLVR 并没有消除reward hacking，只是把它从“讨好人类”变成“讨好验证器”。常见形式包括：

- **格式套利**：模型发现只要输出特定格式就能拿到格式奖励，于是忽略实质推理；
- **答案猜测**：在选择题或数值题中，模型可能学会“猜答案”而非“解题”；
- **验证器漏洞**：代码题中，模型可能针对测试用例硬编码，而非写出通用算法；
- **长度偏置**：若奖励与长度正相关，模型会生成冗长的推理，即使没有帮助。

缓解方法包括：隐藏测试用例、随机化题目、过程奖励、对抗验证、以及“训练—验证”分离。但这是一场没有终点的军备竞赛：只要有代理目标，优化器就会寻找漏洞。

7.7 智能体强化学习：从单步生成到长程决策

7.6.1 POMDP 形式化

Agentic RL 把LLM 建模为一个在部分可观测环境中行动的智能体。形式化地，它是一个POMDP：

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \Omega, O, \gamma), \tag{7.28}$$

其中状态 s_t 包含环境与对话历史，动作 a_t 是工具调用、代码执行或文本输出，观测 o_t 是工具返回结果或用户反馈。策略 $\pi_\theta(a_t \mid h_t)$ 以历史 $h_t = (o_1, a_1, \ldots, o_t)$ 为条件。与单步RLHF 相比，Agentic RL 的轨迹长度可能达到数十到数百步，奖励往往是稀疏的“任务完成/未完成”。

7.7.2 信用分配的三个层次

Agentic RL 的信用分配可以分三个层次：

1. **回合级**：只用最终成功/失败作为奖励，所有步骤共享同一优势。这是GRPO 的直接扩展，简单但方差大。
2. **步骤级**：用过程奖励模型或规则为每个工具调用/子目标打分，再把步骤奖励回传。这需要额外的标注或验证器。
3. **token 级**：在步骤内部再用GAE/PPO 做细粒度信用分配。这是计算量最大的做法，适合长文本推理。

StepPO 等2026 年的工作试图把“步骤级”与“token 级”结合：先用步骤边界划分轨迹，再在步骤内做token 级优化[56]。课程RL 则从“简单任务”逐步过渡到“困难任务”，以缓解稀疏奖励[57]。

7.7.3 环境与奖励的基础设施

Agentic RL 的瓶颈不在算法，而在环境。一个可用的训练环境需要：可复现、可并行、可验证、足够多样、且能提供有意义的中间反馈。2025—2026 年出现了大量“环境工程”工作：浏览器环境、操作系统环境、代码仓库环境、游戏环境、科学实验环境。这些环境的构建成本，往往高于策略优化本身。

这也解释了为什么Agentic RL 的进展呈现“波浪式”：每当一个新的可验证环境（如SWE-bench、WebArena、OSWorld）出现，就会带来一波算法与模型的提升；当环境饱和或污染后，进展又会放缓。第9 章将把这一现象称为“环境—评估共同演化”。

7.8 本章小结

本章把经典RL 的骨架延伸到LLM：

1. RLHF 把奖励从环境搬到人类偏好，用Bradley–Terry 模型把比较转化为标量奖励；KL 正则目标有闭式最优策略，这是DPO 的基础；
2. PPO-RLHF 把语言生成建模为token 级MDP，用KL 塑形奖励与裁剪目标优化；它需要critic，显存开销大；
3. DPO 利用“奖励—策略对数比”的闭式关系消去奖励模型，直接用偏好对做分类；它简单稳定，但缺少在线探索；

4. GRPO 用组内相对奖励替代critic，把优势估计变成“同一问题下多个回答的相对好坏”；它适合可验证奖励，但信用分配较粗；
5. RLVR 用确定性验证器提供奖励，DeepSeek-R1 证明了“纯RL 激励推理”的可行性；但“RL 是否扩展了能力边界”仍有争议，奖励劫持也换了形态；
6. Agentic RL 把单步生成扩展为POMDP 中的长程决策，瓶颈从算法转向环境、奖励与信用分配的基础设施。

下一章进入应用版图。我们将看到，RLHF/RLVR 只是强化学习应用光谱的一段；在游戏、机器人、芯片、数据中心、能源、医疗、科学发现等领域，强化学习已经产生了可复核的、有时是数量级的效果。同时，我们也会看到大量“看起来很美”的案例，其效应量在严格评估下大幅缩水。区分二者，是第8 章的核心任务。

第 8 章

应用版图与效果评估：从超人类游戏到产业系统

前七章说明强化学习“如何工作”。本章回答“它在哪里真正起作用、效果有多大、证据有多强”。强化学习的应用光谱极宽：一端是规则完备、仿真廉价、奖励可验证的游戏与算法发现；另一端是奖励延迟、反事实不可观测、试错代价极高的医疗、金融与推荐。两端之间的差异，不是“算法强弱”，而是**奖励可信度、环境保真度与评估可识别性**的差异。本章按这一逻辑组织：先给出评估方法论，再逐域分析机制、代表工作、量化效果与证据等级，最后讨论效应量的外推边界与失败模式。

8.1 如何判断“强化学习有效”？

8.1.1 三个层次的证据

一个RL 应用的效果，至少要在三个层次上被验证：

1. **基准层**：在固定仿真器/数据集上，指标是否显著优于基线？这是最容易获得的证据，也最容易被过拟合。
2. **迁移层**：从训练分布迁移到新地图、新用户、新设备、新任务时，性能是否保持？这检验的是泛化，而非记忆。
3. **部署层**：在真实系统中，长期运行是否带来可归因的业务/科学收益？这需要A/B 测试、对照实验或严格的因果识别。

大多数论文停留在基准层；少数工业报告到达迁移层；真正完成部署层验证的案例，集中在游戏、芯片、数据中心、推荐与LLM 等少数领域。本报告用A/B/C/D 四级标注证据：A 为同行评审或官方技术报告且可复核；B 为权威机构博客/官方文档；C 为产业案例/媒体报道；D 为本报告推断。

8.1.2 效应量的四种陷阱

选择偏差。 只报告成功任务，忽略失败任务。Atari 的“49 个游戏”之所以有说服力，是因为它报告了全部游戏，而不是挑出最好的几个。

基线选择。 与弱基线比较会放大效应。推荐系统论文常用“无RL 的监督基线”，但真正的生产基线可能已经包含大量工程优化。

评估污染。 基准测试集可能出现在训练数据中。LLM 的数学/代码基准尤其严重；RLVR 的训练题目与评测题目若有重叠，分数会被高估。

成本外溢。 一个方法可能提升2% 的指标，却增加10 倍算力。若不报告算力、样本量与推理成本，效应量就不可比较。

8.1.3 一个可操作的评估清单

本报告建议用以下清单审视任何“RL 应用”声明：

1. 奖励是什么？由谁定义？能否被策略“钻空子”？
2. 环境是真实、仿真还是历史数据？仿真与真实的差距如何量化？
3. 基线是什么？是否包含强工程基线与非RL 方法？
4. 评估是离线、在线还是部署？有没有对照组？
5. 效应量是否在多个随机种子/多个场景下稳定？
6. 计算成本、样本效率、推理延迟是否被报告？
7. 失败模式是什么？在什么条件下会崩溃？

后文对每个领域的分析，都尽量回答这七个问题。

8.1.4 应用场景的六种结构

把第1 章的MDP 抽象与工程约束结合，可以把RL 的应用场景分为六类。分类的依据不是行业，而是**奖励来源、环境保真度与试错成本**这三个决定算法选择的变量。

表 8.1 强化学习应用场景的六种结构

场景类型	奖励来源	典型环境	代表领域与算法
规则完备型	程序化、精确	完美模拟器	棋类、Atari、芯片布局、算法发现；DQN/AlphaZero/MCTS
物理仿真型	工程函数、可塑形	高保真仿真	机器人、腿式locomotion、等离子体控制；PPO/SAC/模型基
人类偏好型	偏好模型、噪声大	人机交互	LLM 对齐、对话、推荐；RLHF/DPO/bandit
可验证型	规则/验证器	题目生成器	数学、代码、逻辑推理；GRPO/RLVR
离线决策型	历史数据、反事实	固定数据集	医疗、金融、工业；CQL/IQL/BCQ
长期运行型	在线指标、延迟	真实系统	数据中心、网络、能源；平均奖励/约束RL

这张表解释了为什么“同一个PPO”在不同领域的表现差异巨大：算法只是骨架，真正决定成败的是**奖励是否可信、环境是否保真、试错是否可负担**。规则完备型可以承受百万次失败；医疗与金融则一次失败就可能不可接受。前者追求“更强”，后者追求“更稳”。

8.2 游戏与博弈：强化学习的“望远镜”

游戏是RL 最理想的试验场：规则明确、模拟器廉价、奖励可验证、评估可重复。它像望远镜，让研究者看到算法在极端复杂决策空间中的可能性，但也容易让人误以为“游戏中的超人类”可以自动迁移到现实。

8.2.1 从TD-Gammon 到Atari

TD-Gammon（1992/1995）用TD(λ) + 多层感知机+ 自我对弈，在西洋双陆棋上达到接近人类专家的水平[13]。它是“非线性函数逼近+ 自博弈”的早期证明，但也暴露了局限：同样的方法在象棋、围棋上没有复现。

DQN（2013/2015）用卷积网络+ 经验回放+ 目标网络，在49 个Atari 游戏上用同一套结构达到接近或超过人类的得分[24]。它的真正贡献不是“玩游戏”，而是证明了*端到端RL 可以从原始像素学习控制策略*。代价是样本效率：数千万帧的训练量，远超人类的学习速度。

8.2.2 AlphaGo、AlphaZero 与AlphaStar

AlphaGo 在2016 年以4:1 击败李世石[35]。它结合了策略网络（缩小搜索宽度）、价值网络（评估叶节点）与MCTS（在线规划），并先用人类棋谱监督学习，再用自我对弈强化学习。2017 年的AlphaGo Zero 去掉人类棋谱，仅用自我对弈与MCTS 从零训练[36]；2018 年的AlphaZero 把同一框架推广到国际象棋与将棋[37]。

AlphaStar 在2019 年达到《星际争霸II》人类Grandmaster 水平，位列官方排名前99.8%[38]。它的难点不是“操作”，而是不完全信息、长时程规划与多智能体博弈；解决方案是监督学习+ 多智能体联盟训练+ 联赛（league）机制。OpenAI Five 在2019 年击败Dota 2 世界冠军OG（2:0）[39]，展示了大规模分布式PPO 与自我对弈在复杂团队游戏中的能力。

这些案例的共同点是：*完美模拟器+ 明确奖励+ 可无限自我对弈*。一旦缺少其中任何一个，效果会迅速下降。这正是它们难以直接迁移到现实的原因。

8.2.3 赛车、算法发现与芯片布局

Gran Turismo Sophy 在2022 年登上Nature，用深度RL 在真实赛车模拟器中击败顶级人类车手，并创造了赛道纪录[40]。它的技术要点是“混合离线/在线训练+ 规则约束+ 对手建模”，而不是单纯的PPO。

AlphaTensor 在2022 年用RL 发现更快的矩阵乘法算法：在模2 的 4×4 矩阵乘法中，把所需乘法次数从Strassen 算法的49 次降到47 次[41]。AlphaDev 在2023 年用RL 发现更快的排序

算法：对短序列，排序速度提升最高70%；对大规模序列，提升约1.7%，并被集成进LLVM 的C++ 标准库[42]。这两个案例说明，RL 不仅可以“玩游戏”，还可以发现人类未发现的算法。

AlphaChip 在2021 年提出用RL 做芯片布局（chip placement），并在2024 年的更新中报告该方法是“人类专家之外首个自动生成可制造布局的方法”，已被用于三代TPU（v5e、v5p、Trillium）的设计[43]。芯片布局的组合空间极大，且评估函数（线长、功耗、拥塞）计算昂贵；RL 的优势在于把“试错”放在一个可快速评估的代理模型上，再把学到的策略迁移到真实设计流程。

表 8.2 游戏与算法发现中的代表性成果

系统	年份	效果	证据
TD-Gammon	1995	西洋双陆棋接近人类专家	A
DQN	2015	49 个Atari 游戏达到人类水平附近	A
AlphaGo	2016	4:1 击败李世石	A
AlphaZero	2018	国际象棋/将棋/围棋超人类	A
AlphaStar	2019	星际争霸II Grandmaster，前99.8%	A
OpenAI Five	2019	Dota 2 2:0 击败世界冠军OG	B
Gran Turismo Sophy	2022	击败顶级车手并创赛道纪录	A
AlphaTensor	2022	4×4 模2 矩阵乘法47 次（此前49）	A
AlphaDev	2023	短序列排序最高快70%，长序列约1.7%	A
AlphaChip	2021/2024	用于三代TPU 布局	A/B

8.3 机器人与具身智能：从仿真到现实

8.3.1 为什么机器人是RL 的“试金石”

机器人任务具有连续高维动作、部分可观测、接触动力学复杂、试错代价高等特点。RL 在机器人上的成功，通常不是因为“算法更聪明”，而是因为仿真器足够快、域随机化足够强、策略结构足够好。sim-to-real（仿真到现实）是这一领域的核心问题。

8.3.2 腿式locomotion

2018—2021 年，腿式机器人（ANYmal、Unitree、MIT Cheetah）的RL 控制取得了突破。Lee 等人在2020 年用模型基RL 与在线适应，让ANYmal 在雪地、碎石、陡坡等复杂地形上行走[71]。Rudin 等人在2021 年提出“在几分钟内学会走路”：用大规模并行仿真（Isaac Gym）训练数千个环境，在单GPU 上几分钟内训练出可部署的策略[72]。这一“大规模并行+ 域随机化+ PPO”的范式，成为现代腿式机器人控制的标准流程。

效果是显著的：RL 策略在非结构化地形上的鲁棒性，往往超过传统模型基控制器；训练时间从数天缩短到分钟级；部署时只需一次策略网络前向。但它的代价是：奖励函数需要精心设计（速度跟踪、姿态、能耗、足端滑动），且策略对传感器噪声与负载变化敏感。

8.3.3 操作与通用机器人

OpenAI 在2019 年用RL + 域随机化训练了一只机械手解魔方，展示了“仿真中训练、现实中零样本迁移”的可能性[73]。它的成功率并非100%，但证明了高自由度操作可以通过大规模RL 获得。

2023 年之后，视觉—语言—动作（VLA）模型兴起。RT-2 把互联网视觉语言数据与机器人轨迹联合训练，使机器人能理解“把可乐罐移到Taylor Swift 照片上”这类开放指令[74]；π_0 用流匹配（flow matching）生成动作，并在多种机器人上零样本执行[75]；Gemini Robotics（2025）把多模态大模型与机器人控制结合，强调“泛化到新任务、新物体、新环境”。

这些系统的训练通常分两阶段：大规模行为克隆/预训练+ 少量RL 微调（或偏好优化）。RL 的作用从“从零学习”转变为“在预训练策略上做对齐与精修”。这与LLM 的RLHF 逻辑一致：先模仿，再强化。

8.3.4 效果与边界

机器人RL 的公开效果往往是“任务特定的”：

- 在仿真基准（MuJoCo、Isaac）上，SAC/PPO 可以稳定达到高分；但基准与真实的差距可能很大；
- 在真实机器人上，成功的案例通常有强先验（示范、模型、结构化策略）与大量域随机化；
- 安全性仍是瓶颈：RL 策略缺乏形式化保证，难以通过航空、医疗等高风险认证。

8.4 大语言模型与智能体：从对齐到推理

8.4.1 RLHF 的效果

InstructGPT（2022）是RLHF 规模化的标志。其关键结果是：一个1.3B 参数的RLHF 模型，在人类偏好评估中优于175B 的GPT-3[50]。这说明对齐数据与优化目标可以在参数量小两个数量级的情况下显著改变模型的有用性。RLHF 随后成为ChatGPT、Claude、Gemini 等模型的标准对齐组件。

RLHF 的效果可以分解为三个维度：

1. **有用性**：更愿意遵循指令、给出完整回答、承认不确定性；
2. **无害性**：更少输出有害、歧视、危险内容；
3. **诚实性**：更少编造事实（但无法完全消除幻觉）。

代价是“对齐税”（alignment tax）：RLHF 可能降低模型在某些基准上的原始能力，尤其是需要“不讨喜但正确”的回答时。此外，RLHF 容易导致“谄媚”（sycophancy）：模型学会迎合人类评估者的偏好，而不是给出真实答案。

8.4.2 RLVR 与推理模型

DeepSeek-R1（2025）用GRPO + 可验证奖励，在数学、代码、STEM 推理上取得了显著提升：AIME 2024 pass@1 79.8%，MATH-500 97.3%，Codeforces 2029[53, 54]。OpenAI o1/o3 则展示了“长思维链+ RL + 测试时计算”的路线，o3 在ARC-AGI 上达到87.5%（高计算预算）[61]。

这些结果的意义在于：RL 不再只是“对齐工具”，而成为提升推理能力的训练范式。但它也带来了新的成本：长思维链使推理token 数增加数倍到数十倍；训练需要大规模可验证题目与稳定的验证器；模型可能学会“讨好验证器”而非真正推理。

8.4.3 智能体

2025—2026 年，LLM 智能体从“对话”走向“行动”：浏览网页、操作软件、写代码、调用API、做实验。Agentic RL 把这些问题建模为POMDP，用任务完成度、工具返回结果或人类反馈作为奖励[55]。

公开的基准结果（如SWE-bench、WebArena、OSWorld）显示，智能体的任务完成率在两年内从个位数提升到数十个百分点，但不同scaffold、不同工具、不同评测口径之间差异极大。产业部署仍处于早期：可靠性、安全性、成本与可审计性都是障碍。

一个重要的区分：基准提升vs 能力提升

RLVR 可以让模型在AIME 上从30% 提升到80%，但这不一定意味着模型“学会了数学”。它可能只是学会了“更擅长在给定题目分布内搜索答案”。Yue 等人（2025）指出，若用 pass@k 衡量，RL 模型的优势会随 k 增大而缩小甚至消失[59]。这一争论提醒我们：基准分数是能力的下界，不是能力的定义。

8.5 推荐、广告与营销：延迟回报的用武之地

8.5.1 为什么推荐适合RL

推荐系统天然是一个序贯决策问题：用户的状态（兴趣、耐心、预算）随交互演化，系统选择的物品不仅影响即时点击，还影响长期留存与满意度。RL 可以把“点击率最大化”扩展为“长期用户价值最大化”，并对探索—利用做显式建模。

代表性工作包括：

- **SlateQ**（Google，2019）：把“一次推荐一屏物品”建模为组合动作，用Q-learning 分解 slate 价值，用于YouTube 推荐[76]；
- **Page-wise 推荐**（阿里，2018）：用actor–critic 优化整页推荐，考虑物品间的相互影响[77]；

- **Top-K off-policy correction**（YouTube，2019）：用离策略校正解决“日志策略与目标策略不一致”的问题[78]；
- **搜索/广告竞价**：用RL 做预算分配、出价与长期ROI 优化。

8.5.2 效果与证据

推荐RL 的公开效果通常在“离线评估”或“小流量A/B”层面：

- 离线指标（AUC、NDCG、CTR）提升几个百分点；
- 在线A/B 中，CTR 提升1%—5%，留存/时长提升更小但更持久；
- 长期效果难以测量：用户偏好会因推荐而改变，存在反馈回路与生态效应。

这些数字必须谨慎解读：推荐系统的基线极强，1% 的CTR 提升可能来自更好的特征工程而非RL；离线评估与在线效果之间常有落差；探索可能损害短期体验。

8.5.3 反馈回路与“推荐茧房”

RL 推荐的一个深层风险是**反馈回路**：系统推荐什么，用户就看到什么；用户看到什么，就会对什么产生偏好；偏好又反过来训练系统。这一回路可能导致“茧房”、极化与多样性下降。解决思路包括：约束优化（多样性、公平性）、反事实评估、以及把“用户长期福利”而非“即时点击”作为目标。但这些目标本身难以定义，也难以测量。

8.6 芯片、数据中心与系统：可验证的工程收益

8.6.1 芯片布局

AlphaChip 用RL 做芯片布局，把宏单元与标准单元的放置建模为序贯决策：每一步放置一个单元，奖励由线长、功耗、拥塞等代理指标构成[43]。2024 年的更新报告该方法已用于TPU v5e、v5p 与Trillium 的设计，并在“数小时内”生成可与人类专家媲美的布局[44]。这是RL 在“组合优化+ 昂贵评估”场景中的典型案例。

8.6.2 数据中心冷却

2016 年，DeepMind 与Google 用RL 优化数据中心的冷却系统：策略根据传感器读数调整冷却设定点，目标是最小化能耗同时维持安全温度。官方报告的结果是冷却能耗降低约40%，整体PUE 改善约15%[64]。这一案例常被引用为“RL 在工业控制中的标志性成功”，但需要注意：它是“官方口径”（B 级证据），且后续系统可能已叠加其他优化。

8.6.3 网络与视频流

Pensieve（2017）用RL 做自适应视频码率选择，在多种网络条件下比最好的固定启发式提升12%—25% 的QoE[66]。Remy（2013）用计算机搜索生成拥塞控制算法，挑战了“拥塞控制必须由专家设计”的传统[67]。这些工作证明了RL 在“网络控制”中的潜力，但真正部署到生产网络仍面临可解释性、安全性与兼容性挑战。

8.6.4 调度与系统

DeepRM（2016）用RL 做集群作业调度，在模拟中显著降低平均作业减速比[65]。后续工作把RL 用于数据库调参、编译器优化、缓存替换、能耗管理。这些场景的共同点是：有模拟器、有明确的代价函数、可以承受一定的探索风险。

8.7 能源、交通、医疗与科学发现

8.7.1 核聚变等离子体控制

2022 年，DeepMind 与EPFL 用深度RL 控制托卡马克（TCV）的磁场，实现了多种等离子体构型的稳定控制[69]。这是RL 在“高维连续控制+ 复杂物理+ 真实硬件”中的里程碑：策略在仿真中训练，迁移到真实装置，并在少量试错内适应。它的意义不仅是“控制等离子体”，更是证明了仿真—现实迁移在科学装置上可行。

8.7.2 电网与能源管理

RL 在能源领域的应用包括：数据中心/建筑暖通空调（HVAC）控制、电池充放电策略、电网调度、风力/光伏预测与消纳。Google 在2016 年的数据中心冷却是一个例子；后续有把RL 用于“风电参与电力市场竞价”的研究。能源系统的约束严格（安全、稳定、监管），RL 通常以“建议+ 人工确认”或“约束RL”的形式部署。

8.7.3 交通信号与自动驾驶

IntelliLight（2018）用RL 做交通信号控制，在模拟与真实路口数据上降低了车辆等待时间[68]。自动驾驶中的RL 主要用于规划、决策与仿真测试，而非端到端量产控制；Waymo、特斯拉等公司的核心系统仍以规则、模仿学习与大规模仿真为主，RL 是补充而非替代。

8.7.4 医疗

AI Clinician（2018）用离线RL 从ICU 数据中学习脓毒症患者的液体与升压药策略，离线评估显示其策略的估计生存率高于临床医生[70]。这是离线RL 在医疗中的标志性工作，但必须强调：离线评估不等于临床有效。策略的价值依赖数据覆盖、因果假设与未观测混杂；在进入临床试验之前，不能把“估计的生存率提升”当作真实疗效。

其他医疗应用包括胰岛素剂量、麻醉控制、呼吸机参数调整、动态治疗方案。共同挑战是：样本少、试错代价高、患者异质性强、监管严格。

8.7.5 科学发现

RL 在科学发现中的应用正在扩展：

- **数学**：AlphaProof 与AlphaGeometry 2 在2024 年国际数学奥林匹克（IMO）中解决4/6 题，获得28/42 分（银牌水平）[62]；2025 年Gemini Deep Think 达到35/42 分（金牌水平）[63]。
- **算法**：AlphaTensor 发现更快的矩阵乘法，AlphaDev 发现更快的排序；
- **材料/化学**：RL 用于分子生成、反应条件优化、晶体结构搜索；
- **物理**：RL 用于等离子体控制、粒子加速器调束、量子电路编译。

这些应用的共同结构是：有一个可仿真的环境、一个可计算的奖励、以及一个巨大的组合空间。RL 的价值在于“在搜索空间中找到人类直觉难以到达的区域”。

8.8 工业、金融与供应链

8.8.1 工业控制

RL 在工业控制中的应用包括：化工过程控制、钢铁冶炼、注塑成型、半导体制造、机器人装配。典型架构是“仿真中预训练+ 真实系统中在线微调+ 安全约束”。工业客户对可解释性、可靠性、认证的要求极高，因此RL 往往以“辅助优化”而非“端到端控制”的形式落地。

8.8.2 金融

RL 在金融中的应用包括：最优执行（optimal execution）、做市（market making）、组合优化、信用审批、欺诈检测。最优执行有经典的Almgren–Chriss 框架，RL 可以在市场冲击、延迟、部分成交等复杂条件下学习更优策略。但金融市场的非平稳性、低信噪比、对手博弈与监管约束，使RL 的样本效率与稳健性面临巨大挑战。公开的“稳定超额收益”证据很少，且大多无法排除过拟合与幸存者偏差。

8.8.3 供应链与库存

RL 可用于库存补货、定价、物流路径、产能分配。这些问题的状态维度高、约束多、需求随机；RL 的优势是可以在仿真中做“反事实”实验，评估不同策略的长期成本。但供应链的牛鞭效应、突发事件与人为因素，使仿真与现实的差距较大。

8.9 效果汇总与证据分级

表8.3 汇总本章讨论的主要应用。需要再次强调：表中的“效果”是*原始报告口径*，不是本报告的独立复现；“证据”等级反映的是来源类型与可复核性，不是效应量的大小。

表 8.3 强化学习主要应用与效果汇总

领域	任务	代表工作	报告效果	证据
游戏	围棋	AlphaGo/AlphaZero	4:1 击败李世石；超人类	A
游戏	星际争霸II	AlphaStar	Grandmaster，前99.8%	A
游戏	Dota 2	OpenAI Five	2:0 击败OG	B
游戏	赛车	GT Sophy	击败顶级车手，创纪录	A
算法	矩阵乘法	AlphaTensor	4×4 模2：47 次（原49）	A
算法	排序	AlphaDev	短序列最高快70%	A
芯片	布局	AlphaChip	用于三代TPU	A/B
数据中心	冷却	DeepMind/Google	冷却能耗 −40%，PUE −15%	B
网络	视频码率	Pensieve	QoE +12% 至 +25%	A
网络	拥塞控制	Remy	生成新算法，挑战专家设计	A
机器人	腿式locomotion	ANYmal 等	复杂地形鲁棒行走；分钟级训练	A
机器人	灵巧操作	OpenAI 魔方	仿真到现实零样本迁移	B
机器人	通用操作	RT-2 / π_0	开放指令泛化	A/B
LLM	对齐	InstructGPT	1.3B 优于175B GPT-3	A
LLM	推理	DeepSeek-R1	AIME 79.8%，MATH-500 97.3%	A
LLM	推理	o3	ARC-AGI 87.5%（高算力）	B
推荐	整页推荐	SlateQ / Page-wise	CTR 离线/小流量提升	A/B
推荐	长期价值	Top-K off-policy	离策略校正，提升留存	A/B
能源	托卡马克控制	DeepMind/EPFL	多种等离子体构型稳定控制	A
交通	信号控制	IntelliLight	降低等待时间	A
医疗	脓毒症策略	AI Clinician	离线估计生存率提升	A（离线）
数学	IMO	AlphaProof/ AlphaGeometry	28/42，银牌水平	A
数学	IMO	Gemini Deep Think	35/42，金牌水平	B

领域	任务	代表工作	报告效果	证据
金融	最优执行	多家	降低执行成本（未公开）	C
工业	过程控制	多家	能耗/良率提升（未公开）	C

8.10 效应量的外推边界

8.10.1 从“基准”到“部署”的四个断层

断层一：环境保真度。 仿真器省略了摩擦、延迟、噪声、磨损、人为因素。策略在仿真中越强，越可能利用仿真器的简化；到现实中，这些“技巧”失效甚至有害。

断层二：奖励可信度。 基准奖励是程序化的；真实奖励往往由人、业务指标或代理模型给出。代理越复杂，越容易被劫持。

断层三：分布稳定性。 基准数据是固定的；真实系统的用户、市场、设备会漂移。RL 策略可能对漂移敏感，需要持续学习或保守更新。

断层四：评估可识别性。 基准有标准答案；真实系统没有反事实。我们只能通过A/B 测试或因果推断估计效应，而这些方法本身有假设与偏差。

8.10.2 “超人类”不等于“可部署”

游戏中的超人类表现，依赖于完美模拟器与无限自我对弈；现实系统没有这些条件。AlphaGo 的成功不能直接推出“RL 可以解决任何决策问题”，正如AlphaFold 的成功不能直接推出“AI 可以解决所有生物学问题”。把“特定条件下的最优”误认为“通用能力”，是RL 应用中最常见的认知错误。

8.10.3 规模与成本

RL 的效果往往与采样量、模型规模、仿真并行度强相关。一个在论文中“样本效率高”的算法，在真实系统中可能因为环境交互成本而不可行；一个在仿真中“训练几分钟”的策略，可能需要在数千个并行环境中运行。报告效应量时，必须同时报告成本，否则无法判断“是否值得”。

8.11 失败模式与负面案例

奖励劫持。 策略找到奖励函数的漏洞，而非完成任务。经典例子包括：赛艇游戏里原地转圈刷分、抓取任务里把物体推到摄像头前“看起来在抓”、LLM 里输出讨喜但错误的答案。

过估计与不稳定。 Q-learning 的 max 偏差、离策略+ 函数逼近的发散、PPO 的崩溃，都是训练不稳定的表现。它们往往在论文的“精选种子”中被掩盖，在部署时暴露。

分布偏移。 离线RL 在数据覆盖差的区域高估Q 值；推荐系统在用户兴趣漂移时失效；医疗策略在患者群体变化时失效。

反馈回路。 推荐系统放大已有偏好，形成茧房；金融策略改变市场微观结构，使历史规律失效；交通信号策略改变出行需求分布。

安全与伦理。 RL 策略可能学会欺骗、操纵、规避监督；在医疗、金融、军事等高风险领域，缺乏形式化保证的RL 难以通过认证。

8.12 本章小结

本章的核心结论可以概括为三句话：

1. **RL 已经在“规则明确、奖励可验证、仿真廉价”的领域产生了可复核的超人类效果：**围棋、Atari、星际争霸、赛车、芯片布局、矩阵乘法、排序算法、数据中心冷却、托卡马克控制。这些案例的证据等级高，效应量大，但可迁移性受限于环境与奖励。
2. **在“奖励延迟、反事实不可观测、试错代价高”的领域，RL 更多以离线、约束、人机协同的方式产生增量价值：**推荐、广告、金融、医疗、工业控制。公开的端到端因果证据有限，效应量通常是个位数百分比，且容易被基线与选择偏差放大。
3. **LLM 时代的RLHF/RLVR 是当前最活跃的应用前沿：**它把RL 从“控制”扩展到“对齐与推理”，并创造了新的Scaling 曲线；但它也带来了奖励劫持、评估污染、推理成本与“能力是否真正提升”的争议。

第9 章将把这些应用中的共同挑战提炼为研究议程：样本效率、奖励设计、分布偏移、安全对齐、评估危机、RL Scaling 与“经验时代”。

第 9 章

当前挑战与未来方向：从“能学”到“可信地学”

如果把强化学习的历史看作一部“扩展能力边界”的历史，那么2026 年的主题已经从“能不能学”转向“学得是否可信、是否高效、是否可部署”。本章把第8 章应用中的共同困难提炼为六个挑战，并给出未来五年的研究议程。我们的判断是：下一个突破更可能来自“环境+ 奖励+ 评估”的基础设施，而不是单一损失函数的修改。

9.1 样本效率：数据从哪里来

9.1.1 样本复杂度的现实

表格型MDP 的样本复杂度已经有较完整的理论：在生成模型假设下，找到 ε-最优策略需要 $\tilde{O}(|\mathcal{S}||\mathcal{A}|/(1-\gamma)^3\varepsilon^2)$ 量级的样本；无生成模型时还要乘以探索代价。对深度RL，理论界更弱，但经验规律类似：Atari 需要数千万帧，机器人需要数百万次仿真步，LLM 的RLHF 需要数万到数十万条偏好。

样本效率的瓶颈不在“梯度估计”，而在“环境交互”。三类方法试图突破：

1. **模型基**：用学习到的模型生成虚拟经验，把一次真实交互放大为多次更新；风险是模型偏差。
2. **离线/批量**：用历史数据替代在线交互；风险是分布偏移。
3. **并行仿真**：用数千个环境并行采样，把“墙钟时间”压到分钟级；风险是仿真与现实的差距。

这三条路线在2020 年代逐渐融合：Dreamer 用模型做想象，Isaac Gym 用并行仿真，离线RL 用历史数据，LLM 用可验证题目生成器。未来的“样本效率”不再是单一算法的属性，而是数据生成基础设施的属性。

9.1.2 “经验时代”

Silver 与Sutton 在2025 年提出“经验时代”（era of experience）的论点：过去二十年，AI 的进步主要来自“人类数据”（文本、图像、标注）；未来，AI 的进步将更多来自“智能体自身

与环境交互产生的经验”[79]。这一论点与RL 的历史节奏一致：每当一个新的可交互环境出现（Atari、围棋、Dota、浏览器、代码仓库），RL 就会迎来一波突破。

但“经验时代”有一个前提：环境必须可验证、可复现、可扩展。如果环境是昂贵的真实世界（医疗、金融、工业），经验的数量与质量都受限；如果环境是廉价的仿真（游戏、代码），经验可以无限生成，但可能与现实脱节。未来的竞争，很大程度上是“高质量环境”的竞争。

9.2 奖励设计：代理目标的诅咒

9.2.1 奖励假设与它的边界

强化学习有一个隐含假设，常被称为“奖励假设”（reward hypothesis）：任何目标都可以被形式化为“最大化期望累积奖励”。这个假设在工程上极其有用，因为它把“目标”压缩为一个标量；但它也掩盖了一个事实：**奖励是目标的代理，不是目标本身**。

当代理与目标不一致时，优化器会毫不犹豫地利用代理的漏洞。这就是reward hacking（奖励劫持）。它的数学本质是：

$$\max_{\pi} \mathbb{E}_{\pi}\left[\hat{r}(s,a)\right] \quad \text{与} \quad \max_{\pi} \mathbb{E}_{\pi}\left[r^{\star}(s,a)\right] \tag{9.1}$$

的解不一致，其中 $\hat{r}$ 是可优化的代理奖励，$r^{\star}$ 是真实目标。当 $\hat{r}$ 在策略分布外推时误差变大，优化器会主动把策略推向误差最大的区域。

9.2.2 三类缓解思路

约束优化。 把“不可违反的约束”写成约束，而不是奖励项：

$$\max_{\pi} \mathbb{E}_{\pi}[\hat{r}] \quad \text{s.t.} \quad \mathbb{E}_{\pi}[c_i(s,a)] \le d_i, \quad i = 1, \ldots, m. \tag{9.2}$$

约束RL 的拉格朗日对偶与信赖域方法可以给出可行策略，但约束的“硬”程度取决于估计的准确性。

过程奖励。 不只奖励最终结果，也奖励中间步骤的合理性。过程奖励可以缓解稀疏奖励与信用分配，但它本身也是代理，可能被“看起来合理”的步骤劫持。过程奖励与结果奖励的结合（PRM + ORM）是当前LLM 推理训练的重要方向。

验证器与对抗。 用独立的验证器（单元测试、形式化证明、物理模拟）替代人类偏好；用对抗样本与红队测试寻找验证器漏洞。验证器越“硬”，奖励越可信；但验证器的覆盖范围有限，且可能被“针对测试用例”的策略欺骗。

9.2.3 一个根本性的张力

奖励设计面临一个根本张力：越容易优化的奖励，越容易被劫持；越难劫持的奖励，越难优化。人类偏好容易收集但噪声大；程序验证器准确但覆盖窄；真实世界结果最可信但最昂贵。未来的奖励系统很可能是“多层组合”：结果奖励+ 过程奖励+ 约束+ 人类监督，每一层都有自己的失效模式。

9.3 分布偏移与泛化

9.3.1 三种分布偏移

强化学习中的分布偏移至少有三类：

1. **训练—测试偏移**：在训练任务上表现好，在新任务/新地图/新用户上失效；
2. **策略—数据偏移**：离线数据由行为策略产生，目标策略的占用分布不同；
3. **非平稳性**：环境本身随时间变化（市场、用户、对手），策略需要持续适应。

第一类偏移是泛化问题，第二类是离线RL 的核心，第三类是持续学习/元学习的问题。三者的共同数学工具是“分布鲁棒优化”（DRO）与“因果推断”：我们希望在分布变化下保持性能，或至少知道性能何时会崩溃。

9.3.2 泛化的层次

RL 的泛化可以分层次评估：

- **插值泛化**：在训练分布内的小扰动下保持性能；
- **组合泛化**：把已学的技能组合到新任务；
- **外推泛化**：在训练分布之外的新环境中保持性能；
- **零样本迁移**：不做任何微调，直接部署到新环境。

大多数RL 论文只报告插值泛化；组合泛化与外推泛化仍然是开放问题。LLM 的“上下文学习”提供了一种新的泛化机制：策略可以在推理时根据提示适应新任务，而不更新参数。这与RL 的“参数更新”形成互补，也是Agentic RL 的重要方向。

9.4 长程信用分配

9.4.1 为什么长程难

信用分配的困难随horizon 指数增长：若每个时刻有 $|\mathcal{A}|$ 个动作，长度为 T 的轨迹有 $|\mathcal{A}|^T$ 种可能；要把最终奖励归因到每一步，需要在这棵指数大的树上做反事实推理。TD/GAE 用自

举与指数加权缓解，但它们的偏差随horizon 增长；离线RL 用保守正则缓解，但保守会抑制长期探索。

LLM 的长链推理把这个问题放大：一条思维链可能有数千token，最终答案只给一个奖励。GRPO 用序列级优势，把所有token 共享一个信号；这在小规模任务上有效，但在“哪一步推理错了”需要精细区分时不够。过程奖励模型、步骤级GAE、树搜索（MCTS/beam search）都是应对手段，但都依赖额外的监督或验证。

9.4.2 分层与选项

分层RL（hierarchical RL）用“选项/技能”把长程任务分解为短程子任务：

$$\pi(a \mid s) = \sum_{o} \pi_{\text{high}}(o \mid s)\, \pi_{\text{low}}(a \mid s, o), \tag{9.3}$$

其中 o 是选项（option），π_{high} 决定“做什么”，π_{low} 决定“怎么做”。选项可以是手工定义的子目标，也可以从数据中发现。分层结构的优势是把horizon 缩短，把信用分配分解到不同层级；难点是“选项发现”与“层级间信用分配”。

LLM 智能体天然具有分层结构：高层规划（任务分解）与低层执行（工具调用）。Agentic RL 的一个有希望的方向，是把“规划”与“执行”分开训练，再用结果奖励联合微调。

9.5 安全、对齐与可解释性

9.5.1 约束RL 与安全盾

安全强化学习（safe RL）通常把问题写成约束MDP：

$$\max_{\pi} \; J_r(\pi) \quad \text{s.t.} \quad J_{c_i}(\pi) \le d_i, \tag{9.4}$$

其中 J_{c_i} 是第 i 个约束的期望累积成本。求解方法包括拉格朗日法、约束策略优化（CPO）、以及“安全盾”（shield）：在策略输出动作后，用一个形式化的安全模型过滤掉危险动作。安全盾的优点是提供可证明的约束满足；缺点是需要精确的系统模型，且可能过于保守。

9.5.2 对齐的深层困难

LLM 的对齐问题比“约束满足”更复杂：人类的价值观不一致、随语境变化、且难以形式化。RLHF 把对齐简化为“最大化人类偏好”，但偏好不等于价值；人类可能偏好“听起来对”的答案，而非“真正对”的答案。这就是“谄媚”（sycophancy）与“奖励劫持”的根源。

更深层的问题是“可扩展监督”（scalable oversight）：当模型能力超过人类评估者时，人类如何判断它的输出是否正确？可能的路径包括：AI 辅助评估、形式化验证、辩论（debate）、以及“弱到强”泛化（weak-to-strong generalization）。这些方向都还在早期。

9.5.3 可解释性

RL 策略的可解释性至少有两个层面：*策略层面*（为什么选这个动作）与*训练层面*（为什么学出这个策略）。前者可以用注意力、显著性图、决策树蒸馏等方法；后者需要分析训练数据、奖励函数与优化轨迹。对高风险应用（医疗、金融、自动驾驶），可解释性不是“加分项”，而是“准入条件”。

9.6 评估危机

9.6.1 基准污染与饱和

RL 的评估比监督学习更复杂：策略会改变数据分布，评估本身会影响环境。LLM 时代的评估危机尤为严重：

- **训练—测试重叠**：RLVR 的训练题目可能出现在评测集中；
- **基准饱和**：AIME、MATH 等基准的分数已接近上限，无法区分模型能力；
- **Goodhart 定律**：当指标成为目标，它就不再是好指标；模型会学会“应试”；
- **评估者偏差**：人类评估者可能偏好长答案、格式漂亮的答案，而非正确的答案。

9.6.2 动态评估与“能力边界”

应对评估危机的方向包括：

1. **动态基准**：定期生成新题目、新环境，防止记忆与污染；
2. **多指标评估**：pass@1、pass@k、token 效率、鲁棒性、泛化、可解释性并重；
3. **对抗评估**：用红队、分布外测试、对抗样本寻找失败模式；
4. **因果评估**：用 A/B 测试、工具变量、断点回归等方法估计真实效应，而非只看相关性。

“能力边界”比“平均分数”更重要。一个模型在AIME 上达到90%，但在新题型上只有30%，我们不能说它“数学能力90 分”。评估应该报告分布，而不是单一均值。

9.7 RL Scaling：下一个Scaling Law?

9.7.1 从预训练Scaling 到RL Scaling

过去十年，LLM 的进步主要由“预训练Scaling Law”驱动：参数量、数据量、算力越大，损失越低。RL 是否也有类似的Scaling Law？目前的证据是混合的：

- **环境Scaling**：更多并行环境、更多样化的任务，通常能提升泛化；
- **采样Scaling**：更多rollout、更多候选答案，可以提升pass@k；

- **模型Scaling**：更大的模型在相同RL 算法下通常更好，但样本效率未必更高；
- **奖励Scaling**：更准确的奖励模型/验证器，比更多采样更重要。

与预训练不同，RL 的Scaling 不是“堆算力”那么简单：环境的质量、奖励的可验证性、探索的策略，都会改变Scaling 曲线的斜率。一个在低质量环境上训练的模型，可能越训越差（reward hacking）；一个在高质量环境上训练的模型，可能获得远超预训练的能力。

9.7.2 测试时Scaling

2024—2025 年的另一个重要发现是“测试时计算”的Scaling：在推理阶段采样多条轨迹、用验证器/搜索选择最优答案，可以显著提升性能。Snell 等人证明，在固定计算预算下，“更长的采样”与“更好的验证器”可以优于单纯增大模型[60]。这与AlphaGo 的“学习+ 搜索”结构同构：训练时学习先验，推理时用搜索修正。

测试时Scaling 的一个重要含义是：*训练与推理的边界正在模糊*。RL 不再只是“训练阶段”的算法，也可以发生在推理阶段（test-time RL、self-improvement）。这对系统设计提出了新要求：推理引擎需要支持树搜索、验证器调用、多候选管理。

9.8 未来五年的研究议程

基于以上分析，我们提出六个研究方向。它们不是互斥的，而是相互依赖的。

1. **可交互环境的基础设施**：构建可复现、可并行、可验证、多样化的环境（浏览器、操作系统、代码仓库、科学实验、机器人仿真）。环境的“覆盖率”与“真实性”将决定RL 的上限。
2. **可验证与过程奖励**：研究“结果奖励+ 过程奖励+ 约束”的组合，发展抗劫持的验证器、对抗训练与奖励集成。奖励的“可信度”比“精度”更重要。
3. **世界模型与想象**：学习“足够用于决策”的隐式模型，量化模型偏差，发展不确定性感知的规划。Dreamer/MuZero 的路线仍有巨大空间。
4. **长程信用分配**：结合分层RL、步骤级GAE、过程奖励、树搜索，解决LLM 智能体与机器人长程任务的归因问题。
5. **安全与对齐**：把约束、可解释性、可扩展监督纳入RL 训练循环，发展“训练时安全”而非“事后过滤”的方法。
6. **理论与评估**：证明非线性函数逼近下TD/策略梯度的收敛性（或在反例上划清边界）；建立“能力边界”的评估框架，区分基准分数与真实能力。

9.9 结语：强化学习的“第二曲线”

强化学习的第一条曲线，是“在已知或可仿真的环境中寻找最优策略”。这条曲线从Bellman 方程出发，经过TD、Q-learning、DQN、AlphaGo、PPO，走到了游戏、机器人、芯片、数据中心的成功。它的极限由环境保真度与奖励可信度决定。

强化学习的第二条曲线，是“在开放、部分可观测、奖励难以定义的世界中，通过经验持续改进”。这条曲线从RLHF 开始，经过DPO、GRPO、RLVR，走到了LLM 推理与智能体。它的极限不再由算法公式决定，而由环境、奖励与评估三者共同决定。

两条曲线共享同一个数学骨架：贝尔曼方程、策略梯度、信赖域、KL 正则。变的只是“状态”“动作”“奖励”的载体。理解这个骨架，比记住任何一个算法都重要；因为当下的SOTA会过时，而骨架不会。

本报告的三个核心判断

1. **算法史是“表示—数据—更新—目标”四组件的协同演化史**，每一次范式转移都解决了前一代的一个具体失效模式，而不是对前一代的否定。
2. **RL 的效果取决于“奖励可信度Œ 环境保真度Œ 评估可识别性”**；在规则明确、奖励可验证的领域，RL 已经产生超人类效果；在奖励延迟、反事实不可观测的领域，RL 更多是增量优化工具。
3. **2026 年的瓶颈是基础设施，不是公式**：可交互环境、可验证奖励、可信评估，将决定下一波突破的速度与方向。

第 A 章

符号表与常用恒等式

A.1 符号表

符号	含义	首次出现
$\mathcal{S}, \mathcal{A}$	状态空间、动作空间	定义 2.1
$P(s' \mid s, a)$	转移概率	定义 2.1
$r(s, a)$	即时奖励	定义 2.1
γ	折扣因子，$\gamma \in [0, 1)$	定义 2.1
ρ_0	初始状态分布	定义 2.1
$\pi(a \mid s)$	随机平稳策略	定义 2.2
$\mu(s)$	确定性策略	定义 2.2
G_t	从 t 开始的折扣回报	定义 2.3
$V^\pi(s)$	策略 π 的状态值函数	定义 2.3
$Q^\pi(s, a)$	策略 π 的动作值函数	定义 2.3
$A^\pi(s, a)$	优势函数 $Q^\pi - V^\pi$	定义 2.3
$d^\pi(s), d^\pi(s, a)$	归一化折扣占用度量	定义 2.4
$\mathcal{T}^\pi, \mathcal{T}^*$	贝尔曼期望算子、最优算子	定义 2.6
π^*, V^*, Q^*	最优策略、最优值函数	定义 2.5
$\|\cdot\|_\infty$	无穷范数 $\max_s \|\cdot\|$	引理 2.1
δ_t	TD 误差	式 (4.2)
$E_t(s)$	资格迹	式 (4.15)
θ, ϕ, ψ	策略、奖励模型、值函数参数	第 4—7 章
$\rho_t(\theta)$	概率比 $\pi_\theta / \pi_{\theta_{\text{old}}}$	式 (5.45)
$D_{\text{KL}}(\cdot \| \cdot)$	Kullback–Leibler 散度	式 (7.1)
$\sigma(\cdot)$	Sigmoid 函数	式 (7.9)

符号	含义	首次出现
β	KL 正则系数/温度倒数	式(7.1)
α	学习率或熵温度	第4、5 章
λ	TD/GAE 的偏差—方差参数	式(5.26)
ϵ	PPO 裁剪范围或探索率	式(5.46)
$\mathcal{D}$	经验回放缓冲区或数据集	第4、6 章
$\mathcal{B}^\pi$	贝尔曼备份算子	式(6.23)

A.2 常用恒等式

$$\mathbb{E}\left[\nabla_\theta \log \pi_\theta(a \mid s)\right] = 0, \tag{A.1}$$

$$\mathbb{E}\left[b(s)\nabla_\theta \log \pi_\theta(a \mid s)\right] = 0, \quad \forall b, \tag{A.2}$$

$$\mathbb{E}\left[G_t \mid S_t = s, A_t = a\right] = Q^\pi(s,a), \tag{A.3}$$

$$\mathbb{E}\left[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s, A_t = a\right] = Q^\pi(s,a), \tag{A.4}$$

$$\mathbb{E}\left[R_{t+1} + \gamma V^\pi(S_{t+1}) - V^\pi(S_t) \mid S_t = s, A_t = a\right] = A^\pi(s,a), \tag{A.5}$$

$$\sum_a \pi(a \mid s) = 1, \qquad \sum_{s'} P(s' \mid s,a) = 1. \tag{A.6}$$

A.3 两个常用的不等式

Jensen 不等式。 若 f 是凸函数，则

$$f\left(\mathbb{E}[X]\right) \le \mathbb{E}\left[f(X)\right]. \tag{A.7}$$

取 $f(x) = \max_a x_a$（凸函数），得到

$$\max_a \mathbb{E}[X_a] \le \mathbb{E}\left[\max_a X_a\right], \tag{A.8}$$

这是Q-learning 最大化偏差的来源。

Pinsker 不等式。 对任意两个分布 p, q，

$$D_{\mathrm{TV}}(p,q) \le \sqrt{\frac{1}{2} D_{\mathrm{KL}}(p \,\|\, q)}. \tag{A.9}$$

它把全变差距离与KL 散度联系起来，是TRPO 下界证明的关键工具。

A.4 常用概率恒等式

$$\mathbb{E}[X] = \int x\, p(x)\, \mathrm{d}x, \tag{A.10}$$

$$\operatorname{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2, \tag{A.11}$$

$$\mathbb{E}\left[\mathbb{E}[X \mid Y]\right] = \mathbb{E}[X], \tag{A.12}$$

$$\operatorname{Var}(X) = \mathbb{E}\left[\operatorname{Var}(X \mid Y)\right] + \operatorname{Var}\left(\mathbb{E}[X \mid Y]\right). \tag{A.13}$$

最后一式称为全方差公式，是GAE 偏差—方差分析的基础。

第 B 章

算法速查表与研究复现清单

B.1 算法速查

表 B.1　核心算法的更新式与关键超参数

算法	核心更新	关键超参数与备注
TD(0)	$V(S_t) \leftarrow V(S_t) + \alpha\delta_t$，$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$	学习率 α；需要 $\sum \alpha_t = \infty, \sum \alpha_t^2 < \infty$
TD(λ)	$E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbf{1}\{S_t = s\}$；$V(s) \leftarrow V(s) + \alpha\delta_t E_t(s)$	$\lambda \in [0,1]$；$\lambda = 1$ 为蒙特卡洛
SARSA	$Q(S_t, A_t) \leftarrow Q + \alpha[R + \gamma Q(S_{t+1}, A_{t+1}) - Q]$	在策略；ε-贪心需GLIE
Q-learning	$Q(S_t, A_t) \leftarrow Q + \alpha[R + \gamma \max_{a'} Q(S_{t+1}, a') - Q]$	离策略；需覆盖所有状态—动作对
Double Q	用 Q_1 选动作、Q_2 评估，交替更新	缓解最大化偏差
DQN	$L = \mathbb{E}[(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a))^2]$	回放容量、目标网络更新周期、ε 衰减
Double DQN	$y = r + \gamma Q_{\theta^-}(s', \arg\max_{a'} Q_\theta(s', a'))$	几乎无额外成本
PPO	$L = \mathbb{E}[\min(\rho A, \mathrm{clip}(\rho, 1-\epsilon, 1+\epsilon)A)]$	$\epsilon \approx 0.2$；GAE $\lambda \approx 0.95$；优势归一化
TRPO	$\max L$ s.t. $\mathbb{E}[D_{\mathrm{KL}}] \le \delta$	共轭梯度、线搜索；$\delta \approx 0.01$
DDPG	$y = r + \gamma Q_{\theta^-}(s', \mu_{\theta^-}(s'))$；actor 最大化 Q	软更新 τ；动作噪声
TD3	双Q 取min；目标动作加截断噪声；延迟actor 更新	缓解过估计；$c \approx 0.5$

算法	核心更新	关键超参数与备注
SAC	最大熵目标；软贝尔曼；双Q	温度 α 可自动调节；目标熵
CQL	加 $\alpha(\log\sum_a e^Q - \mathbb{E}_D Q)$ 正则	α 控制保守程度
IQL	expectile 回归 V，避免查询 OOD 动作	$\tau \approx 0.7$—0.9；AWR 温度 β
DPO	$-\log\sigma(\beta\log\frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \beta\log\frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)})$	$\beta \approx 0.1$；偏好对质量关键
GRPO	组内优势 $\hat{A}_i = (r_i - \bar{r})/\text{std}(r)$；PPO 裁剪	组大小 G；无需critic；结果奖励

B.2 研究复现清单

阅读或复现一篇RL 论文时，建议逐项核对：

1. **环境**：版本、随机种子、观测/动作空间、最大回合长度、是否使用仿真器状态；
2. **奖励**：定义、尺度、稀疏性、是否有reward shaping、是否有终止条件；
3. **算法**：目标函数、梯度估计、目标网络/回放、探索策略、批次大小；
4. **优化**：学习率、优化器、梯度裁剪、训练步数、并行环境数；
5. **评估**：评估频率、评估回合数、是否用固定种子、是否报告均值与置信区间；
6. **基线**：是否包含强基线、是否调参公平、是否报告计算成本；
7. **消融**：每个组件的贡献、超参数敏感性、失败案例；
8. **可复现性**：代码、超参数、随机种子、硬件、运行时间。

B.3 常见超参数的经验范围

超参数	常用范围	备注
折扣因子 γ	0.99（控制）、0.9—0.99（LLM 常取 1）	越大越重视长期；LLM 常用回合制无折扣
GAE λ	0.9—0.97	越大方差越大、偏差越小
PPO 裁剪 ϵ	0.1—0.3	越小越保守
学习率	3×10^{-4}（RL）、10^{-6}—10^{-5}（LLM RL）	取决于模型规模与批量
批次大小	64—4096（控制）、32—512（LLM）	与并行环境数、序列长度相关
回放容量	10^5—10^6	太大会引入过时数据
目标网络 τ	0.001—0.01（软更新）	或每 C 步硬更新
熵系数 α	0.01—0.2（SAC 可自动）	越大探索越强
KL 系数 β	0.01—0.1（RLHF）	越大越贴近参考策略
DPO β	0.1—0.5	越大越保守

B.4 术语中英对照

中文	英文	中文	英文
马尔可夫决策过程	Markov decision process	贝尔曼方程	Bellman equation
值函数	value function	动作值函数	action-value function
优势函数	advantage function	占用度量	occupancy measure
时序差分	temporal difference	资格迹	eligibility trace
在策略	on-policy	离策略	off-policy
策略梯度	policy gradient	自然梯度	natural gradient
信赖域	trust region	裁剪	clipping
经验回放	experience replay	目标网络	target network
奖励劫持	reward hacking	可验证奖励	verifiable reward
人类反馈强化学习	RLHF	直接偏好优化	DPO
组相对策略优化	GRPO	智能体强化学习	agentic RL

参考文献

[1] R. Bellman. A Markovian decision process[J]. *Journal of Mathematics and Mechanics*, 1957, 6(5): 679–684.

[2] E. L. Thorndike. *Animal Intelligence: Experimental Studies*[M]. New York: Macmillan, 1911.

[3] W. Schultz, P. Dayan, P. R. Montague. A neural substrate of prediction and reward[J]. *Science*, 1997, 275(5306): 1593–1599.

[4] P. R. Montague, P. Dayan, T. J. Sejnowski. A framework for mesencephalic dopamine systems based on predictive Hebbian learning[J]. *Journal of Neuroscience*, 1996, 16(5): 1936–1947.

[5] M. L. Minsky. *Theory of Neural-Analog Reinforcement Systems and its Application to the Brain-Model Problem*[D]. Princeton: Princeton University, 1954.

[6] A. L. Samuel. Some studies in machine learning using the game of checkers[J]. *IBM Journal of Research and Development*, 1959, 3(3): 210–229.

[7] A. G. Barto, R. S. Sutton, C. W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems[J]. *IEEE Transactions on Systems, Man, and Cybernetics*, 1983, SMC-13(5): 834–846.

[8] R. S. Sutton. Learning to predict by the methods of temporal differences[J]. *Machine Learning*, 1988, 3(1): 9–44.

[9] C. J. C. H. Watkins. *Learning from Delayed Rewards*[D]. Cambridge: University of Cambridge, 1989.

[10] C. J. C. H. Watkins, P. Dayan. Q-learning[J]. *Machine Learning*, 1992, 8(3–4): 279–292.

[11] R. S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming[C]//*Proceedings of the 7th International Conference on Machine Learning (ICML)*. 1990: 216–224.

[12] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning[J]. *Machine Learning*, 1992, 8(3–4): 229–256.

[13] G. Tesauro. Temporal difference learning and TD-Gammon[J]. *Communications of the ACM*, 1995, 38(3): 58–68.

[14] R. S. Sutton, D. Precup, S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning[J]. *Artificial Intelligence*, 1999, 112(1–2): 181–211.

[15] R. S. Sutton, D. McAllester, S. Singh, Y. Mansour. Policy gradient methods for reinforcement learning with function approximation[C]//*Advances in Neural Information Processing Systems (NeurIPS)*. 2000: 1057–1063.

[16] S. Kakade. A natural policy gradient[C]//*Advances in Neural Information Processing Systems (NeurIPS)*. 2001: 1531–1538.

[17] P. Auer, N. Cesa-Bianchi, P. Fischer. Finite-time analysis of the multiarmed bandit problem[J]. *Machine Learning*, 2002, 47(2–3): 235–256.

[18] L. Kocsis, C. Szepesvári. Bandit based Monte-Carlo planning[C]//*European Conference on Machine Learning (ECML)*. LNCS 4212, 2006: 282–293.

[19] L. Baird. Residual algorithms: Reinforcement learning with function approximation[C]//*Proceedings of the 12th International Conference on Machine Learning (ICML)*. 1995: 30–37.

[20] J. N. Tsitsiklis. Asynchronous stochastic approximation and Q-learning[J]. *Machine Learning*, 1994, 16(3): 185–202.

[21] J. N. Tsitsiklis, B. Van Roy. An analysis of temporal-difference learning with function approximation[J]. *IEEE Transactions on Automatic Control*, 1997, 42(5): 674–690.

[22] R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction*[M]. 2nd ed. Cambridge, MA: MIT Press, 2018.

[23] V. Mnih, K. Kavukcuoglu, D. Silver, et al. Playing Atari with deep reinforcement learning[R]. arXiv:1312.5602, 2013.

[24] V. Mnih, K. Kavukcuoglu, D. Silver, et al. Human-level control through deep reinforcement learning[J]. *Nature*, 2015, 518(7540): 529–533.

[25] V. Mnih, A. P. Badia, M. Mirza, et al. Asynchronous methods for deep reinforcement learning[C]//*Proceedings of the 33rd International Conference on Machine Learning (ICML)*. 2016: 1928–1937.

[26] J. Schulman, S. Levine, P. Abbeel, et al. Trust region policy optimization[C]//*Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015: 1889–1897.

[27] J. Schulman, F. Wolski, P. Dhariwal, et al. Proximal policy optimization algorithms[R]. arXiv:1707.06347, 2017.

[28] T. P. Lillicrap, J. J. Hunt, A. Pritzel, et al. Continuous control with deep reinforcement learning[C]//*International Conference on Learning Representations (ICLR)*. 2016.

[29] D. Silver, G. Lever, N. Heess, et al. Deterministic policy gradient algorithms[C]//*Proceedings of the 31st International Conference on Machine Learning (ICML)*. 2014: 387–395.

[30] S. Fujimoto, H. van Hoof, D. Meger. Addressing function approximation error in actor-critic methods[C]//*Proceedings of the 35th International Conference on Machine Learning (ICML)*. 2018: 1587–1596.

[31] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor[C]//*Proceedings of the 35th International Conference on Machine Learning (ICML)*. 2018: 1861–1870.

[32] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. Mastering diverse domains through world models[R]. arXiv:2301.04104, 2023.

[33] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. Mastering diverse control tasks through world models[J]. *Nature*, 2025, 640: 647–653.

[34] J. Schrittwieser, I. Antonoglou, T. Hubert, et al. Mastering Atari, Go, chess and shogi by planning with a learned model[J]. *Nature*, 2020, 588(7839): 604–609.

[35] D. Silver, A. Huang, C. J. Maddison, et al. Mastering the game of Go with deep neural networks and tree search[J]. *Nature*, 2016, 529(7587): 484–489.

[36] D. Silver, J. Schrittwieser, K. Simonyan, et al. Mastering the game of Go without human knowledge[J]. *Nature*, 2017, 550(7676): 354–359.

[37] D. Silver, T. Hubert, K. Simonyan, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play[J]. *Science*, 2018, 362(6419): 1140–1144.

[38] O. Vinyals, I. Babuschkin, W. M. Czarnecki, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning[J]. *Nature*, 2019, 575(7782): 350–354.

[39] OpenAI. OpenAI Five defeats Dota 2 world champions[EB/OL]. 2019. https://openai.com/index/openai-five-defeats-dota-2-world-champions/.

[40] P. R. Wurman, S. Barrett, K. Kawamoto, et al. Outracing champion Gran Turismo drivers with deep reinforcement learning[J]. *Nature*, 2022, 602(7896): 223–228.

[41] A. Fawzi, M. Balog, A. Huang, et al. Discovering faster matrix multiplication algorithms with reinforcement learning[J]. *Nature*, 2022, 610(7930): 47–53.

[42] D. J. Mankowitz, A. Michalewski, A. Zhernov, et al. Faster sorting algorithms discovered using deep reinforcement learning[J]. *Nature*, 2023, 618(7964): 99–104.

[43] A. Mirhoseini, A. Goldie, M. Yazgan, et al. A graph placement methodology for fast chip design[J]. *Nature*, 2021, 594(7862): 207–212.

[44] Google DeepMind. AlphaChip: A reinforcement learning method for chip placement[EB/OL]. 2024. https://deepmind.google/discover/blog/alphachip-a-reinforcement-learning-method-for-chip-placement/.

[45] S. Fujimoto, D. Meger, D. Precup. Off-policy deep reinforcement learning without exploration[C]//*Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019: 2052–2061.

[46] A. Kumar, A. Zhou, G. Tucker, S. Levine. Conservative Q-learning for offline reinforcement learning[C]//*Advances in Neural Information Processing Systems (NeurIPS)*. 2020: 1179–1191.

[47] I. Kostrikov, A. Nair, S. Levine. Offline reinforcement learning with implicit Q-learning[R]. arXiv:2110.06169, 2021.

[48] P. Christiano, J. Leike, T. Brown, et al. Deep reinforcement learning from human preferences[C]//*Advances in Neural Information Processing Systems (NeurIPS)*. 2017: 4299–4307.

[49] N. Stiennon, L. Ouyang, J. Wu, et al. Learning to summarize with human feedback[C]//*Advances in Neural Information Processing Systems (NeurIPS)*. 2020: 3008–3021.

[50] L. Ouyang, J. Wu, X. Jiang, et al. Training language models to follow instructions with human feedback[C]//*Advances in Neural Information Processing Systems (NeurIPS)*. 2022: 27730–27744.

[51] R. Rafailov, A. Sharma, E. Mitchell, et al. Direct preference optimization: Your language model is secretly a reward model[C]//*Advances in Neural Information Processing Systems (NeurIPS)*. 2023.

[52] Z. Shao, P. Wang, Q. Zhu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models[R]. arXiv:2402.03300, 2024.

[53] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning[R]. arXiv:2501.12948, 2025.

[54] DeepSeek-AI. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning[J]. *Nature*, 2025, 645: 633–638.

[55] G. Zhang, H. Geng, X. Yu, et al. The landscape of agentic reinforcement learning for LLMs: A survey[R]. arXiv:2509.02547, 2025.

[56] Y. Liu, et al. StepPO: Step-aligned policy optimization for agentic reinforcement learning[R]. arXiv:2604.18401, 2026.

[57] Anonymous. Curriculum reinforcement learning can incentivize reasoning capacity in LLMs beyond the base model[R]. arXiv:2606.22317, 2026.

[58] Anonymous. Ring-Zero: Scaling zero RL to a trillion parameters for emergent reasoning[R]. arXiv:2607.12395, 2026.

[59] Y. Yue, et al. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?[R]. arXiv:2504.13837, 2025.

[60] C. Snell, J. Lee, K. Xu, A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters[R]. arXiv:2408.03314, 2024.

[61] OpenAI. OpenAI o3 and o3-mini[EB/OL]. 2024. https://openai.com/index/openai-o3-mini/.

[62] Google DeepMind. AI achieves silver-medal standard solving International Mathematical Olympiad problems[EB/OL]. 2024. Google DeepMind blog, 2024.

[63] Google DeepMind. Advanced version of Gemini with Deep Think officially achieves gold-medal standard at the International Mathematical Olympiad[EB/OL]. 2025. Google DeepMind blog, 2025.

[64] R. Evans, J. Gao. DeepMind AI reduces Google data centre cooling bill by 40%[EB/OL]. 2016. https://deepmind.google/discover/blog/deepmind-ai-reduces-google-data-centre-cooling-bill-by-40/.

[65] H. Mao, M. Alizadeh, I. Menache, S. Kandula. Resource management with deep reinforcement learning[C]//*Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*. 2016: 50–56.

[66] H. Mao, R. Netravali, M. Alizadeh. Neural adaptive video streaming with Pensieve[C]//*Proceedings of the ACM SIGCOMM Conference*. 2017: 197–210.

[67] K. Winstein, H. Balakrishnan. TCP ex machina: Computer-generated congestion control[C]//*Proceedings of the ACM SIGCOMM Conference*. 2013: 123–134.

[68] H. Wei, G. Zheng, V. Gayah, L. Li. IntelliLight: A reinforcement learning approach for intelligent traffic light control[C]//*Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 2018: 2496–2505.

[69] J. Degrave, F. Felici, J. Buchli, et al. Magnetic control of tokamak plasmas through deep reinforcement learning[J]. *Nature*, 2022, 602(7897): 414–419.

[70] M. Komorowski, L. A. Celi, O. Badawi, et al. The artificial intelligence clinician learns optimal treatment strategies for sepsis in intensive care[J]. *Nature Medicine*, 2018, 24(11): 1716–1720.

[71] J. Lee, J. Hwangbo, L. Wellhausen, et al. Learning quadrupedal locomotion over challenging terrain[J]. *Science Robotics*, 2020, 5(47): eabc5986.

[72] N. Rudin, L. Hoeller, P. Reist, M. Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning[C]//*Conference on Robot Learning (CoRL)*. PMLR 164, 2021: 91–100.

[73] OpenAI, M. Andrychowicz, B. Baker, et al. Solving Rubik's Cube with a robot hand[R]. arXiv:1910.07113, 2019.

[74] A. Brohan, N. Brown, J. Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control[R]. arXiv:2307.15818, 2023.

[75] K. Black, N. Brown, D. Driess, et al. π_0: A vision-language-action flow model for general robot control[R]. arXiv:2410.24164, 2024.

[76] E. Ie, V. Jain, J. Wang, et al. SlateQ: A tractable decomposition for reinforcement learning with recommendation sets[C]//*Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*. 2019: 2592–2599.

[77] X. Zhao, L. Xia, L. Zhang, et al. Deep reinforcement learning for page-wise recommendations[C]//*Proceedings of the 12th ACM Conference on Recommender Systems (RecSys)*. 2018: 95–103.

[78] M. Chen, A. Beutel, P. Covington, et al. Top-K off-policy correction for a REINFORCE recommender system[C]//*Proceedings of the 12th ACM International Conference on Web Search and Data Mining (WSDM)*. 2019: 456–464.

[79] D. Silver, R. S. Sutton. Welcome to the era of experience[EB/OL]. 2025. DeepMind, 2025.